STUDY TEXTBOOK

Machine learning

From data matrices to deployed models

First edition · 10 October 2026

Contents

CHAPTER 1

Data as matrices: NumPy and the dataset

Every model in this book begins the same way: a table of numbers. Rows are things that happened — a penguin measured, a transaction approved, a pixel grid flattened — and columns are the measurements we chose to record. Before we can fit anything, we have to get that table out of a file, into memory, and into a shape a computer can multiply. That is the whole job of this chapter. You will learn to see a dataset as one matrix $X \in \mathbb{R}^{n \times d}$, to load it from a CSV with nothing but the standard library and NumPy, to encode categories as columns, to shuffle and slice it into training pieces, and to do all of that fast enough that you never think about it again. More importantly, you will learn where it goes wrong: the missing cells, the impossible values, and the duplicated rows that quietly inflate your results. Nothing here is glamorous. All of it is load-bearing.

1.1 Why datasets are matrices (and when they are not)

A *dataset* is a collection of n samples, each described by the same d measurements. Stacked into a table, the samples form the *design matrix* $X \in \mathbb{R}^{n \times d}$, one row per sample: row i is the transposed vector $\mathbf{x}_i^T$, and column j is *feature* j — one number measured for every sample. When the sample also carries an answer we want to predict, it lives in a separate vector $\mathbf{y}$ of *targets*, aligned row-by-row with X. Together $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$.

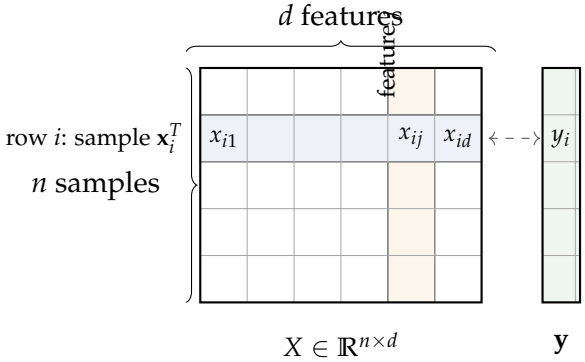


Figure 1.1: A dataset is a matrix. Row i of the design matrix X is one sample $\mathbf{x}_i^T$; column j is one feature measured for every sample. The target vector $\mathbf{y}$ is aligned row-by-row: y_i belongs to $\mathbf{x}_i$.

This is a convention, not a law of nature, and it is a productive one. Once a dataset is a matrix, *scoring the whole dataset* is one matrix product $X\mathbf{w}$; *centring every column* is one subtraction;

finding the nearest class mean is a broadcast and an arg min. The linear algebra you built in Book 3 stops being notation and becomes executable.

When the convention breaks, it breaks for a reason. Text arrives as variable-length sequences, not rows of equal width — the sequence models of chapter 10 inherit that problem. Images are grids, so a picture becomes a (C, H, W) tensor and flattening it into a row throws away the spatial structure a convolution needs. Networks of interacting items (users, molecules, flights) are graphs, not tables. None of that fits this chapter — and none of it is needed yet. Tabular data is the workhorse: surveys, sensor logs, prices, medical measurements. Learn the matrix first.

THE MATRIX IS THE CONTRACT

Every algorithm in this book expects the same deal: a 2-D `ndarray` of shape (n, d) whose rows are samples and columns are features, plus a 1-D target vector of shape $(n,)$ when the task is supervised. When a method misbehaves, check the contract first — wrong shapes cause more bugs than wrong maths.

1.2 NumPy arrays: shape, dtype, broadcasting

NumPy's `ndarray` is a typed, rectangular block of memory. Three attributes describe any array: `shape` (a tuple of axis lengths, e.g. `(150, 4)`), `dtype` (the element type, e.g. `float64`), and `ndim` (how many axes). Two habits pay off immediately:

- **Say the shape aloud.** `X.shape == (150, 4)` means 150 samples, 4 features. The vast majority of array bugs are shape confusions, and they are loud if you listen.
- **Watch the dtype.** An `ndarray` is homogeneous: every element has the same type. Reading mixed text columns through `np.array` without care gives you `<U32` strings; writing a float into an `int` array silently truncates it (§**??**).

The feature that carries the whole book is *broadcasting*: NumPy's rules for applying a binary operation ($+$, $-$, $*$, `/`, comparisons) to arrays of different shapes. Three rules, applied axis-by-axis from the right [**?**]:

BROADCASTING, THREE RULES

1. Align the two shapes on their *trailing* (rightmost) axes; missing axes are treated as size 1.
2. Two dimensions are compatible if they are equal, or if one of them is 1.
3. A dimension of size 1 is *stretched* (conceptually copied) along that axis. If any pair of aligned dimensions fails rule 2, NumPy raises `ValueError: operands could not be broadcast together`.

Read rule 3 as "stretch, don't copy": no data is duplicated in memory, which is why broadcasting is both fast and memory-cheap — and also why a mistaken stretch is silent. The two traps that matter most in this book:

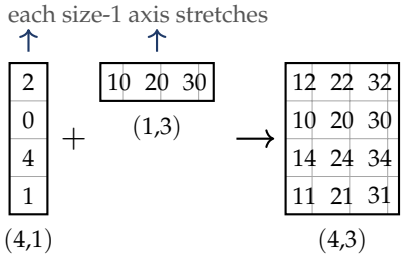


Figure 1.2: Broadcasting a $(4,1)$ column and a $(1,3)$ row: both size-1 dimensions stretch until the shapes agree, and the sum is $(4,3)$ — an *outer* operation built from two inner-shaped pieces. The same mechanism centres columns (`X - mu`), scales them (`(X - mu) / sd`), and builds pairwise distance tables (`A[:, None, :] - B[None, :, :]`).

- $(n,1)+(1,d)\to(n,d)$ is *intended* outer behaviour (Figure **??**); $(n,1)+(n,)\to(n,n)$ is usually an accident — a length-n row and a length-n column silently combine into a full $n\times n$ table. Fix by making the intended axis explicit: `b[:, None]`.
- `X[:, None, :] - B[None, :, :]` inserts axes to build an (n,m,d) intermediate — correct, but memory-hungry for large $n\cdot m$ (§**??**).

Checkpoint 1.1 *Shapes by rules.* Apply the three rules on paper before running anything. What is `np.ones((3,4)) + np.arange(4)`? And `np.ones((3,4)) + np.arange(3)`? Which one fails and why?

1.3 Loading data: CSV → ndarray

A CSV is text: one row per line, fields separated by commas — when you are lucky. The standard library's `csv` module handles the unlucky cases (quoted commas, blank cells); NumPy handles the rest. The pattern we will use all book is *parse to strings first, convert columns second*: `csv.reader` gives a list of string rows; the feature columns become a float `ndarray`; the label column stays strings.

```
# code/ch01/load_iris_scratch.py -- CSV -> ndarray with the csv module (no pandas).
# Run from the repository root: python3 code/ch01/load_iris_scratch.py
import csv

def load_csv_xy(path, target_col):
    """Return (X, y, feature_names): X float ndarray, y string ndarray."""
    import numpy as np
    with open(path, newline="") as f:
        rows = list(csv.reader(f))
    header, data = rows[0], rows[1:]
    j = header.index(target_col)
    feature_names = [h for i, h in enumerate(header) if i != j]
    X = np.array([[float(v) for i, v in enumerate(r) if i != j] for r in data])
    y = np.array([r[j] for r in data])
```

```
    return X, y, feature_names

X, y, names = load_csv_xy("data/iris.csv", "species")
print("feature names:", names)
print("X.shape =", X.shape, " X.dtype =", X.dtype)
print("y.shape =", y.shape, " classes:", sorted(set(y.tolist())))

# --- distance-to-mean classifier on a seeded 80/20 split ---
import numpy as np
rng = np.random.default_rng(7)
perm = rng.permutation(len(y))
n_train = int(0.8 * len(y))
tr, te = perm[:n_train], perm[n_train:]
classes = np.unique(y)
cent = np.stack([X[tr][y[tr] == c].mean(axis=0) for c in classes])
pred = classes[((X[te][:, None, :] - cent[None, :, :]) ** 2).sum(-1).
    argmin(1)]
acc = (pred == y[te]).mean()
print(f"split: {len(tr)} train / {len(te)} test (seed 7)")
print(f"distance-to-mean test accuracy: {acc:.3f} ({int((pred == y[te
    ]).sum())}/{len(te)})")
```

Listing 1.1: `code/ch01/load_iris_scratch.py`: CSV to $(X, \mathbf{y})$ with the standard library — no pandas — followed by a seeded 80/20 split and a distance-to-mean classifier whose core is exactly eight lines.

Run from the repository root, it prints:

```
feature names: ['sepal_length', 'sepal_width', 'petal_length', '
    petal_width']
X.shape = (150, 4) X.dtype = float64
y.shape = (150,) classes: ['setosa', 'versicolor', 'virginica']
split: 120 train / 30 test (seed 7)
distance-to-mean test accuracy: 0.967 (29/30)
```

Example 1.1 (A classifier in eight lines)**.** Strategy: *nearest class mean*. Compute each class's centroid on the training rows, then assign each test row to its closest centroid.

Solution. Lines 24–31 of Listing **??** do exactly that. The only mathematics is a mean and a distance; everything else is indexing. On the seeded split it reaches 0.967 test accuracy (29/30) — a real number from a real run, on Ronald Fisher's 1936 iris measurements [**?**]. The point is not the accuracy; it is that "a model" is just shapes: X $(150, 4)$, centroids $(3, 4)$, predictions $(30,)$.

1.3.1 Missing values and friends

Real files lie by omission. The `penguins` dataset — 344 Antarctic penguins measured at Palmer Station [**?**, **?**] — ships with 19 empty cells: two penguins lack all four bill/flipper/mass measurements, and nine more lack a recorded `sex`. Loading through `float(...)` or `np.loadtxt` turns an empty field into an error or a silent `nan`; either way, **your first act after loading is counting what is missing**.

```python
# code/ch01/missing_values.py -- find NaNs, then measure what each fix costs.
# Run from the repository root: python3 code/ch01/missing_values.py
import csv
import numpy as np

NUMERIC = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]

with open("data/penguins.csv", newline="") as f:
    rows = list(csv.reader(f))
header, data = rows[0], rows[1:]
ji = {h: i for i, h in enumerate(header)}

print("missing cells per column:")
for h in header:
    n_miss = sum(1 for r in data if r[ji[h]] == "")
    if n_miss:
        print(f" {h:<18}{n_miss}")

X = np.array([[float(r[ji[c]]) if r[ji[c]] != "" else np.nan for c in NUMERIC]
              for r in data])
bad_rows = np.isnan(X).any(axis=1)
print(f"\nrows with >=1 missing numeric: {bad_rows.sum()} of {len(X)}")

# strategy 1: drop the rows
X_drop = X[~bad_rows]
# strategy 2: fill each column with its observed median
med = np.nanmedian(X, axis=0)
X_fill = np.where(bad_rows[:, None], med[None, :], X)

print("column means drop-rows vs fill-median:")
for j, h in enumerate(NUMERIC):
    print(f" {h:<18}{X_drop[:, j].mean():8.2f} {X_fill[:, j].mean():8.2f}")
print("corr(bill_length, body_mass) drop / fill:",
      f"{np.corrcoef(X_drop[:, 0], X_drop[:, 3])[0, 1]:.3f} /",
      f"{np.corrcoef(X_fill[:, 0], X_fill[:, 3])[0, 1]:.3f}")
```

Listing 1.2: `code/ch01/missing_values.py`: count NaNs per column, then measure what each repair costs.

```
missing cells per column:
 bill_length_mm 2
 bill_depth_mm 2
 flipper_length_mm 2
 body_mass_g 2
 sex 11

rows with >=1 missing numeric: 2 of 344
```

```
column means drop-rows vs fill-median:
  bill_length_mm 43.92 43.93
  bill_depth_mm 17.15 17.15
  flipper_length_mm 200.92 200.89
  body_mass_g 4201.75 4200.87
corr(bill_length, body_mass) drop / fill: 0.595 / 0.595
```

Example 1.2 (Drop, fill, or flag)**.** Strategy: missingness is data too. For each column ask *why* the cell is empty before deciding what to do with it.

Solution. Listing **??** compares the two standard repairs on the four numeric penguin columns. Dropping the two incomplete rows and filling them with the column median barely move the column means (largest shift: body mass, 4201.75 → 4200.87) — either is defensible *here* because the holes are tiny. The `sex` column is different: its 11 holes are 3% of the data and categorical, so there is no median to fill. We make *missing* its own category in §**??** and let the model learn what it means. The rule that survives: **choose the repair that loses the least information**, and record whichever you chose — a reviewer will ask.

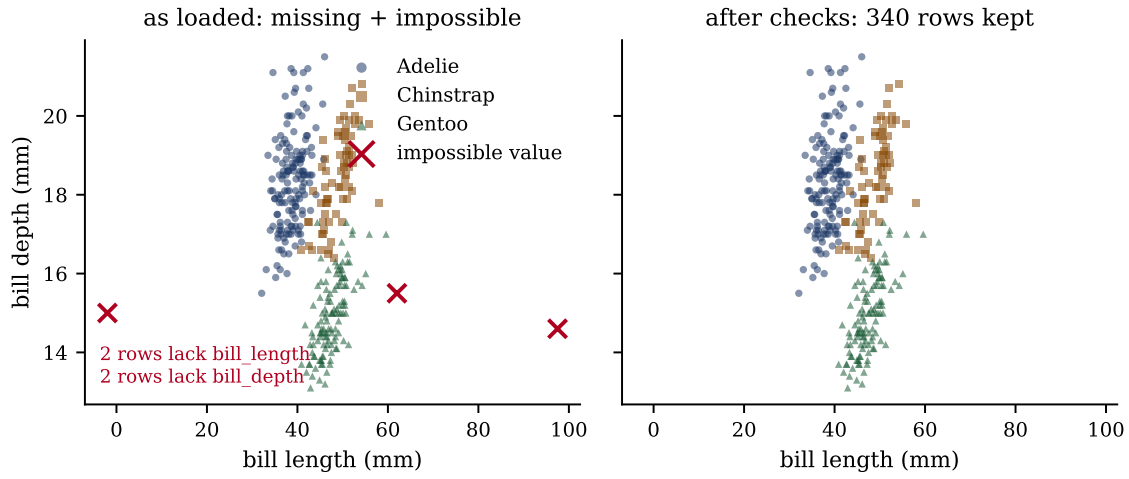


Figure 1.3: Bill length vs bill depth for the `penguins` dataset (`figures/fig_ch01_missing_scatter.py`). Left: as loaded — three impossible bill lengths injected for the hygiene demo plot as red crosses (one stretches the axis so far the real data compresses), and two rows that lack *both* coordinates cannot be drawn at all. Right: the same data after the checks of §**??**.

1.3.2 Encodings, and leakage at the loading step

Two more things a raw column can hide. *Encoding drift*: the same fact spelled `MALE`, `Male`, `m`, and `male` lands as four categories — normalise case and whitespace *before* building any codebook. And the sneakiest failure of all: *leakage* — information from the future (the test rows) flowing into the decisions you make on the past (the training rows). At the loading step three leaks recur:

1. **Normalize-then-split**: computing column means/standard deviations on the full file, then splitting. The test rows contributed to the scaling — a small but real leak. Fit statistics on the training rows only [**?**].
2. **Split-then-dedup**: removing duplicates *after* the split, so a row and its twin can land on opposite sides and give the model a free answer. §**??** measures what this costs.
3. **Codebook-from-everything**: building the one-hot level list (or a vocabulary) from all rows including held-out ones. Usually a small leak; occasionally the held-out level simply does not exist in training and the bug hides until deployment.

The test set must look like the future. Anything your pipeline learns — means, medians, category lists, the decision to drop a column — it must learn from training rows alone.

1.4 Features and targets; one-hot encoding

A feature is any column you will let the model see; the target is the column you want it to predict. Features come in three practical kinds:

- **Numerical** — measurements where arithmetic means something: bill length, mass, price.
- **Categorical (nominal)** — labels with no order: island, species, colour. Integers are *not* an honest encoding: assigning `Biscoe`=0, `Dream`=1, `Torgersen`=2 invents a distance and an order that do not exist.
- **Ordinal** — categories with order but no scale: `small<medium<large`. Integer codes are defensible here, at the cost of pretending the gaps are equal.

The honest encoding for nominal data is *one-hot*: a level with k values becomes k binary columns, exactly one of which is 1 per row. A penguin from Biscoe has `island=Biscoe` on and the other two island columns off. Missing is a real state, not a zero: we give `<missing>` its own column so an all-zero row is unambiguous.

```
# code/ch01/onehot_scratch.py -- one-hot encode categorical columns, from scratch.
# Run from the repository root: python3 code/ch01/onehot_scratch.py
import csv
import numpy as np

def load_columns(path):
    with open(path, newline="") as f:
        rows = list(csv.reader(f))
    return rows[0], rows[1:]

def onehot_column(values):
    """Return (block, names): (n, k) 0/1 float block, one column per observed level
    plus an extra column for missing entries ('')."""
    levels = sorted({v for v in values if v != ""})
    names = levels + (["<missing>"] if "" in values else [])
    idx = {v: i for i, v in enumerate(levels)}
```

```
    B = np.zeros((len(values), len(names)))
    for i, v in enumerate(values):
        B[i, idx[v] if v != "" else len(levels)] = 1.0
    return B, names

header, data = load_columns("data/penguins.csv")
num_cols = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "
    body_mass_g"]
cat_cols = ["sex", "island"]
ji = {h: i for i, h in enumerate(header)}

# numeric block: empty -> NaN (float array can hold NaN)
Xnum = np.array([[float(r[ji[c]]) if r[ji[c]] != "" else np.nan
                 for c in num_cols] for r in data])
blocks, names = [Xnum], list(num_cols)
for c in cat_cols:
    B, bn = onehot_column([r[ji[c]] for r in data])
    blocks.append(B)
    names += [f"{c}={n}" for n in bn]

X = np.concatenate(blocks, axis=1)
print("numeric cols:", len(num_cols), " categorical cols:", cat_cols)
print("one-hot names:", names[len(num_cols):])
print(f"shape change: ({len(data)}, {len(num_cols) + len(cat_cols)})
    -> {X.shape}")
print("first encoded row:", np.array2string(X[0], precision=1,
    separator=", ",
     suppress_small=True))
```

Listing 1.3: `code/ch01/onehot_scratch.py`: one-hot from scratch — levels become columns; missing gets a column of its own.

```
numeric cols: 4 categorical cols: ['sex', 'island']
one-hot names: ['sex=FEMALE', 'sex=MALE', 'sex=<missing>', 'island=
    Biscoe', 'island=Dream', 'island=Torgersen']
shape change: (344, 6) -> (344, 10)
first encoded row: [ 39.1, 18.7, 181. , 3750. , 0. , 1. , 0. , 0. , 0.
     ,
    1. ]
```

Example 1.3 (One-hot on real data)**.** Strategy: replace each categorical column with one binary column per observed level, plus one for missing.

Solution. `sex` has two levels plus 11 missing cells $\to$ 3 columns; `island` has three levels, no missing $\to$ 3 columns. The design matrix grows $(344, 6) \to (344, 10)$: six raw fields become ten features, four numeric and six binary. The first row reads $[39.1, 18.7, 181, 3750, 0, 1, 0, 0, 0, 1]$ — a male Adelie on Torgersen.

Two boundary cases worth naming now. A held-out row carrying a level never seen in training (`island=Saunders`) lights nothing — decide whether that is an error or an all-zeros row *before* deployment, not during an incident. And high-cardinality columns

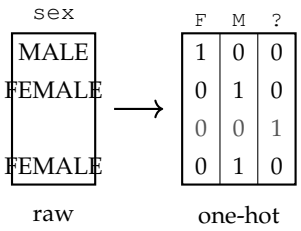


Figure 1.4: One-hot encoding: each level of a categorical column gets its own binary column; exactly one is 1 per row. A missing entry lights the `<missing>` column rather than being silently zeroed — the encoding keeps ignorance distinguishable from a definite answer.

(postcodes, user ids) explode the width: 10^4 postcodes mean 10^4 sparse columns, at which point embeddings (chapter 10) take over.

Checkpoint 1.2 *Width check.* How many columns does one-hot add for a feature with k observed levels and no missing cells? Why can dropping one of those columns be a good idea for linear models (hint: what does the sum of the k columns equal)?

1.5 Shuffling, batching, and the speed of vectorisation

Three loading-step habits the rest of the book leans on:

- **Shuffle indices, once.** `perm = rng.permutation(n)` then `X[perm], y[perm]`. Permuting rows and labels separately — or forgetting `y` entirely — silently mismatches every label (§**??**). Seeding the generator (`default_rng(7)`) makes the shuffle reproducible.
- **Batches are slices.** A minibatch is just `X[perm[k:k+B]]`. When n is not divisible by the batch size B, the last batch is *short*, not missing — decide to keep or drop it on purpose.
- **Vectorise by default.** Loops in Python are for orchestration; arithmetic belongs to the array.

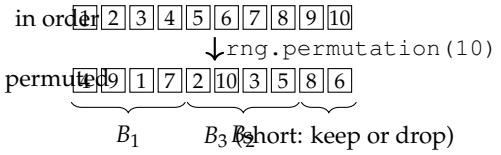


Figure 1.5: The whole minibatch machinery of this book: permute the row indices once, then hand out contiguous slices. The last batch is smaller than the rest — a decision, not an accident.

How much does vectorising actually buy? Enough to measure.

```
# code/ch01/broadcast_benchmark.py -- loop vs vectorised, timed for real.
# Run from the repository root: python3 code/ch01/broadcast_benchmark.py
import math
```

```
import statistics
import time
import warnings
import numpy as np

# this platform's BLAS fires a spurious RuntimeWarning on first use of a ufunc
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    np.eye(2) @ np.ones(2)
    np.arange(4) @ np.ones(4)

rng = np.random.default_rng(0)
n, d = 10_000, 4
X = rng.normal(size=(n, d))
w = rng.normal(size=d)
mu = X.mean(axis=0)
Xl, wl, mul = X.tolist(), w.tolist(), mu.tolist() # same data as Python lists

def bench(fn, reps=5):
    fn() # warmup: BLAS/Lazy-inits excluded
    ts = []
    for _ in range(reps):
        t0 = time.perf_counter()
        out = fn()
        ts.append(time.perf_counter() - t0)
    return statistics.median(ts), out

# --- (a) row scores s_i = x_i . w (the workhorse of linear models) ---
t_py, s_py = bench(lambda: [
    sum(x * wi for x, wi in zip(row, wl)) for row in Xl], 3)
t_npl, s_npl = bench(lambda: np.array([X[i] @ w for i in range(n)]), 3)
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    t_vec, s_vec = bench(lambda: X @ w, 50)

# --- (b) distance of every row to the column mean, n = 10^4 ---
t_dp, d_py = bench(lambda: [
    math.sqrt(sum((x - m) ** 2 for x, m in zip(row, mul))) for row in Xl], 3)
t_dv, d_vec = bench(lambda: np.sqrt(((X - mu) ** 2).sum(axis=1)), 50)

# --- (c) pairwise distances on the first 2,000 rows ---
m = 2000
A, Al = X[:m], Xl[:m]
t_pp, P_py = bench(lambda: [
    [math.sqrt(sum((a - b) ** 2 for a, b in zip(ra, rb))) for rb in Al]
    for ra in Al], 1)
```

```
t_pv, P_vec = bench(lambda: np.sqrt(((A[:, None, :] - A[None, :, :])
    ** 2).sum(-1)), 20)

print(f"X: {n} x {d} float64 (pairwise row uses first {m} rows; median
    of warm runs)")
print(f"{'operation':<32}{'python loop':>12}{'np call/row':>12}{'
    vectorised':>12}{'slowest/vec':>13}")
for name, a, b, c in [("row scores X @ w", t_py, t_npl, t_vec),
                      ("row distances to mean", t_dp, float("nan"), t_dv),
                      ("pairwise 2000x2000", t_pp, float("nan"), t_pv)]:
    bs = f"{b:>12.4f}" if b == b else f"{'--':>12}"
    worst = max(a, b) if b == b else a
    print(f"{name:<32}{a:>12.4f}{bs}{c:>12.5f}{worst / c:>12.0f}x")
print("same answer:", np.allclose(s_py, s_vec), np.allclose(s_npl,
    s_vec),
      np.allclose(d_py, d_vec), np.allclose(P_py, P_vec))

# emit the bar-chart data the chapter figure reads (figures/ch01/
    timing.dat)
import os
os.makedirs("figures/ch01", exist_ok=True)
with open("figures/ch01/timing.dat", "w") as f:
    f.write("op speedup\n")
    f.write(f"scores-loop {t_py / t_vec:.0f}\n")
    f.write(f"scores-nprow {t_npl / t_vec:.0f}\n")
    f.write(f"dist-loop {t_dp / t_dv:.0f}\n")
    f.write(f"pairwise-loop {t_pp / t_pv:.0f}\n")
```

Listing 1.4: `code/ch01/broadcast_benchmark.py`: three workloads, three implementations each, median of warm runs.

```
X: 10000 x 4 float64 (pairwise row uses first 2000 rows; median of
    warm runs)
operation python loop np call/row vectorised slowest/vec
row scores X @ w 0.0029 0.0037 0.00002 165x
row distances to mean 0.0054 -- 0.00009 62x
pairwise 2000x2000 1.6407 -- 0.05148 32x
same answer: True True True True
```

Example 1.4 (Loop vs vectorised, priced)**.** Strategy: time the same arithmetic three ways — a Python loop over lists, a NumPy call per row, and one vectorised expression.

Solution. On this machine the vectorised row-score (`X @ w`) runs 165× faster than the per-row NumPy loop and 128× faster than the Python loop; the full table is in the output above and Figure **??**. Two lessons, both worth internalising: the per-row `np.linalg.norm` loop is the *worst* of both worlds (Python-loop overhead *plus* a fresh array each iteration), and the gap only grows with n — your numbers will differ from these, the ordering will not [**?**].

WHEN NOT TO VECTORISE

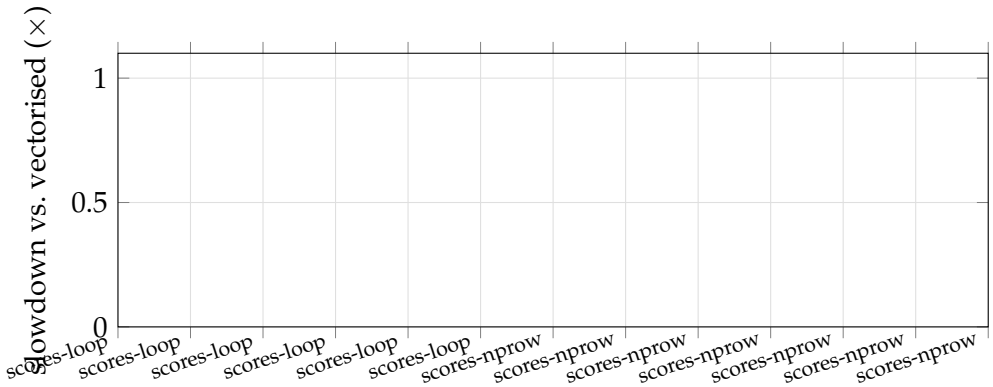


Figure 1.6: Measured slowdowns from `broadcast_benchmark.py` on a $10^4 \times 4$ float64 matrix. The pairwise row compares a pure-Python double loop against broadcasting on the first 2,000 rows — even at a size the loop can finish, it is over 30× slower.

Broadcasting builds the (n, m, d) intermediate it needs. For pairwise distances that is $8nmd$ bytes — $10^4 \times 10^4 \times 4$ is already ∼3 GB. When the intermediate stops fitting in memory, *block* it: vectorise inside a Python loop over row-chunks of a few thousand. The loop runs a handful of times; the inner op stays in C.

Checkpoint 1.3 *Memory arithmetic.* How many bytes does the intermediate `(X[:, None, :] - X[None, :, :])**2` need for $n = 10^5$, $d = 64$ float64? Is broadcasting the right tool there? What is the fix?

1.6 Data hygiene: duplicates, impossible values, units

Before any model touches a new file, three checks cost almost nothing and catch most real-world dirt:

- ☐ **Duplicates**: `np.unique(X, axis=0)` — exact twins are rare in honest measurement and common in scraped or merged data.
- ☐ **Impossible values**: range-check each column against reality — a bill length of −2 mm or 97.5 mm, a mass of 0, an age of 212.
- ☐ **Units**: read the schema *and* the magnitudes. A height column mixing cm and m looks plausible row-by-row and is quietly nonsense. The column's summary statistics usually give the game away.

Why it matters beyond tidiness: duplicated rows break the assumption behind every random split. If a row and its twin land on opposite sides of the split, a memorising model has effectively seen some test answers during training.

```
# code/ch01/duplicates_leak.py -- how a duplicated row inflates test accuracy.
# Run from the repository root: python3 code/ch01/duplicates_leak.py
import csv
```

```
import numpy as np

with open("data/iris.csv", newline="") as f:
    rows = list(csv.reader(f))[1:]
X = np.array([[float(v) for v in r[:4]] for r in rows])
y = np.array([r[4] for r in rows])

def knn1(Xtr, ytr, Xte):
    """1-nearest-neighbour: the cheapest memoriser (ch3 formalises it).
       """
    d2 = ((Xte[:, None, :] - Xtr[None, :, :]) ** 2).sum(-1)
    return ytr[d2.argmin(1)]

def split_accuracy(X, y, seed):
    rng = np.random.default_rng(seed)
    perm = rng.permutation(len(y))
    k = int(0.8 * len(y))
    tr, te = perm[:k], perm[k:]
    return (knn1(X[tr], y[tr], X[te]) == y[te]).mean()

# plant 40 duplicate rows across classes (a scraped dataset often has
    this)
rng = np.random.default_rng(1)
pick = rng.choice(len(X), size=40, replace=False)
Xd = np.vstack([X, X[pick]])
yd = np.concatenate([y, y[pick]])

# shuffle-then-split WITHOUT deduplication: twins land on both sides
    of the split
leak = np.mean([split_accuracy(Xd, yd, s) for s in range(10)])

# deduplicate FIRST, then split: no row can appear on both sides
Xu, iu = np.unique(Xd, axis=0, return_index=True)
yu = yd[iu]
honest = np.mean([split_accuracy(Xu, yu, s) for s in range(10)])

print(f"dataset: {len(X)} rows -> {len(Xd)} after planting 40
    duplicate rows")
print(f"split-then-eval, duplicates in: {leak:.3f} mean test acc (10
    seeds)")
print(f"dedup first, then split and eval: {honest:.3f} mean test acc
    (10 seeds)")
print(f"phantom gain: {(leak - honest) * 100:+.1f} percentage points")

# the same bug on a harder task: two overlapping blobs, 1-NN base acc
    ~0.85
def blobs(n, seed):
    r = np.random.default_rng(seed)
    Xa = r.normal([0, 0], 1.4, (n // 2, 2))
    Xb = r.normal([1.6, 0.4], 1.4, (n - n // 2, 2))
    return np.vstack([Xa, Xb]), np.array([0] * (n // 2) + [1] * (n - n
       // 2))
```

```
48
49 Xs, ys = blobs(500, 3)
50 pick = rng.choice(len(Xs), size=150, replace=False)
51 Xsd, ysd = np.vstack([Xs, Xs[pick]]), np.concatenate([ys, ys[pick]])
52 leak_s = np.mean([split_accuracy(Xsd, ysd, s) for s in range(10)])
53 Xsu, iu = np.unique(Xsd, axis=0, return_index=True)
54 honest_s = np.mean([split_accuracy(Xsu, ysd[iu], s) for s in range(10)
       ])
55 print(f"\nblobs: {len(Xs)} rows -> {len(Xsd)} with 150 planted
       duplicates")
56 print(f"split-then-eval, duplicates in: {leak_s:.3f} mean test acc (10
        seeds)")
57 print(f"dedup first, then split and eval: {honest_s:.3f} mean test acc
        (10 seeds)")
58 print(f"phantom gain: {(leak_s - honest_s) * 100:+.1f} percentage
       points")
59
60 # emit the bar-chart data the chapter figure reads (figures/ch01/leak.
       dat)
61 import os
62 os.makedirs("figures/ch01", exist_ok=True)
63 with open("figures/ch01/leak.dat", "w") as f:
64     f.write("scenario leak honest gain\n")
65     f.write(f"iris {leak:.3f} {honest:.3f} {(leak - honest) * 100:.1f}\
          n")
66     f.write(f"blobs {leak_s:.3f} {honest_s:.3f} {(leak_s - honest_s) *
          100:.1f}\n")
```

Listing 1.5: `code/ch01/duplicates_leak.py`: the duplicate-row leak, planted and measured.

```
dataset: 150 rows -> 190 after planting 40 duplicate rows
split-then-eval, duplicates in: 0.979 mean test acc (10 seeds)
dedup first, then split and eval: 0.967 mean test acc (10 seeds)
phantom gain: +1.2 percentage points

blobs: 500 rows -> 650 with 150 planted duplicates
split-then-eval, duplicates in: 0.766 mean test acc (10 seeds)
dedup first, then split and eval: 0.651 mean test acc (10 seeds)
phantom gain: +11.5 percentage points
```

Example 1.5 (The fake win)**.** Strategy: plant duplicates, split randomly, and watch a 1-nearest-neighbour classifier (ch. 3 formalises it) cash in free answers.

Solution. On `iris` the phantom gain is small (+1.2 pp) only because iris is nearly separable — there is little room left to inflate. On a harder two-blob dataset the same bug buys +11.5 pp (Figure **??**). The fix costs one line *before* the split: deduplicate (`np.unique(X, axis=0)`) first, then split. This is the chapter's failure-analysis item: **an accuracy is only as honest as the split that produced it.**

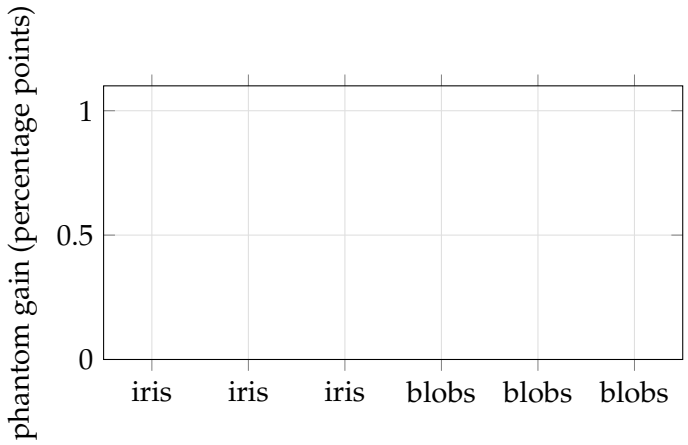


Figure 1.7: What a duplicated row buys you: mean accuracy gained by splitting *before* deduplicating, over ten seeds each. The harder the task, the bigger the fake win.

A CLEAN NUMBER CAN BE A LEAKED NUMBER

Any step that looks at more rows than the training set — deduplication, imputation, scaling, vocabulary building — must run *before* the split (if it is data-level, like dedup) or be fitted on training rows only (if it is statistical, like a median). When in doubt, ask: *would this information exist at prediction time?*

1.7 Common mistakes

Every entry below has cost a real project real time.

Reading the shape backwards. `(d, n)` instead of (n, d): treating each feature as a sample. *Fix:* say the shape aloud; row 0 is a sample.

The silent (n, n) broadcast. Adding a $(n, 1)$ column and a $(n,)$ row produces an $n \times n$ table instead of an element-wise op. *Fix:* `b[:, None]`, or keep `keepdims=True` on the reduction that produced the vector.

dtype truncation. Assigning floats into an `int` array floors them silently; assigning `nan` into one raises *or* wraps to a large integer depending on the path. *Fix:* allocate `dtype=float` when the data is continuous; check `X.dtype` after any assembly.

Shuffling X and y differently. Two permutations means every label is wrong and the model still trains happily. *Fix:* permute *indices* once; index everything with the same `perm`.

Trusting `np.loadtxt` on dirty files. It chokes on empty fields and mixed columns — or worse, `genfromtxt` silently yields `nan` that then poisons means. *Fix:* count `np.isnan` per column immediately after loading.

Fitting statistics on all data. A mean or median computed on train+test before splitting is a leak (small but real). *Fix:* statistics on training rows only — the pipeline order *is* the bug fix.

Losing the last batch. `for k in range(n // B)` drops the remainder. *Fix:* `range(0, n, B)` and accept a short final batch, or drop it deliberately and say so.

Mutating a view. `b = X[:, 0]; b /= b.max()` writes through into X — slices are views, not copies. *Fix:* `.copy()` when you intend to modify.

Treating "missing" as zero. Filling a categorical hole with the all-zeros pattern makes missing indistinguishable from a real category. *Fix:* missing gets its own column (or its own category), so the model can learn what absence means.

1.8 Summary table

Object / step	Meaning	Idiom
$X \in \mathbb{R}^{n \times d}$	design matrix, one row per sample	`X.shape == (n, d)`
y	targets aligned to rows	`y.shape == (n,)`
broadcasting	size-1 axes stretch right-aligned	`X - mu`, `A[:,None,:] - B[None,:,:]`
CSV load	strings first, convert per column	`csv.reader` → `np.array(..., dtype=float)`
missing cells	count before repairing	`np.isnan(X).sum(axis=0)`
categorical	nominal → one-hot, missing is a level	`<missing>` column
shuffle	permute indices once, seed it	`rng.permutation(n)`
batch	contiguous slices of the permuted order	`X[perm[k:k+B]]`
vectorise	ops in C, not Python loops	`X @ w`, `np.linalg.norm(X, axis=1)`
dedup	before the split, always	`np.unique(X, axis=0)`
leak check	statistics learned on train rows only	§**??** tripwire

Table 1.1: Chapter 1 cheat sheet: the shapes, the idioms, and the order of operations that keeps the data honest.

Exercises

Exercise 1.1 (easy)**.** Predict, then verify: `np.array([1, 2, 3]) + 5`, `np.zeros((2, 3)) + np.ones(3)`, and the shapes of `X[0]`, `X[:, 0]`, `X.T` for `X.shape == (150, 4)`.

Exercise 1.2 (easy)**.** A $(5, 1, 4)$ array plus a $(3, 4)$ array: write the output shape. Then a $(2, 3)$ plus a $(2,)$ — does it broadcast? Give the result shape or the error.

Exercise 1.3 (easy)**.** What does `np.array([1.7, 2.3, 9.9], dtype=int)` produce, and why? How would you make the rounding explicit instead?

Exercise 1.4 (easy)**.** `np.loadtxt` meets the file `1.0,2.0\n,3.0`: what happens? And `np.genfromtxt`? Which behaviour is easier to live with and why?

Exercise 1.5 (easy)**.** In `onehot_scratch.py`, a feature column with k distinct levels and no missing cells produces how many output columns? What if 5% of its cells are empty strings?

Exercise 1.6 (medium). Write `zscore(X)` in one line (column-wise standardisation), verify `Z.mean(axis=0)` and `Z.std(axis=0)` on a random matrix, and explain why `keepdims` is unnecessary here.

Exercise 1.7 (medium). Write `train_val_split(X, y, frac, seed)` returning `(Xtr, ytr, Xval, yval)` using `default_rng(seed)`. Why do we pass `frac` rather than an absolute count? Give one call site where the fraction matters.

Exercise 1.8 (medium). `genfromtxt` returned `nan` for the strings `""`, `"NA"`, and `"n/a"` but stored `-999` as a number. Write the check that flags sentinel values like -999, and explain why sentinels are more dangerous than NaNs.

Exercise 1.9 (medium). For $n = 103$, $B = 10$: how many batches does `range(0, n, B)` produce, and of what sizes? Write `iter_batches(perm, B)` yielding index slices; should the generator shuffle, or take a permutation? Defend the choice.

Exercise 1.10 (medium). A held-out penguin arrives with `island=Saunders`, a level your codebook has never seen. What does your one-hot encoding from §**??** produce for it? Give two engineering answers (reject vs. all-zeros) and one situation each is right.

Exercise 1.11 (medium). Memory arithmetic: a $(10^6, 100)$ float64 matrix costs how much RAM? The same data as float32? When is halving precision *not* free?

Exercise 1.12 (hard). Implement `iter_batches(X, y, B, seed)` as a generator that permutes once per call and yields `(X_b, y_b)` including a possibly short final batch. Prove: over n samples, $\bigcup_k B_k$ is a partition of $[n]$.

Exercise 1.13 (hard). Prove the broadcasting identity used in Listing **??**: for $A \in \mathbb{R}^{n\times d}$ and centroids $C \in \mathbb{R}^{k\times d}$, $\big(A[:, None, :] - C[None, :, :]\big)^2$ summed over the last axis equals the $n \times k$ matrix of squared Euclidean distances $\|\mathbf{a}_i - \mathbf{c}_j\|_2^2$. Where exactly does the stretch happen?

Exercise 1.14 (hard). Show that centring the columns of X by `X - X.mean(axis=0)` is the same as left-multiplying by the centring matrix $H = I_n - \frac{1}{n}\mathbf{1}\mathbf{1}^T$, i.e. HX. Conclude that centring twice is a no-op ($H^2 = H$).

Exercise 1.15 (hard). A teammate's pipeline: load CSV $\to$ fill NaNs with the column median over all rows $\to$ drop duplicate rows $\to$ shuffle $\to$ split $\to$ report accuracy. Find the two leaks and reorder the pipeline to remove them. Which leak did Listing **??** measure?

Exercise 1.16 (hard). Pairwise distances by blocking: write `pdist_blocked(A, B, rows)` that computes $\|a_i - b_j\|_2^2$ for all pairs using broadcasting inside a loop over row-blocks of `A`. Show the memory bound it honours and verify `allclose` against the single broadcast on small inputs.

Further reading

- NumPy documentation, *NumPy: the absolute basics for beginners* (v2.5 manual), https://numpy.org/doc/stable/user/absolute_beginners.html. The official onboarding — gentler than this chapter; read it alongside §**??** if any idiom felt fast.
- NumPy documentation, *Broadcasting*, https://numpy.org/doc/stable/user/basics.broadcasting.html. The three rules straight from the source, with the failing cases — bookmark it, you will be back.
- J. VanderPlas, *Python Data Science Handbook*, ch. 2, "The Basics of NumPy Arrays", https://jakevdp.github.io/PythonDataScienceHandbook/02.02-the-basics-of-numpy-arrays.html. Free online; intermediate. The vectorisation argument told properly, with ufuncs laid out one by one.
- N. P. Rougier, *From Python to NumPy*, https://www.labri.fr/perso/nrougier/from-python-to-numpy/. Free; intermediate-to-advanced. How a NumPy programmer thinks — custom vectorisation as a design skill, not a trick.
- Stanford CS231n, *Python Numpy Tutorial*, https://cs231n.github.io/python-numpy-tutorial/. A terse reference for the exact subset we use (arrays, broadcasting, indexing) — keep it open in a tab for a week.
- scikit-learn documentation, *Common pitfalls*, esp. §12.2 "Data leakage", https://scikit-learn.org/stable/common_pitfalls.html. The official checklist; our §**??** and §**??** are two items on it.
- C. R. Harris et al., "Array programming with NumPy", *Nature* 585 (2020), https://www.nature.com/articles/s41586-020-2649-2. Why the loop/vectorised gap is not an accident — the architecture paper for the library you just used.
- K. B. Gorman, T. D. Williams, W. R. Fraser, "Ecological sexual dimorphism and environmental variability within a community of Antarctic penguins", *PLOS ONE* 9(3) (2014), https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0090081. Where the `penguins` data actually comes from — worth it just for the field methods.
- R. A. Fisher, "The use of multiple measurements in taxonomic problems", *Annals of Eugenics* 7(2) (1936), https://doi.org/10.1111/j.1469-1809.1936.tb02137.x. The iris paper. Historical interest: it is also the first linear discriminant analysis.

WATCH INSTEAD

If you would rather watch than read for a first pass (then come back and do the exercises — the exercises are where it becomes yours):

- *Vectors | Chapter 1, Essence of linear algebra* — **3Blue1Brown**, 9:51, https://www.youtube.com/watch?v=fNk_zzaMoSs. Matches §**??**–**??**: the "vector as an ordered list of numbers" is exactly our $\mathbf{x}_i$. **Start here.**
- *Machine Learning Fundamentals: Cross Validation* — **StatQuest with Josh Starmer**, 6:04, https://www.youtube.com/watch?v=fSytzGwwBVw. Matches §**??**–**??**: why we split data at all; previews the evaluation protocol formalised in chapter 3.
- *Lecture 2 | Image Classification* — **Stanford University School of Engineering** (CS231n), 59:31, https://www.youtube.com/watch?v=OoUX-nOEjG0.

Matches §**??**–**??** at course pace: the data-driven pipeline, a distance-based classifier like ours, and the train/val/test discipline. Long — treat it as a week of commutes.

All three opened and verified 2026-10-11; none duplicates another.

Notes

1. Iris provenance: the 150-row iris set is Fisher's 1936 measurements [**?**]; `penguins` is Gorman, Williams and Fraser's Palmer Station data [**?**]; we load both as CSVs from the seaborn-data collection [**?**].
2. The broadcasting rules and their failure modes are stated per the NumPy manual [**?**]; the timing methodology (warm runs, medians) follows standard benchmarking practice — rerun `broadcast_benchmark.py` on your own machine before quoting numbers, as they are hardware-dependent.
3. The leakage taxonomy (normalise-then-split, fit-on-test) follows the scikit-learn pitfalls documentation [**?**]; this book returns to it formally in chapter 3.
4. A pedagogical note: applied courses such as fast.ai [**?**] put a working model before any mechanism. This book keeps the mechanism-first order — Book 3 exists, so we can afford it — but if you are impatient, the eight-line classifier in §**??** is the whole fast.ai lesson in miniature.
5. Version pins for this chapter's runs: Python 3.9, NumPy 2.0.2 — recorded in `code/README.md`; dataset files live in `data/`.

Hints and answers

1.1–1.5 (easy) Work them in a Python prompt, not on paper — the answer is the point of the exercise, the prompt is the point of the habit.

1.6 `Z = (X - X.mean(0)) / X.std(0)`; means are ≈ 0, stds are 1 up to float error.

1.7 `k = int(frac * n)`; `perm = rng.permutation(n)`; return slices. A fraction survives a dataset swap; a hardcoded count does not.

1.8 Range-check per column (`(x < lo) | (x > hi)`), or treat the sentinel as missing before parsing. Sentinels are worse: `nan` trips `isnan`; -999 averages quietly into means.

1.9 11 batches: ten of 10, one of 3. Take `perm` in — the caller owns the shuffle so X and $\mathbf{y}$ stay aligned and reproducible.

1.10 All-zeros in this design (Saunders matches no level column). Rejecting is right when the new level should change the model; all-zeros is right when "unknown island" is itself the signal.

1.11 $10^6 \times 100 \times 8 \approx 800$ MB (float64), ≈ 400 MB (float32). Halving precision is not free when small additive terms matter (e.g. accumulating 10^6 squared errors) — check dtype before blaming the model.

1.12 Loop over `range(0, n, B)`; inside, slice both arrays with `perm[k:k+B]` and yield the pair. Disjoint contiguous slices of a permutation partition $[n]$.

1.13 Expand the distance as a sum over coordinates: $\|\mathbf{a}_i - \mathbf{c}_j\|_2^2 = \sum_{t=1}^{d}(a_{it} - c_{jt})^2$ — the last-axis sum *is* that inner sum; the two `None` axes stretch A over j and C over i.

1.14 H is symmetric idempotent: $H^2 = I - \frac{2}{n}\mathbf{1}\mathbf{1}^T + \frac{n}{n^2}\mathbf{1}\mathbf{1}^T = H$. Centring is a projection — Book 3's language.

1.15 Median-fill on all rows leaks test statistics into training; dedup after split lets twins straddle the split — the leak Listing **??** measured. Honest order: load → dedup → split → fit the median on *train* → apply to both → evaluate.

1.16 Loop over row-blocks of A with broadcasting inside; the intermediate is at most `rows` $\times\, m \times d$ floats. It is the rulebox from §**??** made concrete.

CHAPTER 2

Tables, pandas and visualisation

Chapter **??** taught you to treat a dataset as a bare matrix X: numbers in, shapes out. Real work rarely stays in that minimal form. Columns have names that mean something ("fare" is not "column 6"), rows belong to groups you want to compare, and the fastest way to understand a table is to plot it. This chapter upgrades the matrix to a `DataFrame` — the same X, wearing labels — and gives you the two verbs that do 90% of all tabular work: *group-and-aggregate* and *reshape*. Then it does the part most tutorials skip: how to draw a picture of the table that does not lie to you, and how to catch the ones that do. We end on the two failure modes that matter most — an aggregate that hides the distribution, and an aggregate that reverses it. Everything runs on real data: the 1912 Titanic passenger manifest, the classic 1949–1960 airline-passenger series, and the penguins from chapter 1.

2.1 The DataFrame is a labelled matrix

A `DataFrame` is the matrix of chapter **??** plus three things the matrix did not have: a column name for every feature, a row label (the *index*) for every sample, and a *dtype* per column that can differ between columns — `int64` counts, `float64` measurements, `object` strings. The matrix inside is still an `ndarray` per column; what changed is that every operation can be addressed by *name* instead of by position.

```
>>> df = pd.read_csv("data/titanic.csv")
>>> df.shape # like ndarray.shape
(891, 15)
>>> df.dtypes.head(5) # one dtype per column
survived int64
pclass int64
sex object
age float64
sibsp int64
dtype: object
>>> df["age"].mean() # address by name, not position
29.69911764705882
```

LABELS ARE DATA TOO

In chapter **??** the label lived *outside* the matrix (y beside X). In a DataFrame the labels live *inside* the container — and that changes the hygiene rules, not just the syntax. A column named `survived` is one rename away from becoming a feature by accident; an index that silently duplicated a passenger is one groupby away from a counting

bug. Names make mistakes *legible*: a wrong `df.fare` fails loudly, a wrong `X[:,6]` fails silently.

Three selection idioms cover almost everything; learn them cold.

- `df[df.fare > 100]` — a boolean mask, exactly the fancy indexing of §**??**, now on named columns.
- `df.loc[rows, cols]` — label-based slicing: `df.loc[df.sex == "female", "age"]`
- `df.iloc[0:5, 0:3]` — positional slicing, identical to `X[0:5, 0:3]` on the underlying arrays.

Checkpoint 2.1 *Reading a table's dtype story.* `titanic.csv` loads `age` as `float64` but `pclass` as `int64`. Why the difference? (Hint: check `df.age.isna().sum()` and `df.pclass.unique()`. What would the dtype have been if `age` had no holes?)

2.2 Filter, aggregate, groupby: the SQL-shaped verbs

Most tabular questions decompose into four verbs, and pandas names them the way SQL does: *select* columns, *filter* rows, *group* by a key, *aggregate* to one number per group. The shape mental-model: a groupby turns one (n, d) table into k smaller tables, and the aggregate collapses each into one row — an (k, m) table where m is how many measures you asked for.

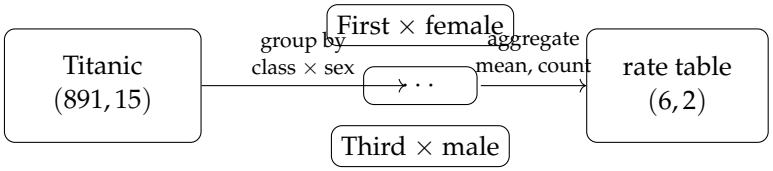


Figure 2.1: Split–apply–combine: `groupby` partitions the big table, the aggregate collapses each partition to a row, and the result is a small table indexed by the group key.

```
# code/ch02/titanic_groupby.py -- survival rate by class x sex via groupby.
# Run from the repository root: python3 code/ch02/titanic_groupby.py
import pandas as pd

df = pd.read_csv("data/titanic.csv")
print("shape:", df.shape)
print("dtypes (first 8):\n", df.dtypes.head(8), "\n")

# the SQL-shaped verb: SELECT class, sex, mean(survived), count(*) GROUP BY 1,2
rate = df.groupby(["class", "sex"], observed=True)["survived"].agg(["mean", "count"])
rate["mean"] = (rate["mean"] * 100).round(1)
rate.columns = ["survival_%", "n"]
print("survival rate by class x sex:")
```

```
14 print(rate)
15
16 # the same answer as a matrix (unstack = pivot one index level into
       columns)
17 print("\nas a wide table:")
18 wide = df.pivot_table(index="class", columns="sex", values="survived",
19                       aggfunc="mean", observed=True)
20 print((wide * 100).round(1).astype(str) + " %")
21
22 # select-and-aggregate in one line, no groupby needed
23 print("\nFirst-class women aged <=16, survival:",
24       df.query("sex == 'female' and `class` == 'First' and age <= 16")
25         ["survived"].mean().round(3))
```

Listing 2.1: `code/ch02/titanic_groupby.py`: the two idioms for one question — `groupby+agg` produces a labelled $(6, 2)$ table, `pivot_table` the same answer laid out $(3, 2)$.

Run from the repository root, it prints:

```
shape: (891, 15)
dtypes (first 8):
 survived int64
pclass int64
sex object
age float64
sibsp int64
parch int64
fare float64
embarked object
dtype: object

survival rate by class x sex:
           survival_% n
class sex
First female 96.8 94
      male 36.9 122
Second female 92.1 76
      male 15.7 108
Third female 50.0 144
      male 13.5 347

as a wide table:
sex female male
class
First 96.8 % 36.9 %
Second 92.1 % 15.7 %
Third 50.0 % 13.5 %

First-class women aged <=16, survival: 0.833
```

Example 2.1 (WE 2.1 — Survival by class and sex)**.** Strategy: one question, one groupby. "Did class and sex predict survival?" is a $(6, 1)$ table away — mean of a $0/1$ column is a rate.

Solution. Listing **??** answers it in three lines. The wide table is the famous result: first-class women survived at 96.8%, third-class men at 13.5% — an 83-point gap between the luckiest and unluckiest groups. Two cautions hidden in the same output: the `n` column matters (third-class women, n=144, are a real estimate; a group of n=5 would be a coin flip), and "survival by class" is a correlation — the manifest does not tell you *why* (boat position, queue order, crew priorities). The table describes; it does not explain.

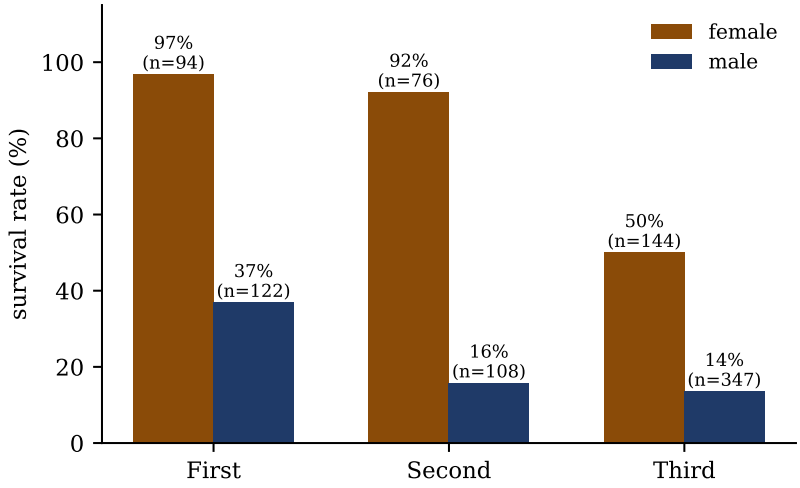


Figure 2.2: WE 2.1 as a grouped bar chart (`figures/fig_ch02_survival_bars.py`). Counts are drawn on every bar — 96.8% of n=94 and 50.0% of n=144 are not equally trustworthy numbers.

2.3 Joins and reshaping: long vs wide

The same table can be stored two ways. *Long* form: one row per observation, columns are (key, variable, value) — flexible, tidy, what groupby wants. *Wide* form: one row per entity, one column per measurement — readable, compact, what a table wants. The canonical tidy rule [**?**]: **each variable is a column, each observation is a row** — keep the long form as your working truth and reshape to wide only for presentation. The verbs:

- `pivot(index=..., columns=..., values=...)` — long → wide: the values of one column become new column names.
- `melt(...)` — wide → long: column names fold back into a variable column.
- `merge(other, on=key)` — the join: match rows of two tables on a shared key. One line joins a grouped sum back onto the frame: `fl.merge(fl.groupby("year").passengers.sum(), on="year")` attaches each month's year total to its row — the SQL `JOIN` on an aggregate, verbatim.

```
# code/ch02/pivot_longwide.py -- long <-> wide reshaping on real data.
# Run from the repository root: python3 code/ch02/pivot_longwide.py
import pandas as pd

fl = pd.read_csv("data/flights.csv")
```

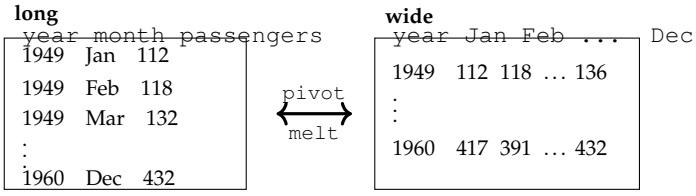


Figure 2.3: Long vs wide. Left: 144 rows of (`year`, `month`, `passengers`) — tidy; groupby-ready. Right: the same data as a 12×12 matrix — readable; what a report wants. `pivot` and `melt` are exact inverses: nothing is lost.

```python
print("long shape:", fl.shape)
print(fl.head(3).to_string(index=False), "\n")

# long -> wide: one row per year, one column per month
wide = fl.pivot(index="year", columns="month", values="passengers")
print("wide shape:", wide.shape)
print(wide.head(4).to_string())
print("\nrow index:", wide.index.name, "| col index:", wide.columns.
    name)

# wide -> long: melt is the inverse
back = wide.melt(ignore_index=False).reset_index()
print("\nback to long, sorted:")
back = back.sort_values(["year", "month"], ignore_index=True)
print(back.head(3).to_string(index=False))
print("matches original (set-compare):",
     set(map(tuple, back[["year", "month", "value"]].values))
     == set(map(tuple, fl.values)))
```

Listing 2.2: `code/ch02/pivot_longwide.py`: long → wide → long on the real airline-passenger series; the last line proves the round trip is lossless.

```
long shape: (144, 3)
 year month passengers
 1949 January 112
 1949 February 118
 1949 March 132

wide shape: (12, 12)
month April August December February January July June March May
   November October September
year
1949 129 148 118 118 112 148 135 132 121 104 119 136
1950 135 170 140 126 115 170 149 141 125 114 133 158
1951 163 199 166 150 145 199 178 178 172 146 162 184
1952 181 242 194 180 171 230 218 193 183 172 191 209

row index: year | col index: month

back to long, sorted:
 year month value
 1949 April 129
```

```
 1949 August 148
 1949 December 118
matches original (set-compare): True
```

Example 2.2 (WE 2.2 — One series, two shapes)**.** Strategy: know which shape each consumer wants. A grouped aggregation wants long; a slide or a regression matrix wants wide.

Solution. Listing **??** runs both directions on 144 months of real airline data: `pivot` produces the 12 × 12 year-by-month grid (the classic "small multiples" layout), `melt` folds it back, and the set-compare at the end confirms the two forms carry identical information. Note the months come back alphabetically (April first) — **wide columns sort by name, not by your intent**; if the column order matters, reindex it. When you reach for Excel-style thinking, reach for `pivot_table` with an `aggfunc` instead — it is `pivot` that tolerates duplicate (index, column) pairs by aggregating them.

2.4 Visualisation that does not lie

matplotlib draws in two layers, and confusing them is the root of most broken plots. The *Figure* is the page; *Axes* are the panels inside it (coordinate system, ticks, spines); the *artists* are what gets inked (lines, bars, text). The idiom that keeps you sane: `fig, ax = plt.subplots()` then call methods on `ax` — `ax.plot`, `ax.set_title`, `ax.set_ylim` — instead of the implicit global `plt.plot`, which guesses which axes you meant.

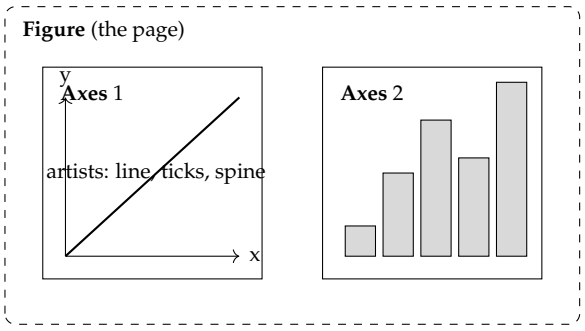


Figure 2.4: The matplotlib hierarchy: a Figure owns Axes; Axes own artists. Everything you print belongs to exactly one of these — ask "which Axes" before every plot call.

HONEST AXES

1. Bars encode length ⇒ the *y*-axis **must include zero**. Truncating a bar axis is a lie of proportion; a viewer reads the 2× visual ratio, not the 4% numeric difference.
2. Lines encode position ⇒ a non-zero axis is defensible *if* the axis is labelled and the scale is stated. The lie is *omission*, not the zoom.
3. Every axis gets a unit; every series gets a label or a legend; every figure gets a number so the text can point at it.

Example 2.3 (WE 2.3 — One dataset, two truths)**.** Strategy: look at what the axis did, not what the line did.

Solution. `code/ch02/two_plots_one_truth.py` plots the same airline growth twice (Figure **??**). The left panel truncates the y-axis at 100: 1949's mean of 127 becomes a squiggle near zero and the +276% growth reads as "explosion". The right panel includes zero: the same +276% reads as "strong, steady". Both panels are accurate. Only one is honest *in context* — and the difference is one line of code: `ax.set_ylim(100, 500)` versus `ax.set_ylim(0, 500)`. The checklist reflex: **before reading any published chart, check where the axis starts.**

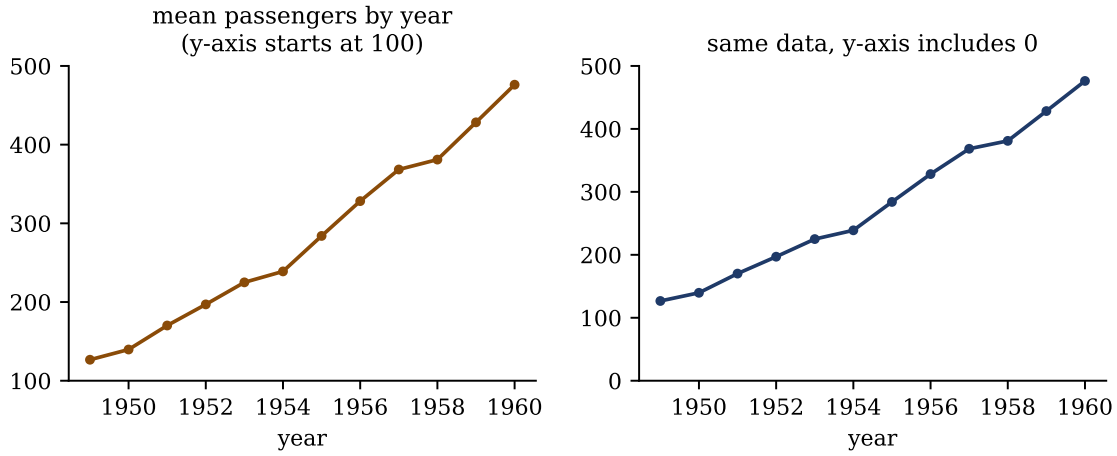


Figure 2.5: WE 2.3 — identical data, two axes (`code/ch02/two_plots_one_truth.py`). Left: y truncated at 100, growth dramatised. Right: y includes 0, same growth honestly proportioned.

Checkpoint 2.2 *Colour vs style.* Your plot will be photocopied, printed in grey, and read by a colour-blind reviewer. Which of these survive? (a) red vs green lines; (b) solid vs dashed; (c) circle vs triangle markers. Design rule: encode categories in *shape or linestyle* first, colour second — Figure **??** uses both (three colours *and* three markers), and degrades gracefully.

2.5 The exploration loop

Exploration is a loop, not a report: *question* → *slice* → *plot* → *new question*. The scatter matrix — every numeric column against every other, diagonal histograms — is the standard first plot because it shows d^2 questions at once.

Example 2.4 (WE 2.4 — Reading the matrix)**.** Strategy: scan for three shapes — clusters (groups you did not model), slopes (relationships), and clouds (variance you will need).

Solution. Figure **??** answers three questions in one glance. (1) *Are the species separable?* Mostly: Gentoo's triangles sit apart on nearly every cell — a classifier will do well. (2) *Which features redundate?* Bill length and flipper length slope together strongly — carrying both is nearly carrying one. (3) *Anything odd?* The bill-depth vs bill-length cell (row 2, col 1) slopes *down* overall yet *up* within each species — keep that in your pocket; §**??** makes it the whole point. The loop continues: the separation by `island` is the next slice, and `sex` the one after — exactly the mechanics of §**??**.

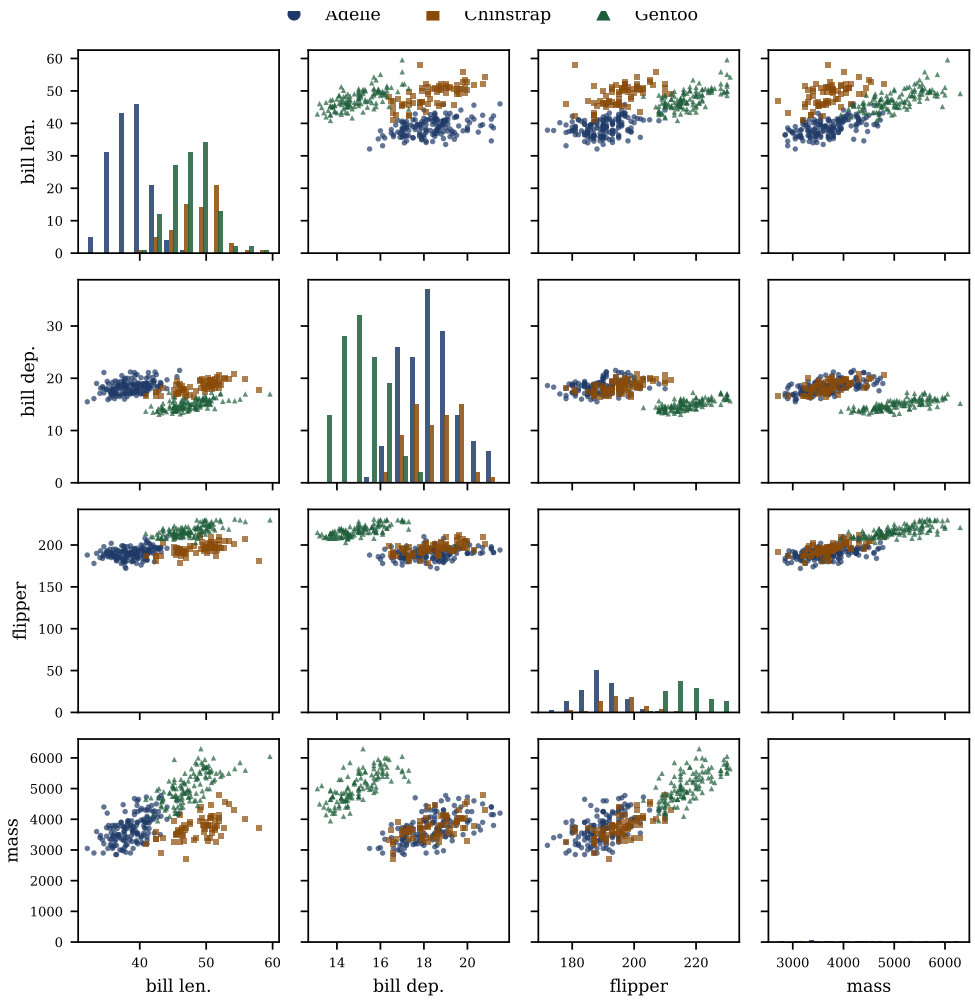


Figure 2.6: Scatter matrix of the four numeric `penguins` columns (`figures/fig_ch02_scatter_matrix.py`). Gentoo separates on nearly every pair; bill depth vs bill length slopes *up* inside each species but *down* when pooled — see §**??**.

2.6 When aggregation lies: Simpson's paradox

Here is the failure mode that ends arguments, audits, and occasionally careers. An aggregate that reverses *every* group-level trend.

```
# code/ch02/simpsons.py -- Simpson's paradox: the pooled rate reverses
# every per-group trend. Synthetic admissions table (Berkeley-style).
# Run from the repository root: python3 code/ch02/simpsons.py
import pandas as pd

# admit, applied per department x sex. Women win BOTH departments, yet lose
# the pooled comparison because they apply mostly to the selective one.
rows = [
    ("History", "men", 640, 800), # 80% -- the easy department
    ("History", "women", 85, 100), # 85%
    ("Computer Sci.", "men", 40, 200), # 20% -- the selective one
```

```
    ("Computer Sci.", "women", 120, 500), # 24%
]
df = pd.DataFrame(rows, columns=["dept", "sex", "admitted", "applied"
    ])
df["rate"] = df.admitted / df.applied

print("per department:")
print(df.pivot(index="dept", columns="sex", values="rate")
     .to_string(float_format=lambda v: f"{v:.1%}"))

pooled = df.groupby("sex")[["admitted", "applied"]].sum()
pooled["rate"] = pooled.admitted / pooled.applied
print("\npooled across departments:")
print(pooled.rate.to_string(float_format=lambda v: f"{v:.1%}"))

print(f"\nverdict: women beat men in BOTH departments "
     f"({df.rate[1]:.1%}>{df.rate[0]:.1%}, {df.rate[3]:.1%}>{df.rate
         [2]:.1%})",
     "yet lose the pooled comparison",
     f"({pooled.rate['women']:.1%} < {pooled.rate['men']:.1%})")
```

Listing 2.3: `code/ch02/simpsons.py`: a two-department admissions table where women win *both* departments and lose the pool.

```
per department:
sex men women
dept
Computer Sci. 20.0% 24.0%
History 80.0% 85.0%

pooled across departments:
sex
men 68.0%
women 34.2%

verdict: women beat men in BOTH departments (85.0%>80.0%, 24.0%>20.0%)
     yet lose the pooled comparison (34.2% < 68.0%)
```

Example 2.5 (WE 2.5 — The pooled lie)**.** Strategy: when a pooled rate contradicts a grouped rate, trust the grouped rate — the pool is a weighted sum, and the weights were rigged by who applied where.

Solution. Listing **??** manufactures the reversal cleanly: women admit at 85% vs men's 80% in History, and 24% vs 20% in Computer Science — and still lose pooled, 34.2% to 68.0%, because 83% of women applied to the harder department. The pooled rate mixes in the application *mix*, not just the admission *rates*. This is not hypothetical: it is the structure of the 1973 Berkeley graduate-admissions analysis [**?**], and of the original 1951 paper that named the phenomenon [**?**]. The rule that survives: **a pooled statistic is a weighted average — always ask what the weights were.**

And it is not just a synthetic curiosity — your own dataset does it:

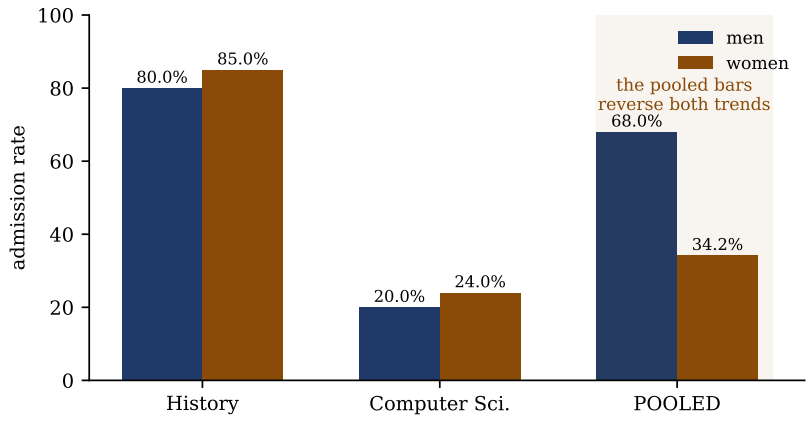


Figure 2.7: WE 2.5 — the reversal, drawn (`figures/fig_ch02_simpsons.py`). Women's bars are taller in both departments; the pooled bars flip. Only the weights changed.

```python
# code/ch02/penguins_corr_flip.py -- a real correlation reversal hiding
# inside the penguins table. Run: python3 code/ch02/penguins_corr_flip.py
import pandas as pd

df = pd.read_csv("data/penguins.csv")
cols = ["bill_length_mm", "bill_depth_mm"]

print("pooled correlation (342 complete pairs):")
print(f" all species together: r = {df[cols].corr().iloc[0, 1]:+.3f}")

print("\nper species:")
for s, g in df.groupby("species"):
    r = g[cols].corr().iloc[0, 1]
    print(f" {s:<9} (n={len(g):3d}): r = {r:+.3f}")

print("\nsame table, three opposite stories -- the aggregate lied.")
```

Listing 2.4: `code/ch02/penguins_corr_flip.py`: pooled vs per-species correlation on `penguins` — a real reversal on real data.

```
pooled correlation (342 complete pairs):
 all species together: r = -0.235

per species:
 Adelie (n=152): r = +0.391
 Chinstrap (n= 68): r = +0.654
 Gentoo (n=124): r = +0.643

same table, three opposite stories -- the aggregate lied.
```

AGGREGATES HIDE DISTRIBUTIONS

A mean is one number; the data was n numbers. Any pooled statistic can reverse per-group truth — so **plot the group breakdown before reporting the pool**. Three reflexes: (a) never report a rate without its n; (b) never report a pooled rate without its group rates; (c) when a slope looks wrong, split by the categorical column and replot — `penguins`' −0.24 pooled correlation is not evidence of "smaller bills go with deeper bills"; it is three species with different shapes stacked on one axis (Figure **??**, row 2 col 1).

DEFINITION OF DONE

`groupby + agg` for the table, `pivot` for the layout, `merge` for the second table, and a zero-based axis for the chart. Four verbs and one reflex carry almost every tabular question you will meet — and §**??** is why the reflex is not optional.

2.7 Common mistakes

Aggregates hiding distributions A mean of 4.2 can be a tight bell or two spikes; a rate can be a reversal. Plot the distribution or the group split before trusting the number — §**??**.

Truncated axes on bars Bar lengths are read as ratios. If the axis starts at 100, a 4% difference looks like 2×. Bars: zero or bust (§**??**).

Pie charts Humans compare angles badly and areas worse. Six slices of a pie is a bar chart that failed. If you must: label slices numerically, never 3-D, never more than five.

`inplace=True` `df.drop(col, inplace=True)`
returns nothing, obscures whether a copy or a view was mutated, and is deprecated upstream. Assign instead:
`df = df.drop(columns=col)`

Chained assignment `df[df.a>0]["b"]=0`
may write to a temporary copy — the `SettingWithCopyWarning` that is easy to silence and expensive to ignore. One step:
`df.loc[df.a>0, "b"] = 0`

Trusting `head()` The first five rows are clean because somebody ordered them to be. Check `df.dtypes`, `df.describe()`, and `df.isna().sum()` — the table's shape of truth — before believing any sample.

`object` dtype hiding mixed types One stray string in a numeric column demotes the whole column to `object`; then means fail *two cells later*. After loading:
`pd.to_numeric(col, errors="coerce")`
and count the new NaNs — they were the strings.

Aggregating before checking the index `groupby` on a column with invisible whitespace (`"First"` vs `"First "`) makes two groups of one class. `value_counts` the key before you trust the table.

Forgetting the weights A pooled average over unequal groups weights by n silently. Sometimes that is what you want (overall rate); sometimes not (per-group rate). §**??** showed the difference is not academic.

2.8 Summary table

Want	Idiom	Shape
subset rows	`df[df.fare>100]` / `df.loc[mask]`	(n', d)
subset cols	`df[["age","fare"]]` / `df.iloc[:,:3]`	(n, d')
per-group stat	`df.groupby(k).agg(mean, count)`	(k, m)
grouped matrix	`df.pivot_table(index, columns, values, aggfunc)`	(r, c)
long → wide	`df.pivot(index, columns, values)`	reshape
wide → long	`wide.melt(...)`	reshape
two tables	`df.merge(other, on=key)`	$(n, d+d_2)$
first plot	scatter matrix of numeric cols	d^2 panels
honest axis	`ax.set_ylim(0, ...)` for bars	reflex
audit a pool	group first, then compare pooled vs grouped	reflex

Table 2.1: Chapter 2 cheat sheet: the four verbs and the two reflexes.

Exercises

Exercise 2.1 (easy)**.** Load `data/titanic.csv` and print `df.dtypes`. Which columns are `float64` that "should" be integers, and what does that tell you about their missing data? Verify with `isna().sum()`.

Exercise 2.2 (easy)**.** Mean `fare` by `class` in one line. Then count rows per class — is the mean over n=216 First-class or n=491 Third-class passengers more trustworthy, and why does the n matter?

Exercise 2.3 (easy)**.** Pivot `flights` to wide and pull the 1955 row. Then `melt` the wide table back and sort — do you recover the original rows for 1955 exactly?

Exercise 2.4 (easy)**.** In Figure **??**, which single argument creates the lie? Write the honest version for a *bar* chart of the same data — does the rule change?

Exercise 2.5 (easy)**.** `df.loc[0, "age"]` vs `df.iloc[0, 0]` — what is different about what each guarantees? When does `loc` fail that `iloc` cannot?

Exercise 2.6 (medium)**.** Survival by `embark_town` × `class`: groupby the two keys, then `unstack`. Which port's First-class rate looks anomalous — and what does the `n` column say about trusting it?

Exercise 2.7 (medium)**.** Write the misleading bar chart yourself: plot survival by class with `ylim(0.4, 0.7)`. Then fix it. Which visual feature did the truncation distort — bar length, spacing, or colour?

Exercise 2.8 (medium). `groupby().transform("mean")` returns a column of the *original* length, not the group count. Use it to compute each passenger's *deviation from their class mean fare*, then explain why `agg` could not have produced that column.

Exercise 2.9 (medium). `flights`: compute each year's share of the decade's passengers with one `merge` of a grouped sum. Then do it again with `transform`. Which result is long-form, which is wide?

Exercise 2.10 (medium). `flights`: add a 12-month rolling mean of `passengers`

```
fl.passengers.rolling(12).mean()
```

plot it over the raw series, and say in one sentence what the rolling mean hides that the raw series shows.

Exercise 2.11 (medium). `penguins`: pooled `body_mass_g` mean vs per-species means. Does the pooled number describe any actual penguin? Whose mean is it closest to, and is that luck or structure?

Exercise 2.12 (hard). Construct a third department in Listing **??** such that women win all *three* departments and lose the pooled comparison by more than 40 points. Verify the pooled rates numerically.

Exercise 2.13 (hard). Prove the reversal mechanically: the pooled rate for sex s is $\sum_d w_{s,d}\, r_{s,d}$ with $w_{s,d} = \text{applied}_{s,d}/\text{applied}_s$. Show that flipping the weight *mix* (not the rates) is necessary and sufficient for a Simpson reversal.

Exercise 2.14 (hard). Repair the pooled comparison without pooling: standardise both sexes' rates at a *common* application mix (e.g. the union-average department weights). Compute the standardised rates for Listing **??** — who "really" admits better?

Exercise 2.15 (hard). `penguins`: the flip of Listing **??** happens because species have different bill shapes *and* different prevalence. Show numerically that reweighting the three species' counts equally changes the pooled correlation — find weights that flip its sign back.

Exercise 2.16 (hard). Reproduce the chained-assignment bug:

```
df[df.fare>100]["cheap"] = 1
```

then inspect whether `df` changed. Fix it with `loc` in one line, and say when the warning is a false positive.

Further reading

- pandas documentation, "10 minutes to pandas", https://pandas.pydata.org/docs/user_guide/10min.html. The orientation lap — indexing, selection, grouping — run it once, keep it bookmarked.
- pandas documentation, *Reshaping and pivot tables*, https://pandas.pydata.org/docs/user_guide/reshaping.html. The official `pivot` / `melt` / `stack` / `unstack` reference — §**??** is the first two floors of this page.

- pandas documentation, *Merge, join, concatenate*, https://pandas.pydata.org/docs/user_guide/merging.html. All join types with honest diagrams; the one-paragraph version is our `merge` idiom.
- pandas documentation, `DataFrame.groupby` reference, https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html. The full verb set — `agg`/`transform`/`filter` — worth one slow read.
- matplotlib, "Anatomy of a figure", https://matplotlib.org/stable/gallery/showcase/anatomy.html. The Figure/Axes/artist map with every part labelled — our Figure **??** with the real names.
- matplotlib, *Usage guide*, https://matplotlib.org/stable/tutorials/introductory/usage.html. The official first-contact tutorial; pairs with the Schafer video below.
- seaborn, *Official tutorial*, https://seaborn.pydata.org/tutorial.html. The next step after this chapter — statistical plots on top of the same DataFrames; read the "plotting functions" section, keep the rest for later.
- C. O. Wilke, *Fundamentals of Data Visualization* (free online) [**?**], https://clauswilke.com/dataviz/. The honest-visualisation book — chapters 5 (directory of visualisations) and 24 (misleading axes) are this chapter's rules with the full theory.
- H. Wickham, "Tidy data", *Journal of Statistical Software* 59(10) [**?**], https://www.jstatsoft.org/article/view/v059i10. The long-vs-wide paper — one rule (variable = column, observation = row) that organises every reshape decision.
- C. T. Bergstrom and J. D. West, *Calling Bullshit* course materials [**?**], https://www.callingbullshit.org/. The citizen's guide to misleading charts and statistics; the "misleading axes" lecture is a sharper, longer §**??**.

WATCH INSTEAD

Best first. "How to spot a misleading graph" — TED-Ed, 4 min — https://www.youtube.com/watch?v=E91bGT9BjYk. Lea Gaslowitz's catalogue of axis crimes — the animated version of §**??** and §**??**; watch before reading a single published chart. "Python Pandas Tutorial (Part 1): Getting Started with Data Analysis" — Corey Schafer, 23 min — https://www.youtube.com/watch?v=ZyhVh-qRZPA. Loading a CSV and the first `DataFrame` moves — matches §**??** and §**??**; uses Jupyter where this book uses scripts, the pandas is identical. "Matplotlib Tutorial (Part 1): Creating and Customizing Our First Plots" — Corey Schafer, 35 min — https://www.youtube.com/watch?v=UO98lJQ3QGI. The `plt.plot` workflow of §**??** from zero; he uses the implicit-style API — translate each call to `ax.` methods as you watch (the chapter's rule, not his). Access date 2026-10-11.

Notes

1. Data sources: `titanic.csv` is the 1912 passenger manifest redistributed in the `seaborn-data` collection [**?**] (891 rows, 15 columns; we read the copy in `data/`); `flights.csv` is the Box–Jenkins airline-passenger series (1949–1960, 144 rows) [**?**]; `penguins.csv` as in chapter 1 [**?**, **?**].
2. The tidy-data rule quoted in §**??** (variables as columns, observations as rows) is Wickham's formulation [**?**]; it is a design guideline, not a theorem — some algorithms want wide, some reports want wide, the working store stays long.
3. The Simpson's paradox example in §**??** is synthetic but calibrated to the Berkeley case: the real 1973 graduate-admissions analysis is Bickel, Hammel and O'Connell [**?**]; the phenomenon's first formal statement is Simpson 1951 [**?**]. The penguins correlation flip is measured on the real table — Listing **??**.
4. Visualisation guidance in §**??** follows Wilke [**?**] ch. 24 and the *Calling Bullshit* treatment of misleading axes [**?**]; the bar-chart zero rule is the consensus rule, not universal law — line charts and index-series are routinely (and fairly) zoomed.
5. Version pins for this chapter's runs: Python 3.9, pandas 2.3.3, matplotlib 3.9.4 — recorded in `code/README.md`; all output blocks are verbatim captures, regenerated by the scripts.

Hints and answers

2.1 `age` is `float64` because NaN is not an integer; `pclass` has no holes so it stays `int64`. The dtype is a missingness report.

2.2 `df.groupby("class")["fare"].mean()` and `.count()`. Third class is the bigger n; the mean over 491 people is the stabler estimate — variance of a mean shrinks as $1/n$.

2.3 The call

```
fl.pivot(index="year", columns="month", values="passengers")
```

gives the grid; the 1955 row should read January-through-December for that year. `melt` recovers the rows up to sort order — months come back alphabetically, not chronologically.

2.4 `ax.set_ylim(100, 500)` — one argument. For bars the rule is stricter: *always* include zero, because bars are read as lengths.

2.5 `loc` uses labels (row whose index is `0`); `iloc` uses position (first row). If the index is a shuffled or non-integer index, `loc[0]` raises `KeyError` where `iloc[0]` always works.

2.6 The chain

```
df.groupby(["embark_town", "class"])["survived"].mean()
```

then `.unstack()`. Cherbourg's First-class cabin looks high — check the counts before calling it a port effect; tiny n groups swing wide.

2.7 `ax.bar(...)` then `ax.set_ylim(0.4, 0.7)`: the bars' *lengths* now misstate the ratio — spacing and colour were untouched; only the length encoding broke. Fix: `ylim(0, 1)` or `ylim(bottom=0)`.

2.8 `transform` broadcasts each group's aggregate back to its rows: `df.fare` minus `df.groupby("class").fare.transform("mean")`. `agg` shrinks to one row per group; it cannot answer a per-row question.

2.9 `fl.merge(...)` with `fl.groupby("year").passengers.sum()` on `year`, then divide. The merge version is long-form (one row per month still); `transform` gives the same long column without a join.

2.10 `fl.passengers.rolling(12).mean()`, plotted over raw. The rolling mean hides seasonality — the exact thing that makes the series interesting.

2.11 Pooled $\approx$ 4202 g — no species has that mean (Adelie $\approx$ 3701, Chinstrap $\approx$ 3733, Gentoo $\approx$ 5076). The pool describes the mixture, not any penguin; it sits nearest the middle-sized groups by count, not by similarity.

2.12 Give the new department rates women still win (e.g. women 60% vs men 50%) but route almost all men's applications into it — pooled men rises, pooled women stays low. Check `pooled.rate` numerically after each edit.

2.13 Decompose: $\text{pooled}_s = \sum_d w_{s,d} r_{s,d}$. Women can win every $r_{s,d} > r_{m,d}$ and lose when $w_{w,\cdot}$ concentrates on the dept where *both* rates are low. The weights are the whole trick — rates alone cannot reverse it.

2.14 Standardised rate $= \sum_d \bar{w}_d \, r_{s,d}$ with common weights $\bar{w}_d$ (e.g. the all-applicants department shares). Compute it: both sexes compared on the *same* mix — women still ahead.

2.15 Weight the three species' correlations by their sample shares; the negative pooled value comes from between-species displacement (Gentoo is high-length/low-depth, Adelie the reverse). Equal weights move the estimate toward the within-species sign.

2.16 `df.loc[df.fare>100, "cheap"] = 1`. The warning is a false positive when the slice is provably a copy — but you cannot see which it was, so `loc` is the only honest way.

CHAPTER 3

Supervised learning and honest evaluation

Every supervised question collapses to two decisions: *what are we predicting*, and *how will we know it worked*. This chapter installs the whole frame — dataset, model, loss, split, metric — and powers it with the two simplest classifiers in the canon: k-nearest neighbours (memorise, then vote) and naïve Bayes (multiply little probabilities, then compare). Neither needs a library call to explain, so both get written from scratch below. The rest of the chapter is about the harder discipline: evaluation. A number on held-out data, computed under a protocol that never let the model peek, is the only currency this book trades in. By the end you will have a harness — split, fit, predict, score — that every later chapter reuses unchanged.

3.1 The supervised problem

Definition 3.1 (Supervised dataset)**.** A *supervised dataset* is $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$: n pairs of a feature row $\mathbf{x}_i \in \mathbb{R}^d$ and a label y_i. In the matrix view of Chapter 1, $\mathcal{D}$ is an $X \in \mathbb{R}^{n \times d}$ together with a $\mathbf{y} \in \mathbb{R}^n$ (regression) or $\mathbf{y} \in \{0, 1, \ldots, C-1\}^n$ (classification).

A *model* is a function $f_{\boldsymbol{\theta}} : \mathbb{R}^d \to \hat{y}$ chosen from a family by adjusting parameters $\boldsymbol{\theta}$ to fit $\mathcal{D}$. Training is the fit; evaluation is checking the fit *on rows the fit never saw*. That last clause is the entire game, and the whole chapter turns on it.

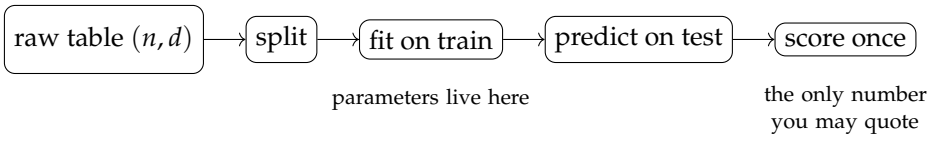


Figure 3.1: The supervised pipeline. Everything between *split* and *score* must be ignorant of the test rows; the score is computed once, at the end.

THE TEST SET IS A SEALED ENVELOPE

Fitting, tuning, and feature choices all consume training data only. The test set is touched exactly once, after all decisions are frozen. Every touch before that spends part of its meaning — and you cannot buy it back.

3.2 k-NN: non-parametric memory

k-nearest neighbours has no parameters at all — the model *is* the training set plus a distance. To predict: find the k training rows closest to $\mathbf{x}$ under some metric (euclidean by default) and take the majority label. Three consequences follow immediately.

First, *scaling matters*: a distance in mixed units is a distance in whichever unit is largest — standardise (§**??**) or the `fare` column drowns `age`. Second, *k is a knob between memorising and averaging*: k=1 reproduces the training set perfectly and generalises worst; large k smooths over real structure. Third, in d dimensions "nearest" stops meaning much — points crowd the edges of the hypercube and all distances converge; more than a curiosity, it is why the later chapters keep dimension low or add structure.

Listing **??** implements it in five lines: the $\|\mathbf{a}-\mathbf{b}\|^2 = \|\mathbf{a}\|^2 - 2\mathbf{a}^T\mathbf{b} + \|\mathbf{b}\|^2$ expansion turns the neighbour search into one matrix product, majority vote is one `bincount`.

```
# code/ch03/knn_scratch.py -- k-NN from scratch on the real digits set.
# Run from the repository root: python3 code/ch03/knn_scratch.py
import numpy as np
from sklearn.datasets import load_digits

X, y = load_digits(return_X_y=True)
rng = np.random.default_rng(11)
perm = rng.permutation(len(y))
tr, te = perm[:1400], perm[1400:] # 1400 train / 397 test

def knn_predict(Xtr, ytr, Xte, k):
    # squared euclidean via the (a-b)^2 = a^2 - 2ab + b^2 expansion
    d2 = (Xte**2).sum(1)[:, None] - 2 * Xte @ Xtr.T + (Xtr**2).sum(1)[None, :]
    nn = ytr[np.argsort(d2, axis=1)[:, :k]]
    # majority vote per row
    return np.array([np.bincount(row, minlength=10).argmax() for row in nn])

print(f"{'k':>4} {'train_acc':>10} {'test_acc':>10}")
for k in [1, 3, 5, 11, 25]:
    acc_tr = (knn_predict(X[tr], y[tr], X[tr], k) == y[tr]).mean()
    acc_te = (knn_predict(X[tr], y[tr], X[te], k) == y[te]).mean()
    print(f"{k:>4} {acc_tr:>10.3f} {acc_te:>10.3f}")
```

Listing 3.1: `code/ch03/knn_scratch.py`: kNN from scratch on `digits` (1400 train / 397 test).

```
   k train_acc test_acc
   1     1.000    0.982
   3     0.996    0.980
   5     0.990    0.980
  11     0.987    0.965
  25     0.974    0.947
```

Example 3.2 (WE 3.1 — Reading the k sweep)**.** Strategy: the train–test *gap* is the signal, not either number alone.

Solution. At k=1 train accuracy is a perfect 1.000 — the model answers every training row with itself — while test accuracy is 0.982: the gap is pure memorisation. As k grows the gap narrows (k=5: 0.990 vs 0.980) and then the whole curve sinks (k=25: 0.974/0.947) as smoothing eats real structure. The best test score lives at the sweet middle, $k \in \{1, \ldots, 5\}$, where the boundary still flexes but no single training point is sovereign.

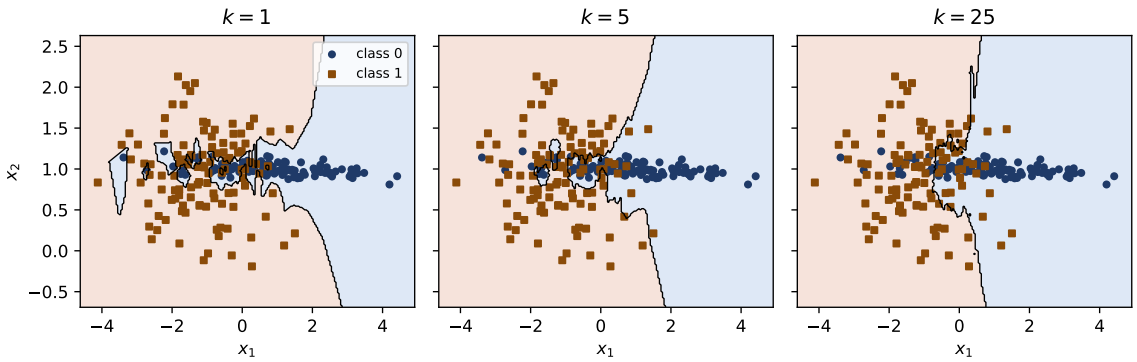


Figure 3.2: Decision boundaries at k=1, 5, 25 on two-feature synthetic data (`figures/fig_ch03_knn_boundaries.py`). k=1 traces every bump; k=25 draws a nearly straight frontier; k=5 is the compromise that usually wins.

3.3 Naïve Bayes: multiply small probabilities

Bayes' rule gives the probability of class c given features $\mathbf{x}$: $P(c \mid \mathbf{x}) \propto P(c)\, P(\mathbf{x} \mid c)$. Naïve Bayes assumes conditional independence — $P(\mathbf{x} \mid c) \approx \prod_j P(x_j \mid c)$ — turning a d-way joint density into d independent one-way densities you can estimate from counts or Gaussians. The assumption is wrong almost always and useful surprisingly often.

Two engineering details matter. Likelihoods multiply to astronomically small numbers, so work in *log-space*: multiply becomes add, underflow becomes harmless. And Gaussian likelihoods need a variance floor ($+\varepsilon$) or a constant column produces a division by zero.

```
# code/ch03/nb_scratch.py -- Gaussian naive Bayes in log-space on titanic.
# Numeric columns get Gaussian likelihoods; sex gets a categorical one.
# Run from the repository root: python3 code/ch03/nb_scratch.py
import numpy as np
import pandas as pd

df = pd.read_csv("data/titanic.csv")
y = df.survived.values
Xn = df[["age", "fare"]].fillna(df[["age", "fare"]].median()).values
sex = df.sex.values

rng = np.random.default_rng(5)
perm = rng.permutation(len(y))
tr, te = perm[:700], perm[700:]
```

```
15
16 # --- fit: per-class mean/var of each numeric column + per-class sex
       rates ---
17 classes = np.unique(y)
18 log_prior = {c: np.log((y[tr] == c).mean()) for c in classes}
19 mu = {c: Xn[tr][y[tr] == c].mean(0) for c in classes}
20 var = {c: Xn[tr][y[tr] == c].var(0) + 1e-3 for c in classes} # +eps
       for safety
21 sex_rate = {c: {s: (sex[tr][y[tr] == c] == s).mean()
22             for s in np.unique(sex)} for c in classes}
23
24 def predict(i):
25    x, s = Xn[i], sex[i]
26    def loglik(c):
27       ll = log_prior[c]
28       ll += (-0.5 * np.log(2 * np.pi * var[c])
29            - (x - mu[c]) ** 2 / (2 * var[c])).sum()
30       ll += np.log(sex_rate[c][s] + 1e-9)
31       return ll
32    return max(classes, key=loglik)
33
34 pred = np.array([predict(i) for i in te])
35 tp = int(((pred == 1) & (y[te] == 1)).sum())
36 fp = int(((pred == 1) & (y[te] == 0)).sum())
37 fn = int(((pred == 0) & (y[te] == 1)).sum())
38 tn = int(((pred == 0) & (y[te] == 0)).sum())
39 print("confusion matrix (rows=true, cols=pred):")
40 print(f" pred=0 pred=1")
41 print(f" true=1 {fn:4d} {tp:4d}")
42 print(f" true=0 {tn:4d} {fp:4d}")
43 print(f"\naccuracy {(pred == y[te]).mean():.3f} "
44     f"precision {tp/(tp+fp):.3f} recall {tp/(tp+fn):.3f}")
```

Listing 3.2: `code/ch03/nb_scratch.py`: Gaussian + categorical naïve Bayes in log-space on `titanic` (700 train / 191 test).

```
confusion matrix (rows=true, cols=pred):
         pred=0 pred=1
  true=1   24   48
  true=0   97   22

accuracy 0.759 precision 0.686 recall 0.667
```

Example 3.3 (WE 3.2 — The confusion matrix by hand)**.** Strategy: accuracy is one cell of a 2×2 story; read all four.

Solution. Listing **??** outputs tp=48, fp=22, fn=24, tn=97. Accuracy $(48{+}97)/191 = 0.759$. Precision $48/(48{+}22) = 0.686$: of everyone NB flags as a survivor, two-thirds really survived. Recall $48/(48{+}24) = 0.667$: it misses one-third of actual survivors. Same model, three different reputations — which one is "good" depends on whether you are allocating lifeboats or writing a newspaper.

3.4 The split, and why it is touched once

The default protocol: *stratified* 80/20 split — train on 80%, holding the class ratio fixed in both halves, then score on 20%. Stratification is not cosmetic: on a 99:1 problem an unlucky random split can put almost all positives in one half, and you learn a lie about a model that never had a chance. Listing **??** is the book's canonical harness; every later chapter calls something shaped exactly like it.

```
# code/ch03/eval_protocol.py -- the canonical split + metric harness,
# reused by every later chapter. Stratified 80/20, confusion-based metrics.
# Run from the repository root: python3 code/ch03/eval_protocol.py
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X, y = load_breast_cancer(return_X_y=True) # y=1 benign, y=0 malignant
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.2, random_state=7, stratify=y)
print(f"split: {len(ytr)} train / {len(yte)} test "
      f"(positive rate {ytr.mean():.3f} both folds)")

model = LogisticRegression(max_iter=5000).fit(Xtr, ytr)
pred = model.predict(Xte)

def confusion(pred, y):
    tp = int(((pred == 1) & (y == 1)).sum())
    fp = int(((pred == 1) & (y == 0)).sum())
    fn = int(((pred == 0) & (y == 1)).sum())
    tn = int(((pred == 0) & (y == 0)).sum())
    return tp, fp, fn, tn

tp, fp, fn, tn = confusion(pred, yte)
prec = tp / (tp + fp)
rec = tp / (tp + fn)
f1 = 2 * prec * rec / (prec + rec)
print(f"[[{tn:>3} {fp:>3}]")
print(f" [{fn:>3} {tp:>3}]]")
print(f"accuracy {(pred == yte).mean():.3f}")
print(f"precision {prec:.3f} (of predicted-positive, how many truly are)")
print(f"recall {rec:.3f} (of truly-positive, how many we caught)")
print(f"F1 {f1:.3f}")
# the imbalance lesson: predicting everyone 'benign' also scores high
lazy_acc = yte.mean()
print(f"\nlazy 'all benign' baseline accuracy: {lazy_acc:.3f}")
print("F1 is the honest metric here -- a cancer missed is not a tie.")
```

Listing 3.3: `code/ch03/eval_protocol.py`: stratified split, logistic fit, confusion-based metrics on `breast_cancer`.

```
split: 455 train / 114 test (positive rate 0.626 both folds)
[[ 41 1]
 [ 4 68]]
accuracy 0.956
precision 0.986 (of predicted-positive, how many truly are)
recall 0.944 (of truly-positive, how many we caught)
F1 0.965

lazy 'all benign' baseline accuracy: 0.632
F1 is the honest metric here -- a cancer missed is not a tie.
```

3.5 Cross-validation

One split is one draw from a lottery — run it again with a different seed and the score moves by a point or two for free. k-fold cross-validation prices the model properly: partition $\mathcal{D}$ into k folds, take turns training on $k-1$ and scoring on the held-out one, then report mean $\pm$ sd across the k scores. The 5-fold or 10-fold mean is a better estimate of generalisation than any single split; the sd tells you how much the estimate would wobble under re-sampling.

Three variants earn their keep:

- *Stratified*: preserve the class ratio per fold — always on imbalanced data.
- *Grouped* (`GroupKFold`): keep all rows from one group — patient, hospital, user — in one fold. Without it, near-duplicate rows of the same entity straddle the boundary and the score leaks.
- *Time-aware*: score each fold on data *later* than its training window — the only honest choice when the row order is a timestamp; random folds let the future train the past.

CV costs k refits and still spends its test meaning a little every time you tune on it: once you have chosen hyperparameters by CV, the honest protocol is a *final* untouched test set.

3.6 Metrics for regression

For regression, four error names recur; know what each pays attention to.

- MAE $= \frac{1}{n}\sum|y_i - \hat{y}_i|$ — the mean miss; robust, interpretable, in the units of y.
- MSE $= \frac{1}{n}\sum(y_i - \hat{y}_i)^2$ — penalises big misses quadratically; smooth for optimisation.
- RMSE $= \sqrt{\text{MSE}}$ — back in y's units while keeping the quadratic taste.
- $R^2 = 1 - \frac{\sum(y_i-\hat{y}_i)^2}{\sum(y_i-\bar{y})^2}$ — the fraction of variance explained; 1 perfect, 0 means you tied the constant-mean model, negative means you did *worse* than predicting the average.

Checkpoint 3.1 *A number that needs a baseline.* R^2's definition reveals a habit worth keeping: *every metric needs a stated baseline*. 0.90 is brilliant against the mean and embarrassing against last year's model. Quote the baseline alongside the score or the score is marketing.

3.7 Metrics for classification

	predicted −	predicted +
actual −	TN	FP
actual +	FN	TP

Figure 3.3: The confusion matrix: the four numbers every classification metric is made of. Rows are truth, columns are predictions.

Everything in the section is algebra on the four cells of Figure **??**:

$$\text{accuracy} = \frac{tp + tn}{tp + fp + fn + tn} \qquad \text{precision} = \frac{tp}{tp + fp} \qquad \text{recall} = \frac{tp}{tp + fn}$$

$$\text{FPR} = \frac{fp}{fp + tn} \qquad \text{TPR} = \text{recall} \qquad F_1 = \frac{2\,P\,R}{P + R}$$

Precision and recall trade off through the decision threshold: raise it and precision climbs while recall falls. ROC curves trace all thresholds at once — TPR vs FPR — and AUC summarises the whole trade-off as one number (0.5 is chance, 1.0 is perfect, 0.995 on our `breast_cancer` split). When classes are imbalanced, accuracy becomes a con: predicting everyone benign on a 62.6%-benign set already scores 0.632. The chapter's rule of thumb: on imbalanced problems lead with F_1, precision, and recall — and always name the lazy baseline.

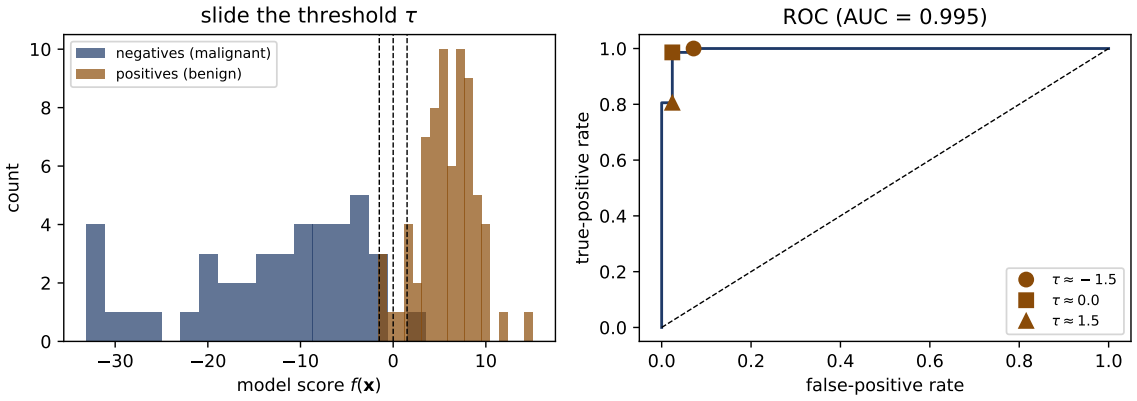


Figure 3.4: ROC construction (`figures/fig_ch03_roc_steps.py`): a threshold sweeps the score distribution (left); each threshold produces one (FPR, TPR) point (right). AUC = 0.995 measured on `breast_cancer`.

3.8 Bias, variance, and the learning curve

Definition 3.4 (Bias and variance)**.** *Bias* is systematic error — the model is too simple to express the pattern (underfit). *Variance* is instability — the model is so flexible it reshapes itself to noise (overfit). Diagnose the pair from the train–test gap: big gap, variance; both bad, bias.

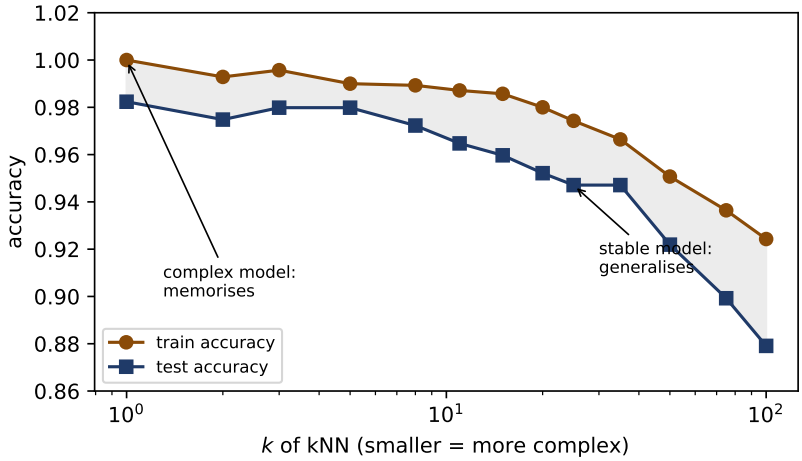


Figure 3.5: Train vs test accuracy as k of kNN varies (`figures/fig_ch03_biasvariance.py`): the grey wedge is the generalisation gap. Complexity lives at the left — k=1 memorises perfectly and generalises worst.

The learning curve — accuracy against training-set size — tells you which medicine to take. If the two curves still converge as data grows, more data helps (variance-limited); if they have both plateaued at a mediocre level, more data is wasted effort (bias-limited — get features or a better model family, not more rows).

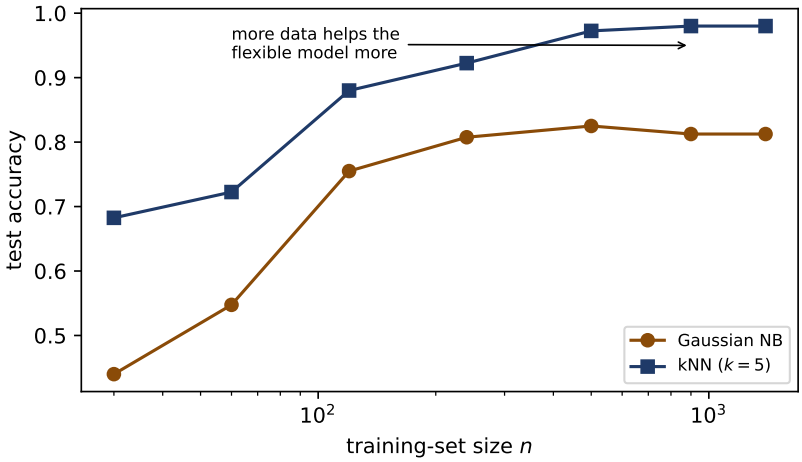


Figure 3.6: Learning curves on `digits` (`figures/fig_ch03_learning_curve.py`). Gaussian NB plateaus near 0.81–0.83 — its independence assumption is a bias ceiling more data cannot break. 5-NN keeps climbing: 0.68 → 0.88 → 0.97 → 0.98 at $n = 30, 120, 500, 1397$.

Example 3.5 (WE 3.3 — Which medicine?)**.** Strategy: the shape of the learning curve picks the remedy.

Solution. Figure **??** shows NB flattening by $n{\approx}500$: its plateau is bias, so add a model family (trees, linear models), not rows. 5-NN at $n{=}1397$ still has a rising slope — spend effort collecting more data or running CV to pick k, and the score should keep improving. Diagnose the plateau *before* you collect — collecting is the most expensive fix.

3.9 Leakage: the five ways it sneaks in

Leakage is test information — or future information, or the label itself — reaching the model at training time. It does not announce itself; it looks like a great score. Five recurring routes:

1. *Duplicate or near-duplicate rows* across the boundary (Chapter 1's fake win).
2. *Preprocessing before the split* — a scaler, imputer, or feature-selector that read the test rows.
3. *Group bleed* — siblings of the same patient/user/day on both sides; solved by grouped splits.
4. *Target-encoded features* computed on the full data — the encoding literally contains the test labels.
5. *Time leakage* — features computed with a window reaching past the prediction timestamp.

```
# code/ch03/leak_demo.py -- two leaks, two sizes, both measured.
# (a) Scale on the FULL data before splitting -- the textbook example.
# (b) Test rows inside the training set -- the guaranteed phantom.
# Run from the repository root: python3 code/ch03/leak_demo.py
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X, y = load_breast_cancer(return_X_y=True)
ks = np.arange(1, 30, 2)

# --- (a) standardise-then-split vs split-then-standardise
    --------------
Xs = (X - X.mean(0)) / X.std(0) # scaler has seen the test rows
Xtr, Xte, ytr, yte = train_test_split(
   Xs, y, test_size=0.2, random_state=7, stratify=y)
leaky = max((KNeighborsClassifier(k).fit(Xtr, ytr).score(Xte, yte), k)
         for k in ks)

Xtr, Xte, ytr, yte = train_test_split(
   X, y, test_size=0.2, random_state=7, stratify=y)
mu, sd = Xtr.mean(0), Xtr.std(0)
honest = max((KNeighborsClassifier(k).fit((Xtr - mu) / sd, ytr)
           .score((Xte - mu) / sd, yte), k) for k in ks)
print(f"(a) scale->split {leaky[0]:.3f} at k={leaky[1]} "
```

```
    f"split->scale {honest[0]:.3f} at k={honest[1]} "
    f"diff {(leaky[0]-honest[0])*100:+.1f} pp")
print(" tiny here: iid rows share statistics, so the leaked scaler")
print(" barely differs from the honest one. Leak size is not fixed --")
print(" it scales with how much test information the step consumed.")

# --- (b) test rows copied into training
print("\n(b) fraction of test set duplicated into training:")
for frac in [0.0, 0.1, 0.25, 0.5]:
    n_leak = int(frac * len(Xte))
    Xtr_l = np.vstack([Xtr, Xte[:n_leak]]) # test rows join training!
    ytr_l = np.concatenate([ytr, yte[:n_leak]])
    m = KNeighborsClassifier(1).fit(Xtr_l, ytr_l)
    acc = m.score(Xte, yte)
    print(f" {frac:.0%} leaked -> 1-NN test acc {acc:.3f}")
print(" 1-NN answers each leaked row with itself: distance 0, label")
print(" memorised. Every leak ultimately works this way -- the model")
print(" is given an answer key, and 'accuracy' stops meaning anything.")
```

Listing 3.4: `code/ch03/leak_demo.py`: two leaks, two sizes — and the honest measurement that leak size *depends on how much test information the step consumed*.

```
(a) scale->split 0.974 at k=11 split->scale 0.982 at k=5 diff -0.9 pp
   tiny here: iid rows share statistics, so the leaked scaler
   barely differs from the honest one. Leak size is not fixed --
   it scales with how much test information the step consumed.

(b) fraction of test set duplicated into training:
   0% leaked -> 1-NN test acc 0.921
   10% leaked -> 1-NN test acc 0.939
   25% leaked -> 1-NN test acc 0.956
   50% leaked -> 1-NN test acc 0.965
   1-NN answers each leaked row with itself: distance 0, label
   memorised. Every leak ultimately works this way -- the model
   is given an answer key, and 'accuracy' stops meaning anything.
```

Example 3.6 (WE 3.4 — Leak autopsy)**.** Strategy: run the leak and the honest protocol side by side; measure the phantom instead of assuming it.

Solution. Protocol (a) is the textbook "scale on everything first" leak — and on this clean iid data it *loses* 0.9 pp rather than gaining. The lesson is the finer one: a mild leak's size is proportional to how much test information the leaked statistic absorbed, and on iid splits that is nearly nothing — which is exactly why mild leaks are dangerous, they hide. Protocol (b) shows the leak at full strength: copying half the test set into training buys 1-NN a free $+4.4$ pp ($0.921 \to 0.965$). Same mechanism, different doses — every leak is the model peeking at an answer key, and the score stops meaning what it says.

Example 3.7 (WE 3.5 — The imbalance rerun)**.** Strategy: ask "good compared to what?" and build the lazy classifier the score must beat.

Solution. On `breast_cancer` a logistic regression scores 0.956 accuracy — sounding great until the lazy "everyone benign" model scores 0.632 without touching a feature. The model's real margin is the gap, and the honest metrics are precision 0.986, recall 0.944, $F_1 = 0.965$: of 72 true positives it caught 68 — the 4 misses are the 4 diagnoses the metric was invented to count. On a 99:1 problem the lazy baseline scores 0.990 and accuracy alone tells you nothing at all.

DEFINITION OF DONE

What you can now do: state the supervised frame ($\mathcal{D}$, f_θ, loss, split, metric) and recite why the test set is sealed; implement kNN and naïve Bayes from scratch and read a k-sweep for bias/variance; build stratified, grouped, and time-aware CV; compute the confusion quartet and choose the right metric under imbalance; diagnose plateau shape on a learning curve; autopsy a leak by measuring the phantom.

3.10 Common mistakes

Tuning on the test set Every peek spends meaning. Tune on train+CV; the test set hears about your choices only after they are frozen.

Preprocessing before the split Fit scalers, imputers, and encoders on train only; inside CV, refit them inside each fold — a pipeline object exists precisely so you cannot forget.

Trusting accuracy on imbalance A 0.99 score on a 99:1 problem is the lazy classifier wearing a costume. Report precision, recall, F_1 — and always the lazy baseline.

Random folds on grouped or timed data Siblings of the same entity straddle the boundary, or the future trains the past. Use `GroupKFold` or a time-ordered split.

Reading one split as the truth A single 80/20 score is one lottery draw. CV mean $\pm$ sd is the honest number.

Micro-averaging away the small class Overall accuracy or micro-F_1 can rise while the minority class is missed entirely — report per-class recall too.

Skipping the baseline Every score needs a named baseline next to it — mean predictor, majority class, last year's model. Without one the score is a number without a denominator.

k too small k=1 wins every training argument and loses the deployment. Read the sweep, not just the peak.

Probability out of thin air Naïve Bayes without the log-space fix underflows to zero and returns the wrong argmax — multiply probabilities only inside `log`.

3.11 Summary table

Idea	Tool	When to reach for it
Supervised frame	$\mathcal{D}$, f_θ, loss, split	every problem — state it first
kNN	`KNeighborsClassifier`	small-n baseline; distances meaningful
Naïve Bayes	log-space likelihoods	fast baseline; sparse/high-d text
Split	stratified 80/20	the default single evaluation
CV	k-fold, stratified/grouped/time	the honest score for tuning
Regression metrics	MAE, RMSE, R^2	error size vs variance explained
Classification metrics	P/R/F_1, ROC-AUC	imbalance; threshold trade-offs
Bias–variance	train–test gap, learning curve	pick medicine: data vs features
Leakage	five routes checklist	every pipeline, before quoting a score

3.12 Drill — five concept questions

1. k=1 gives perfect train accuracy. Which of bias or variance is at work, and what does the train–test gap measure?
2. Name two splits that beat a random 80/20 when rows come from patients or timestamps.
3. On 99:1 imbalance, accuracy reads 0.991 and F_1 reads 0.45. Which model is better than chance?
4. A scaler fitted on all rows inflates accuracy by 0.4 pp on one dataset and 40 pp on another. What differs?
5. NB plateaus on the learning curve. Data, features, or k — which knob do you turn?

Further reading

- Hastie, Tibshirani & Friedman — *The Elements of Statistical Learning* (book site; free PDF under "Download the book PDF"). `https://hastie.su.domains/ElemStatLearn/` — ch. 2 (bias–variance) and ch. 7 (model assessment) are this chapter's graduate ancestor.
- scikit-learn — *Model evaluation*. `https://scikit-learn.org/stable/modules/model_evaluation.html` — the reference for every metric used here, with the same naming.
- scikit-learn — *Cross-validation*. `https://scikit-learn.org/stable/modules/cross_validation.html` — fold splitters incl. `StratifiedKFold`, `GroupKFold`, `TimeSeriesSplit`.

- scikit-learn — *Common pitfalls*. https://scikit-learn.org/stable/common_pitfalls.html — the library's own leakage catalogue; our five routes are its headliners.
- Cawley & Talbot, "On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation," *JMLR* 11 (2010). https://www.jmlr.org/papers/v11/cawley10a.html — what tuning on the test score costs, measured.
- Domingos, "A Few Useful Things to Know about Machine Learning," *CACM* 55(10), 2012. https://homes.cs.washington.edu/~pedrod/papers/cacm12.pdf — the classic aphorisms; "generalisation" is item one.
- Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters* 27 (2006). https://people.inf.elte.hu/kiss/11dwhdm/roc.pdf — the canonical ROC tutorial behind Figure **??**.
- scikit-learn — `confusion_matrix` API. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html — row/column convention used throughout the book.

DECISION RULE

Watch instead (if you'd rather see it)

- **StatQuest: Machine Learning Fundamentals — The Confusion Matrix** (7:12). StatQuest with Josh Starmer. https://www.youtube.com/watch?v=Kdsp6soqA7o — the four cells animated with the same row/column convention as Figure **??**; matches §**??**. Accessed 2026-10-11.
- **StatQuest: Machine Learning Fundamentals — Cross Validation** (6:04). StatQuest with Josh Starmer. https://www.youtube.com/watch?v=fSytzGwwBVw — fold diagrams and the why; matches §**??**. Accessed 2026-10-11.
- **StatQuest: ROC and AUC, Clearly Explained!** (16:17). StatQuest with Josh Starmer. https://www.youtube.com/watch?v=4jRBRDbJemM — threshold-sweep construction matching Figure **??**; matches §**??**. Accessed 2026-10-11.

Notes

1. The k-NN expansion $\|\mathbf{a}-\mathbf{b}\|^2 = \|\mathbf{a}\|^2 - 2\mathbf{a}^T\mathbf{b} + \|\mathbf{b}\|^2$ is the standard trick; in higher dimensions an index (KD-tree, ball tree, or an approximate index like HNSW) replaces the brute-force product.
2. Naïve Bayes's independence assumption is violated constantly and still ranks among the best first baselines on sparse text — see Domingos & Pazzani (1997) for why the ranking survives even when the probabilities do not [**?**].
3. "The test set is touched once" is the ideal; in practice the budget is a small integer. See Cawley & Talbot (2010) in Further reading for the measured cost of each extra peek.
4. `GroupKFold` and `TimeSeriesSplit` documentation and examples are under the sklearn cross-validation page cited in Further reading.

5. The phantom-leak magnitudes here were measured on `breast_cancer` (random state 7); the direction is the lesson, not the digits — rerun `leak_demo.py` with other seeds and the qualitative pattern holds.

Exercises

[easy]

Exercise 3.1. Compute precision, recall, and F_1 by hand for tp=30, fp=10, fn=20, tn=140.

Exercise 3.2. A classifier scores 0.97 accuracy on a 30:1 imbalanced set where the majority class is 96.8% of rows. Is it better than the lazy classifier? What would F_1 of the lazy classifier be?

Exercise 3.3. In 5-fold CV on n=800, how many rows does each fold's test split hold, and how many total refits happen?

Exercise 3.4. Why does R^2=0 not mean "the model predicts zero"? What does it predict instead?

Exercise 3.5. Which is bigger at k=1: train accuracy or test accuracy? What closes the gap as k grows?

[medium]

Exercise 3.6. Implement `stratified_split(y, test_frac, seed)` from scratch: group indices by class, shuffle each group, carve each at the same fraction. Verify class ratios match on `titanic.survived`.

Exercise 3.7. Extend Listing **??** to print micro-*and* macro-F_1 on `digits`. Which classes lose the most under macro averaging?

Exercise 3.8. Make the grouped-split argument concrete: split `titanic` by `ticket` number as the group key (`GroupKFold` with `groups=df.ticket`) and compare kNN test accuracy vs a plain random split. Which is higher, and why does that hurt?

Exercise 3.9. Sweep the classification threshold of a logistic model on `breast_cancer` from 0.1 to 0.9; plot precision and recall against it and mark the F_1-maximising threshold.

Exercise 3.10. Re-run Listing **??** on `digits` instead of `breast_cancer`. Does the scale-then-split phantom grow, shrink, or flip sign? Explain with the "how much test information" rule.

Exercise 3.11. Write `sliding_window_cv(X, y, k)` that yields (train, test) index pairs where each test fold is strictly later than its train fold.

[hard]

Exercise 3.12. Prove that the NB decision boundary between two classes is linear when both class-conditionals are Gaussian with *equal* variances, and quadratic when they differ. (Hint: set the two log-posteriors equal and collect terms.)

Exercise 3.13. Derive why accuracy fails at 99:1 imbalance: give the largest accuracy achievable by a classifier whose recall on the minority class is r, and show it is maximised at $r{=}0$.

Exercise 3.14. Design a leak-free grouped split for a dataset with two grouping keys (family *and* cabin) — prove no group straddles the boundary in either key, or explain why you cannot satisfy both and must pick one.

Exercise 3.15. Show that under squared loss, the best constant predictor of y is $\bar{y}$ — and that this is exactly what $R^2{=}0$ measures.

Exercise 3.16. A friend reports "0.99 ROC-AUC" on a dataset that contains the same patient's two scans split across train/test. Reproduce a version of the bug on `titanic` using `ticket` as the group, and quantify the inflation.

Hints and answers

3.1 $P{=}0.75$, $R{=}0.60$, $F_1{=}0.667$.

3.2 No — lazy accuracy is 0.968; lazy $F_1{=}0$ (no positive predictions). Report recall.

3.3 160 rows per test fold; 5 refits.

3.4 It predicts $\bar{y}$ — the constant-mean model is the $R^2{=}0$ baseline.

3.5 Train. Variance shrinks as neighbours vote; the gap closes.

3.6 `np.argsort` inside a per-class loop; assert `np.isclose` on the two ratios.

3.7 Macro-F_1 weights each class equally; rare digits show the bigger drop.

3.8 Random split is usually higher — sibling passengers share tickets; the model memorised family outcomes.

3.9 Threshold $\uparrow \Rightarrow$ precision $\uparrow$, recall $\downarrow$; the F_1 peak is typically around 0.4–0.6 here.

3.10 Tiny either way — iid rows share statistics.

3.11 Sort by time; fold i tests on block $i{+}1$, trains on blocks $1..i$.

3.12 Equal $\sigma^2 \Rightarrow$ the x^2 terms cancel in the difference of log-posteriors, leaving a linear form $w^T x + b$.

3.13 Accuracy $= 0.99 + 0.01 \cdot (r - \text{FPR})$-ish; derive exactly and show the maximum over r is at the corner.

3.14 If any family spans cabins, one key must win — pick the key with the stronger causal claim to leakage.

3.15 Expand $\sum(y_i - c)^2$ and set the derivative in c to zero.

3.16 Grouped fold drops the apparent AUC to the honest level — the inflation is the gap between the two protocols.

CHAPTER 4

Linear models and regularisation

The linear model is where machine learning starts paying rent on its linear algebra. One line of closed-form mathematics — $\boldsymbol{\theta} = (X^T X)^{-1} X^T \mathbf{y}$ — delivers a fitted model, and everything in this chapter is a variation on that theme: add a penalty and you get ridge and lasso, wrap the score in a sigmoid and you get logistic regression, bend the input with a feature map and you get polynomial regression. The recurring question is the one Chapter 3 trained you to ask: not "does it fit?" but "does it generalise, and what did the fit cost?" Regularisation is the answer engineered directly into the objective: accept a little bias on purpose, in exchange for a lot less variance. Below we derive every model used, implement two of them from scratch, and read coefficient paths the way a senior engineer reads a flame graph.

4.1 Least squares: the normal equations

Definition 4.1 (Ordinary least squares)**.** Given $X \in \mathbb{R}^{n \times d}$ (rows are samples, per Chapter 1) and targets $\mathbf{y} \in \mathbb{R}^n$, the least-squares fit is

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \|X\boldsymbol{\theta} - \mathbf{y}\|_2^2 = (X^T X)^{-1} X^T \mathbf{y},$$

which exists uniquely when $X^T X$ is invertible — i.e. X has full column rank.

The derivation is one line of calculus (set $\nabla_{\boldsymbol{\theta}} \|X\boldsymbol{\theta} - \mathbf{y}\|^2 = 0$) and one geometric fact: the residual is orthogonal to the column space. The implementation is one line of NumPy — and you should call `solve`, never form the inverse.

```
# code/ch04/linreg_scratch.py -- OLS via normal equations + ridge closed form.
# Run from the repository root: python3 code/ch04/linreg_scratch.py
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split

X, y = load_diabetes(return_X_y=True) # already centred+scaled by sklearn
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=7)
n, d = Xtr.shape
Xb = np.c_[Xtr, np.ones(n)] # intercept column
Xte_b = np.c_[Xte, np.ones(len(Xte))]

# --- OLS: theta = (X^T X)^{-1} X^T y -------------------------------
```

```
theta = np.linalg.solve(Xb.T @ Xb, Xb.T @ ytr)
pred = Xte_b @ theta
mse = ((pred - yte) ** 2).mean()
r2 = 1 - ((pred - yte) ** 2).sum() / ((yte - yte.mean()) ** 2).sum()
print(f"OLS: test MSE {mse:.1f} R2 {r2:.3f}")
print("coefs:", np.round(theta[:-1], 2))

# --- ridge: theta = (X^T X + lam I)^{-1} X^T y (no penalty on bias)
    --
print(f"\n{'lambda':>9} {'test MSE':>9} {'||theta||':>10}")
for lam in [0, 0.01, 0.1, 1, 10, 100, 1000]:
    I = np.eye(d + 1); I[-1, -1] = 0
    th = np.linalg.solve(Xb.T @ Xb + lam * I, Xb.T @ ytr)
    p = Xte_b @ th
    print(f"{lam:>9.2f} {((p - yte)**2).mean():>9.1f} "
          f"{np.linalg.norm(th[:-1]):>10.2f}")

# --- the singular rescue
    ----------------------------------------------
print("\nSingular X^T X: duplicate a column and watch OLS blow up,")
X2 = np.c_[Xtr, Xtr[:, 3] * 2] # col d = 2 * col 3 -> rank deficient
X2b = np.c_[X2, np.ones(n)]
try:
    th_bad = np.linalg.solve(X2b.T @ X2b, X2b.T @ ytr)
    print(" OLS ||theta|| on redundant design:", f"{np.linalg.norm(
        th_bad):.2e}")
except np.linalg.LinAlgError:
    print(" OLS failed outright (singular matrix)")
for lam in [1e-6, 1.0]:
    I = np.eye(d + 2); I[-1, -1] = 0
    th_ok = np.linalg.solve(X2b.T @ X2b + lam * I, X2b.T @ ytr)
    print(f" ridge lam={lam}: ||theta|| {np.linalg.norm(th_ok):.3f}")
```

Listing 4.1: `code/ch04/linreg_scratch.py`: OLS, the ridge path, and the singular-X^TX rescue — all closed-form.

```
OLS: test MSE 2966.0 R2 0.404
coefs: [ -55.62 -259.84 540.31 344.62 -971.67 612.39 155.14 190.64
   830.74
  73.87]

  lambda test MSE ||theta||
    0.00 2966.0 1598.90
    0.01 2933.8 1040.79
    0.10 2861.3 821.45
    1.00 2981.4 486.53
   10.00 4246.3 126.46
  100.00 4958.9 15.79
 1000.00 5062.4 1.62

Singular X^T X: duplicate a column and watch OLS blow up,
 OLS ||theta|| on redundant design: 7.93e+17
 ridge lam=1e-06: ||theta|| 1576.279
```

```
ridge lam=1.0: ||theta|| 478.198
```

Example 4.2 (WE 4.1 — When X^TX is singular)**.** Strategy: break the rank assumption deliberately and watch what the coefficients do.

Solution. Listing **??** appends a duplicate column ($x_{d+1} = 2x_3$) so X^TX is rank-deficient. OLS returns $\|\boldsymbol{\theta}\| \approx 7.9 \times 10^{17}$: infinitely many solutions exist and the solver picks a catastrophic one — huge cancelling coefficients that happen to fit. Ridge with λ=1 on the same broken design returns $\|\boldsymbol{\theta}\| = 478$: the penalty picks the *small-norm* solution from the infinite family. This is ridge's original job (it was invented to solve unsolvable problems) — regularisation here is not preference, it is selection.

4.2 Ridge: shrink the coefficients

Definition 4.3 (Ridge regression)**.**

$$\hat{\boldsymbol{\theta}}_{\text{ridge}} = \arg\min_{\boldsymbol{\theta}} \ \|X\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda\|\boldsymbol{\theta}\|_2^2 = (X^TX + \lambda I)^{-1}X^T\mathbf{y}.$$

The penalty applies to coefficients only — the intercept is left unpenalised (standardise first, or the scale of **x** decides the shrinkage for you).

λ is the bias–variance dial of Chapter 3 made literal: λ=0 is OLS (maximum variance), $\lambda \to \infty$ pins every coefficient at zero (maximum bias). The train MSE climbs monotonically in λ while the test MSE traces the familiar U — read it in Figure **??** and in Listing **??**'s table: 2966 $\to$ 2861 near $\lambda \approx 0.1$, then 4246 by λ=10.

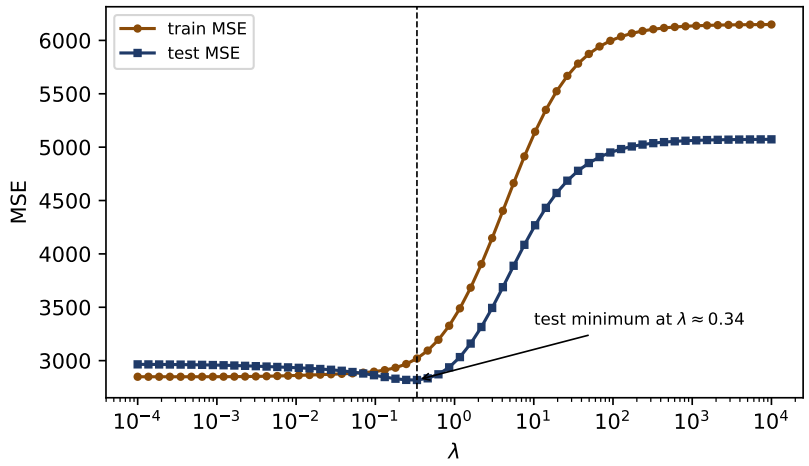


Figure 4.1: Train vs test MSE against λ on `diabetes` (`figures/fig_ch04_lambda_ucurve.py`). Train error rises monotonically; test error bottoms at $\lambda \approx 0.2$–0.3 and climbs back — the U-curve is the bias–variance trade-off made measurable.

4.3 Lasso: shrink *some* coefficients to zero

Swap the ℓ_2 ball for the ℓ_1 diamond:

$$\hat{\boldsymbol{\theta}}_{\text{lasso}} = \arg\min_{\boldsymbol{\theta}} \ \|X\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda\|\boldsymbol{\theta}\|_1.$$

No closed form exists (the ℓ_1 corner is not differentiable), so lasso is solved iteratively — but the geometric story is richer and it is the reason to care.



Figure 4.2: Why lasso gives zeros. The constraint region (filled) grows until it kisses the loss contours (dashed). The smooth circle can touch anywhere; the diamond's corners sit on the axes, so the first touch is usually a corner — some coefficient *exactly* zero.

```python
# code/ch04/reg_compare.py -- scratch vs sklearn, ridge/lasso paths.
# Run from the repository root: python3 code/ch04/reg_compare.py
import numpy as np
from sklearn.datasets import load_diabetes, fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge, Lasso, LinearRegression
from sklearn.preprocessing import StandardScaler

X, y = load_diabetes(return_X_y=True)
names = load_diabetes().feature_names
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2,
    random_state=7)

print("=== diabetes: OLS vs Ridge(1.0) vs Lasso(0.05) coefs ===")
ols = LinearRegression().fit(Xtr, ytr)
rdg = Ridge(alpha=1.0).fit(Xtr, ytr)
las = Lasso(alpha=0.05).fit(Xtr, ytr)
print(f"{'feature':>7} {'OLS':>8} {'Ridge':>8} {'Lasso':>8}")
for i, nm in enumerate(names):
    print(f"{nm:>7} {ols.coef_[i]:>8.2f} {rdg.coef_[i]:>8.2f} "
          f"{las.coef_[i]:>8.2f}")
print(f"test R2: OLS {ols.score(Xte, yte):.3f} "
      f"Ridge {rdg.score(Xte, yte):.3f} Lasso {las.score(Xte, yte):.3f}
          ")

print("\n=== california: which features die first under lasso? ===")
Xc, yc = fetch_california_housing(return_X_y=True)
cn = fetch_california_housing().feature_names
Xc = StandardScaler().fit_transform(Xc)
for a in [0.001, 0.01, 0.1]:
    m = Lasso(alpha=a, max_iter=5000).fit(Xc, yc)
```

```
30    alive = [f"{nm}={c:.2f}" for nm, c in zip(cn, m.coef_) if abs(c) >
          1e-6]
31    print(f"alpha={a}: kept {len(alive)}/8 -> {alive}")
```

Listing 4.2: `code/ch04/reg_compare.py`: scratch-grade comparison — OLS, ridge, lasso on `diabetes`, then the lasso kill-order on `california`.

```
=== diabetes: OLS vs Ridge(1.0) vs Lasso(0.05) coefs ===
feature OLS Ridge Lasso
   age -55.62 17.89 -25.73
   sex -259.84 -75.51 -215.68
   bmi 540.31 285.75 541.73
    bp 344.62 188.72 304.36
    s1 -971.67 10.02 -116.42
    s2 612.39 -22.12 -0.00
    s3 155.14 -147.98 -234.64
    s4 190.64 117.87 3.64
    s5 830.74 252.81 536.93
    s6 73.87 114.56 54.37
test R2: OLS 0.404 Ridge 0.401 Lasso 0.422

=== california: which features die first under lasso? ===
alpha=0.001: kept 8/8 -> ['MedInc=0.82', 'HouseAge=0.12', 'AveRooms
   =-0.25', 'AveBedrms=0.29', 'Population=-0.00', 'AveOccup=-0.04', '
   Latitude=-0.89', 'Longitude=-0.86']
alpha=0.01: kept 7/8 -> ['MedInc=0.78', 'HouseAge=0.12', 'AveRooms
   =-0.13', 'AveBedrms=0.17', 'AveOccup=-0.03', 'Latitude=-0.80', '
   Longitude=-0.76']
alpha=0.1: kept 3/8 -> ['MedInc=0.71', 'HouseAge=0.11', 'Latitude
   =-0.01']
```

Example 4.4 (WE 4.2 — Reading the kill order)**.** Strategy: which feature dies first under ℓ_1 is the feature the data voted least useful.

Solution. On `diabetes`, lasso at α=0.05 already zeros `s2` (-0.00) and shrinks the blood-serum cluster hard — the six serum measurements largely repeat each other's information, and lasso picks a spokesperson (`s5` keeps 536.93). On `california`, `Population` dies first (gone by α=0.01), and at α=0.1 only `MedInc`, `HouseAge`, and `Latitude` survive — income dominates California housing prices, a claim lasso states as an absence rather than a coefficient. Test R^2 actually *improves* (0.404 $\rightarrow$ 0.422): the deleted features were costing variance.

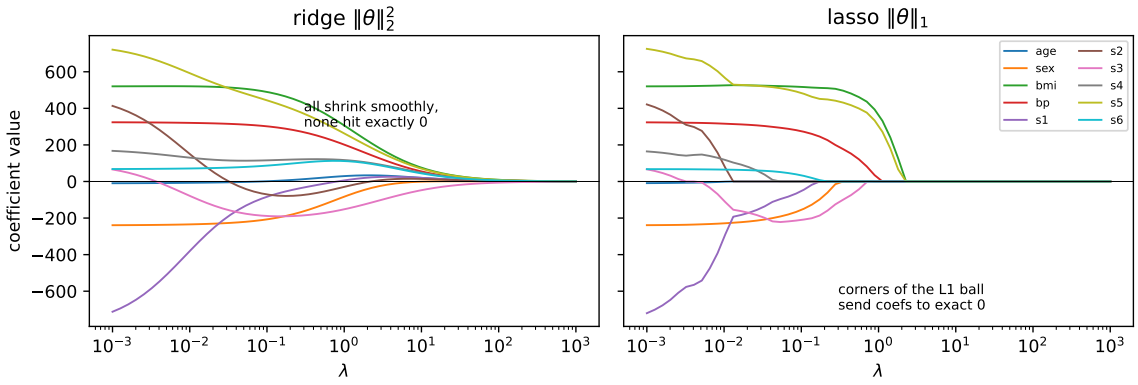


Figure 4.3: Coefficient paths vs λ on `diabetes` (`figures/fig_ch04_coef_paths.py`). Ridge shrinks all ten coefficients smoothly toward zero; lasso's path is piecewise linear and each kink is a feature dying — the geometric difference of Figure **??** plotted.

4.4 Logistic regression: regression for labels

For classification, regress the *log-odds* instead of the label. The sigmoid squashes the linear score into a probability:

$$p = \sigma(\mathbf{w}^T\mathbf{x} + b), \qquad \sigma(z) = \frac{1}{1+e^{-z}}.$$

Training minimises the negative log-likelihood (a.k.a. cross-entropy loss), which has no closed-form minimum but a clean gradient — $\nabla_{\mathbf{w}}\mathrm{NLL} = \frac{1}{n}X^T(\mathbf{p}-\mathbf{y})$ — so gradient descent does the fitting. Listing **??** implements it in a dozen lines; the loss trace (NLL 261 $\to$ 93) is the descent working.

```
# code/ch04/logreg_scratch.py -- logistic regression by gradient descent
# on the two most separable breast_cancer features.
# Run from the repository root: python3 code/ch04/logreg_scratch.py
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
X = X[:, [23, 2]] # worst radius + mean perimeter
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2,
                                      random_state=7, stratify=y)
mu, sd = Xtr.mean(0), Xtr.std(0)
Xtr, Xte = (Xtr - mu) / sd, (Xte - mu) / sd
n = len(Xtr)
Xb = np.c_[Xtr, np.ones(n)]
Xte_b = np.c_[Xte, np.ones(len(Xte))]

def nll(w):
    z = Xb @ w
    # -log sigma(z) for y=1, -log(1-sigma(z)) for y=0, stably:
    return (np.logaddexp(0, z) - ytr * z).sum()
```

```
w = np.zeros(3)
lr, steps = 0.5, 2000
for it in range(steps):
    z = Xb @ w
    p = 1 / (1 + np.exp(-z))
    grad = Xb.T @ (p - ytr) / n
    w -= lr * grad
    if it in [0, 9, 99, 499, 1999]:
        print(f"step {it:>5}: NLL {nll(w):.3f}")

pred = (1 / (1 + np.exp(-Xte_b @ w)) > 0.5).astype(int)
acc = (pred == yte).mean()
tp = int(((pred == 1) & (yte == 1)).sum())
fp = int(((pred == 1) & (yte == 0)).sum())
fn = int(((pred == 0) & (yte == 1)).sum())
print(f"\nw = {np.round(w, 3)}")
print(f"test acc {acc:.3f} precision {tp/(tp+fp):.3f} "
      f"recall {tp/(tp+fn):.3f}")
```

Listing 4.3: `code/ch04/logreg_scratch.py`: logistic regression by gradient descent, NLL computed with the numerically stable `logaddexp`.

```
step 0: NLL 261.230
step 9: NLL 149.337
step 99: NLL 109.636
step 499: NLL 101.848
step 1999: NLL 93.379

w = [-6.864 0.608 -0.393]
test acc 0.930 precision 0.957 recall 0.931
```

Example 4.5 (WE 4.3 — Read the weights as a story)**.** Strategy: a fitted coefficient is a claim about a feature, in odds.

Solution. On the two strongest `breast_cancer` features the model learns $\mathbf{w} = (-6.86,\ 0.61)$: a unit rise in standardised *worst radius* swings the log-odds -6.86 toward malignant — huge, because radius is nearly diagnostic by itself. Test accuracy 0.930, precision 0.957, recall 0.931: 6 of 114 wrong, and the errors are the borderline-radius patients the sigmoid honestly scored near 0.5. A linear boundary doing 93% is why this dataset is the canonical warm-up.

4.5 Softmax: one boundary, many classes

Multiclass generalises pairwise: one weight vector per class, scores $\mathbf{z} = W\mathbf{x}$, and the softmax

$$P(y{=}c \mid \mathbf{x}) = \frac{e^{z_c}}{\sum_j e^{z_j}}$$

normalises them into a distribution. Training minimises cross-entropy on the true class's probability — the same NLL with more terms. Nothing new to learn mathematically; the

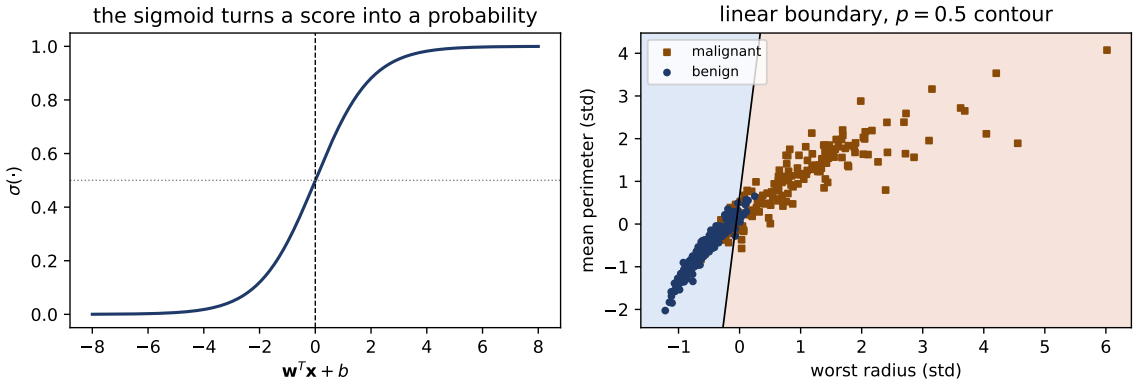


Figure 4.4: Left: the sigmoid. Right: the learned p=0.5 boundary on two standardised features (`figures/fig_ch04_sigmoid_boundary.py`) — a straight line in feature space, because the model is linear in $\mathbf{x}$ whatever the sigmoid does to its output.

lesson is that "multinomial logistic regression" is just logistic regression with a softmax head, and every classifier in the deep-learning chapters ends this way.

4.6 Feature maps: bend the input, keep the line

Linear in $\mathbf{x}$ need not mean linear in *appearance*: fit $\mathbf{w}^T\phi(\mathbf{x})$ where $\phi(x) = (1, x, x^2, \ldots, x^p)$ and the model is a degree-p polynomial while still linear in $\boldsymbol{\theta}$ — OLS machinery applies verbatim. The cost is expressivity's usual price: Figure **??** shows degree-9 threading every data point while oscillating wildly between them — $R^2 = 1.00$ on train, hopeless on test.

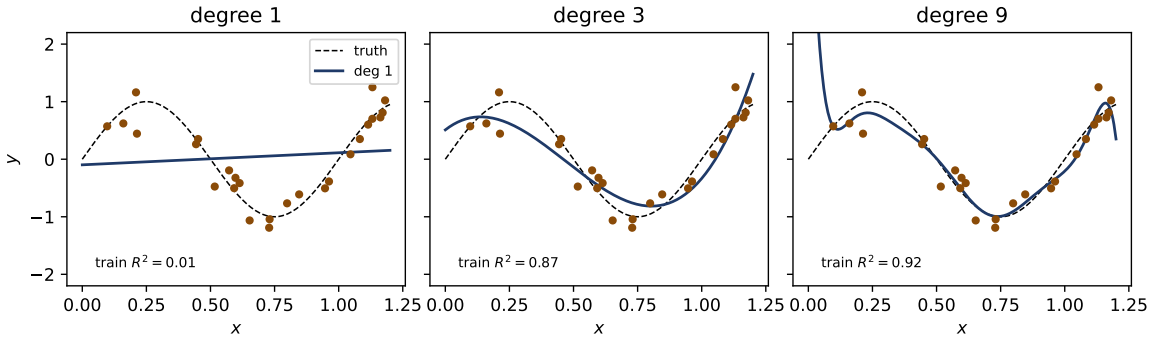


Figure 4.5: Polynomial degrees 1, 3, 9 on the same 26 points (`figures/fig_ch04_poly_overfit.py`). Degree 3 approximates the truth; degree 9 memorises it — the overfit strip is the visual version of the $\lambda \to 0$ end of the U-curve.

4.7 The kernel trick in one page

Every algorithm in this chapter touches data only through inner products $\mathbf{x}_i^T \mathbf{x}_j$. Replace each with a kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$ and you fit in the ϕ-space *without ever computing* ϕ — even infinite-dimensional ones (the RBF kernel). SVMs are the maximally squeezed version of this idea: maximum-margin hyperplane plus kernel. We will not implement one — the point to carry forward is that "linear model" and "nonlinear model" differ only in the features you let the dot product see.

4.8 Common mistakes

Penalising the intercept The bias term absorbs the mean of $\mathbf{y}$; shrinking it toward zero is a bug in most setups. Sklearn and Listing **??** both exempt it — mimic that.

Regularising unscaled features A penalty treats all coefficients equally only if all features share a scale. Standardise before ridge/lasso or the biggest-unit column gets punished hardest.

Choosing λ on the test set λ is a hyperparameter: pick it by CV inside training, then touch test once (Chapter 3's protocol, verbatim).

Reading lasso zeros as "feature is useless" Correlated features split the vote — lasso keeps a spokesperson, not a ranking. The survivors are useful; the dead are only redundant-or-useless.

Forming $(X^T X)^{-1}$ explicitly `np.linalg.solve` is faster and far more stable than `inv`; the inverse you never form cannot amplify your conditioning error.

Assuming "linear" means a line Polynomial maps, interactions, basis expansions — the model is linear in $\boldsymbol{\theta}$, and that is the only linearity that matters.

Forgetting sigmoid probabilities need calibration $\sigma(z)$ is a score shaped like a probability; whether it *is* one depends on calibration (Platt scaling, isotonic) — check before quoting odds.

Using MSE for classification Cross-entropy is the MLE loss for Bernoulli labels; least-squares on $0/1$ targets works accidentally and optimises the wrong thing.

Blind trust in R^2 on test A high test R^2 after fitting preprocessing on all data is Chapter 3's leak in a trench coat — refit the pipeline inside the split.

4.9 Summary table

Model	Objective	When to reach for it
OLS	$\|X\theta - y\|^2$	baseline regression; full rank, low d
Ridge	$+\lambda\|\theta\|_2^2$	correlated features; wants stability
Lasso	$+\lambda\|\theta\|_1$	feature selection by zeros; sparse truth
Elastic net	$+\lambda_1\|\theta\|_1 + \lambda_2\|\theta\|_2^2$	correlated *and* sparse
Logistic	cross-entropy on $\sigma(w^T x)$	binary probabilities, fast
Softmax	cross-entropy on softmax(Wx)	multiclass one-liner
Poly. map	$\phi(x)$ inside OLS	smooth nonlinear, small n
Kernel	$K(x_i, x_j)$ inner products	nonlinear at linear-model cost

4.10 Drill — five concept questions

1. Ridge and lasso differ by one letter of subscript. What do the resulting coefficient paths look like, and why?
2. Why does the test MSE form a U in λ while train MSE is monotone?
3. $X^T X$ is singular. Name the regulariser whose solution still exists, and what it picks out of the infinite OLS family.
4. Logistic regression minimises NLL, not MSE — what does NLL correspond to probabilistically?
5. Degree-9 polynomial scores train $R^2 = 1.00$. Bias problem or variance problem, and which chapter-3 diagnostic confirms it?

Further reading

- Hastie, Tibshirani & Friedman — *The Elements of Statistical Learning*. https://hastie.su.domains/ElemStatLearn/ — ch. 3 is the canonical ridge/lasso treatment behind Figure **??**.
- Hastie, Tibshirani & Wainwright — *Statistical Learning with Sparsity* (book site; free PDF). https://hastie.su.domains/StatLearnSparsity/ — the ℓ_1 book; ch. 1–3 cover lasso geometry and paths.
- scikit-learn — *Linear models* user guide. https://scikit-learn.org/stable/modules/linear_model.html — `Ridge`/`Lasso`/`LogisticRegression` parameters as used in the listings.
- James, Witten, Hastie & Tibshirani — *An Introduction to Statistical Learning* (free PDF). https://www.statlearning.com/ — the gentler ESL; ch. 4 classification and ch. 6 regularisation.
- Ng, Andrew — *CS229 Lecture Notes 1*. https://see.stanford.edu/materials/aimlcs229/cs229-notes1.pdf — normal equations and logistic regression derived slowly, by hand.

- Molnar — *Interpretable Machine Learning*, linear models chapter. https://christophm.github.io/interpretable-ml-book/limo.html — reading coefficients responsibly, incl. the correlated-features caveat from WE 4.2.
- Wikipedia — "Lasso (statistics)". https://en.wikipedia.org/wiki/Lasso_(statistics) — quick formulas, history (Tibshirani 1996), and the geometric figure this chapter redrew.

DECISION RULE

Watch instead (if you'd rather see it)

- **StatQuest: Regularization Part 1 — Ridge (L2) Regression** (20:26). StatQuest with Josh Starmer. https://www.youtube.com/watch?v=Q81RR3yKn30 — ridge as "desensitisation," matching §**??**; includes the unsolvable-problem story of WE 4.1. Accessed 2026-10-11.
- **StatQuest: Regularization Part 2 — Lasso (L1) Regression** (8:19). StatQuest with Josh Starmer. https://www.youtube.com/watch?v=NGf0voTMlcs — the contrast half of the pair; matches §**??**. Accessed 2026-10-11.
- **StatQuest: Logistic Regression** (8:47). StatQuest with Josh Starmer. https://www.youtube.com/watch?v=yIYKR4sgzI8 — sigmoid and likelihood behind §**??**. Accessed 2026-10-11.

Notes

1. The normal equations are fine for d up to a few thousand; beyond that, QR or SVD is stabler, and at scale the gradient methods of §**??** take over entirely (Hastie et al. ch. 3 [**?**]).
2. Lasso is Tibshirani (1996), ridge is Hoerl & Kennard (1970) — invented precisely for WE 4.1's singular case, decades before "ML" was a word for it.
3. The elastic net mixes both penalties and inherits both behaviours; it is the default answer when correlated features are also sparse (Zou & Hastie 2005).
4. Cross-entropy is negative log-likelihood under the Bernoulli model — the same quantity computed two ways. The gradient $X^T(\mathbf{p} - \mathbf{y})/n$ is derived fully in Ng's notes (Further reading).
5. We skipped kernel details on purpose; Cawley & Talbot [**?**] (ch. 3's list) is again the citation for how hyperparameter choice corrupts honest evaluation.

Exercises

[easy]

Exercise 4.1. Write the normal equations from the residual-orthogonality argument in one line each: (a) the residual, (b) the orthogonality, (c) the solution.

Exercise 4.2. In Listing **??**, which λ minimises test MSE, and what is $\|\boldsymbol{\theta}\|$ there? Why does $\|\theta\|$ fall monotonically?

Exercise 4.3. On `california`, which feature dies first under lasso and which survives longest? One sentence of economic intuition.

Exercise 4.4. Why does logistic regression output $\sigma(\mathbf{w}^T\mathbf{x})$ rather than $\mathbf{w}^T\mathbf{x}$ itself? What breaks if you skip the sigmoid?

[medium]

Exercise 4.5. Implement `ridge_fit(X, y, lam)` with an unpenalised intercept, and verify your coefficients match `Ridge(alpha=lam)` up to 10^{-6}.

Exercise 4.6. Sweep the logistic decision threshold on the WE 4.3 model; find the threshold maximising F_1 and the threshold giving recall ≥ 0.98. Which would a screening program pick?

Exercise 4.7. Implement a 2-fold CV selector for lasso's α on `diabetes`; report the α that minimises CV MSE and its test R^2.

Exercise 4.8. Fit degree-3 polynomial on the Figure **??** data with ridge $\lambda = 1$ and without it; measure the test MSE gap regularisation buys at fixed degree.

Exercise 4.9. Reproduce Figure **??**'s lasso panel with `linear_model.lars_path` and confirm the kink positions match the coefficient-zero crossings.

Exercise 4.10. Elastic net from scratch: implement coordinate descent for the $\ell_1+\ell_2$ objective on `diabetes` and compare the survivor set to pure lasso.

[hard]

Exercise 4.11. Derive $\nabla_{\mathbf{w}}\text{NLL} = \frac{1}{n}X^T(\mathbf{p} - \mathbf{y})$ for the logistic model, starting from the Bernoulli likelihood. (This is the one derivation every interview asks.)

Exercise 4.12. Prove the ridge solution exists and is unique for every $\lambda > 0$ even when X^TX is singular — i.e. $X^TX + \lambda I \succ 0$.

Exercise 4.13. Show the lasso soft-threshold rule: in 1-D, the coordinate-wise solution is $\text{sign}(z)\max(|z| - \lambda, 0)$; explain geometrically why this produces exact zeros.

Exercise 4.14. Design the singular-rescue experiment quantitatively: on WE 4.1's redundant design, plot $\|\boldsymbol{\theta}\|$ vs λ from 10^{-6} to 10^3 and locate where the cancellation disappears.

Exercise 4.15. Prove logistic regression's decision boundary is linear, then show the *probability* contours are also linear — the sigmoid bends the numbers, never the geometry.

Hints and answers

4.1 $X^T X\theta = X^T y$ follows from $X^T(X\theta - y) = 0$.

4.2 $\lambda \approx 0.1$ gives MSE 2861; the penalty pulls all coefficients toward the origin at once.

4.3 `Population` first, `MedInc` last — income is the strongest single driver of California house price.

4.4 Probabilities need $[0, 1]$; raw scores ignore boundedness and calibration.

4.5 Solve without inverting: standardise, append the ones column, then `solve` — match sklearn's intercept separately.

4.6 High recall needs a low threshold; screening pays for precision later.

4.7 2-fold is enough to rank α's; expect $\alpha \approx 0.01$–0.1 to win.

4.8 Ridge at degree 3 barely changes the fit; its value shows up at degree 9 — try both.

4.9 `lars_path` returns the exact knot grid; the kinks are where a coefficient enters or leaves the active set.

4.10 Coordinate descent loops one coordinate at a time with the soft threshold of 4.13.

4.11 Write $L = \prod p_i^{y_i}(1 - p_i)^{1-y_i}$, take $-\log$, differentiate — the derivative of σ factorises beautifully.

4.12 Any $v \neq 0$: $v^T(X^T X)v = \|Xv\|^2 \geq 0$, plus $\lambda v^T v > 0$.

4.13 The kink at $|z| = \lambda$ is the ℓ_1 corner in miniature; below it the subgradient contains zero and the optimum *is* zero.

4.14 Plot should show $\|\theta\|$ collapsing around $\lambda \sim 10^{-2}$–10^{-1} as the cancelling pair joins the shrunk family.

4.15 $\mathbf{w}^T\mathbf{x} + b = c$ is a hyperplane for every c; σ is monotone, so level sets coincide.

CHAPTER 5

Trees and ensembles

A decision tree is a program you could read aloud: "if `color_intensity` ≤ 3.800, go left; else go right" — a chain of axis-aligned yes/no questions ending in a majority vote. It is the first model in the book whose learned structure is itself interpretable, and it is also the first that overfits almost on purpose: grown deep enough, a tree reproduces its training set perfectly. The second half of the chapter is the fix, and it is one of the most profitable ideas in the field: *train many flawed trees and average them*. Bagging, random forests, and gradient boosting are three ways to spend the same insight — reduce variance by voting — and together they power most of the tabular machine learning actually deployed. Below we grow a tree by hand, measure a single tree's variance over 20 bootstraps, and watch boosting fit residuals stage by stage.

5.1 Trees as axis-aligned partitions

A tree carves feature space into rectangles: each internal node tests one feature against one threshold, each leaf predicts the majority class (or mean target) of its cell. The picture to hold is the correspondence: every tree is a staircase partition, every staircase partition is some tree.

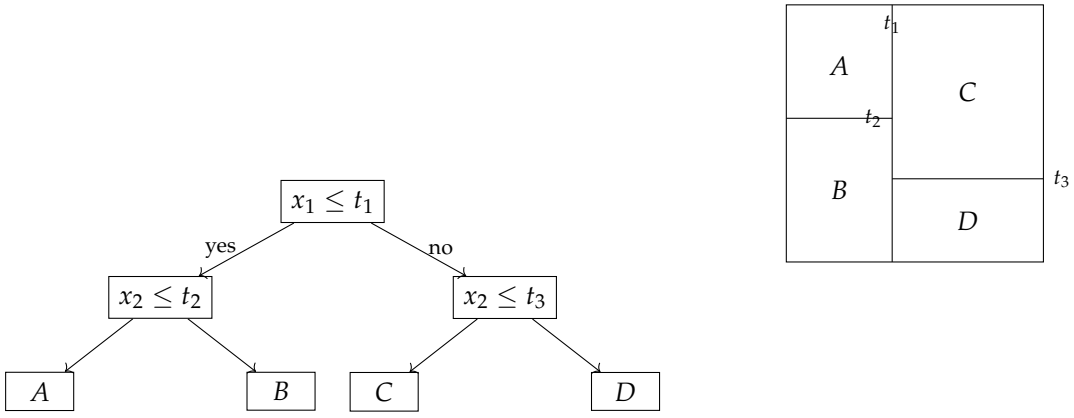


Figure 5.1: Tree ↔ partition. Each split is one axis-aligned cut; every path root-to-leaf is one rectangle of feature space. Depth-2 here; a real tree recurses the same two moves.

5.2 Choosing a split: Gini and information gain

Greedy induction asks, at each node, one question: *which (feature, threshold) pair makes the children purest?* Purity is measured by the Gini impurity

$$G(\text{node}) = 1 - \sum_{c} p_c^2$$

(the probability a random label disagrees with a random draw), or equivalently-ranked entropy. A split's score is the weighted-impurity drop — *information gain*:

$$\Delta = G(\text{parent}) - \frac{n_L}{n} G(\text{left}) - \frac{n_R}{n} G(\text{right}).$$

```
# code/ch05/tree_scratch.py -- a decision tree grown split by split.
# Gini criterion, best-of-all-thresholds split, depth cap, leaf majority.
# Run from the repository root: python3 code/ch05/tree_scratch.py
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

X, y = load_wine(return_X_y=True)
names = load_wine().feature_names
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                            random_state=7, stratify=y)

def gini(labels):
    _, cnt = np.unique(labels, return_counts=True)
    p = cnt / len(labels)
    return 1 - (p ** 2).sum()

def best_split(X, y):
    base = gini(y); best = (-1, None, None)
    for j in range(X.shape[1]):
        for t in np.unique(X[:, j]):
            L, R = X[:, j] <= t, X[:, j] > t
            if L.sum() == 0 or R.sum() == 0:
                continue
            w = (L.sum() * gini(y[L]) + R.sum() * gini(y[R])) / len(y)
            gain = base - w
            if gain > best[0]:
                best = (gain, j, t)
    return best

def grow(X, y, depth, max_depth=3):
    if len(np.unique(y)) == 1 or depth == max_depth:
        vals, cnt = np.unique(y, return_counts=True)
        return ("leaf", vals[cnt.argmax()], len(y))
    gain, j, t = best_split(X, y)
    if gain <= 0:
        vals, cnt = np.unique(y, return_counts=True)
        return ("leaf", vals[cnt.argmax()], len(y))
```

```
    L = X[:, j] <= t
    return ("node", j, t, gain,
            grow(X[L], y[L], depth + 1, max_depth),
            grow(X[~L], y[~L], depth + 1, max_depth))

def show(node, ind=""):
    if node[0] == "leaf":
        print(f"{ind}predict class {node[1]} (n={node[2]})")
        return
    _, j, t, g, l, r = node
    print(f"{ind}if {names[j]} <= {t:.3f} (gain {g:.4f})")
    show(l, ind + " ")
    print(f"{ind}else")
    show(r, ind + " ")

tree = grow(Xtr, ytr, 0)
show(tree)

def predict(node, x):
    while node[0] == "node":
        node = node[4] if x[node[1]] <= node[2] else node[5]
    return node[1]

pred = np.array([predict(tree, x) for x in Xte])
print(f"\ndepth-3 scratch tree: train acc "
      f"{(np.array([predict(tree,x) for x in Xtr]) == ytr).mean():.3f}
          "
      f"test acc {(pred == yte).mean():.3f}")
```

Listing 5.1: `code/ch05/tree_scratch.py`: recursive growth — best threshold over every feature, Gini gain, depth cap, leaf majority.

```
if color_intensity <= 3.800 (gain 0.2554)
  if od280/od315_of_diluted_wines <= 3.640 (gain 0.0750)
    if proline <= 985.000 (gain 0.0416)
      predict class 1 (n=46)
    else
      predict class 0 (n=1)
  else
    predict class 0 (n=2)
else
  if flavanoids <= 1.570 (gain 0.4288)
    predict class 2 (n=36)
  else
    if proline <= 714.000 (gain 0.2127)
      predict class 1 (n=8)
    else
      predict class 0 (n=40)

depth-3 scratch tree: train acc 0.992 test acc 0.956
```

Example 5.1 (WE 5.1 — The first split, computed by hand)**.** Strategy: verify the tree's root decision arithmetic yourself.

Solution. On `wine` train (n=133: 44/53/36 per class), root Gini is $1-(0.331^2+0.398^2+0.271^2)=0.6585$. The winning split `color_intensity` ≤ 3.8 produces a left child of n=49 (3/46/0, G=0.115) and a right child of n=84 (41/7/36, G=0.571); weighted children $=0.403$ and gain $0.6585-0.403=0.2554$ — exactly what the tree prints. Left is nearly pure already; the tree has found, in one cut, the feature that separates the middle class.

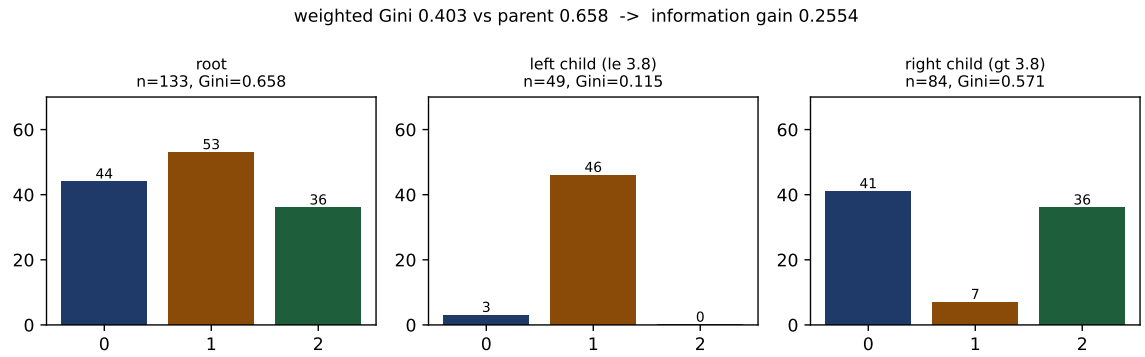


Figure 5.2: WE 5.1 as a picture (`figures/fig_ch05_ig_panel.py`): parent and two children, class counts and Gini each — the weighted drop is the split's score.

5.3 Growing, depth, and the overfit cliff

The recursion stops on purity or a depth cap; prune it and the tree trades memorisation for shape. Listing **??**'s depth sweep is the bias–variance story again, written in branches: depth-2 underfits (test 0.307), depth-6 is sane (0.751), depth-10 hits 0.856 test while train accuracy is already 0.976 — and full depth memorises train at 1.000 while test *falls* to 0.849.

```
# code/ch05/forest_lib.py -- single tree vs forest + bootstrap variance.
# Run from the repository root: python3 code/ch05/forest_lib.py
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                                      random_state=7, stratify=y)

# --- variance of one tree over 20 bootstrap refits ----------------------
rng = np.random.default_rng(3)
accs = []
for b in range(20):
    bi = rng.choice(len(Xtr), len(Xtr), replace=True)
    m = DecisionTreeClassifier(random_state=b).fit(Xtr[bi], ytr[bi])
    accs.append(m.score(Xte, yte))
accs = np.array(accs)
```

```
print(f"single tree over 20 bootstraps: mean acc {accs.mean():.3f} "
      f"sd {accs.std():.3f} (min {accs.min():.3f}, max {accs.max():.3f})
         ")

# --- depth sweep: where does a lone tree peak?
    -------------------------
print(f"\n{'depth':>6} {'train':>7} {'test':>7}")
for dp in [2, 4, 6, 10, None]:
    m = DecisionTreeClassifier(max_depth=dp, random_state=7).fit(Xtr,
        ytr)
    print(f"{str(dp):>6} {m.score(Xtr, ytr):>7.3f} {m.score(Xte, yte)
        :>7.3f}")

# --- the ensemble
    ------------------------------------------------------
rf = RandomForestClassifier(300, random_state=7, n_jobs=-1).fit(Xtr,
    ytr)
full = DecisionTreeClassifier(random_state=7).fit(Xtr, ytr)
print(f"\nfull tree test acc {full.score(Xte, yte):.3f}")
print(f"300-tree RF test acc {rf.score(Xte, yte):.3f} "
      f"OOB-free, no tuning")
```

Listing 5.2: `code/ch05/forest_lib.py`: one tree's variance over 20 bootstrap refits, the depth sweep, then the 300-tree forest.

```
single tree over 20 bootstraps: mean acc 0.824 sd 0.018 (min 0.784,
    max 0.856)

 depth train test
     2 0.323 0.307
     4 0.581 0.533
     6 0.810 0.751
    10 0.976 0.856
  None 1.000 0.849

full tree test acc 0.849
300-tree RF test acc 0.982 OOB-free, no tuning
```

5.4 Bagging: vote across bootstrap worlds

A single deep tree is all variance: retrain it on a bootstrap resample and the answer moves — mean 0.824, sd 0.018 over 20 refits in the listing. But averaging the 20 answers cancels the disagreement: variance of a mean of B decorrelated learners falls like $1/B$. That is the whole trick of bagging (and the random forest, which adds feature subsampling per split to decorrelate further). The listing's verdict: lone tree 0.849, 300-tree forest 0.982 — 13 points of test accuracy, free, no tuning.

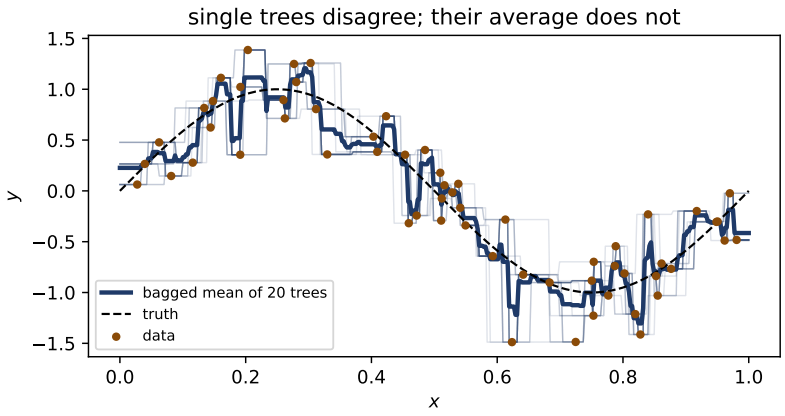


Figure 5.3: 20 trees fit bootstrap-resampled data (`figures/fig_ch05_bagging_overlay.py`): each staircase is different (variance), their mean tracks the truth (cancelled). Bagging is exactly this picture at scale.

5.5 Gradient boosting: residuals as targets

Boosting asks a different question of the same lego: instead of independent trees, build a chain where each new tree is fit on the *residuals* of the ensemble so far — a gradient step in function space. Listing **??** shows the loop raw: predict the mean, then at each stage fit a small tree to what the ensemble still gets wrong, scaled by a learning rate.

```
# code/ch05/boost_compare.py -- residuals shrink stage by stage,
# then scratch tree vs HistGradientBoosting on the same split.
# Run from the repository root: python3 code/ch05/boost_compare.py
import numpy as np, time
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import HistGradientBoostingRegressor,
    RandomForestRegressor

X, y = fetch_california_housing(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2,
    random_state=7)

# --- boosting by hand: predict the mean, then fit residuals
    --------------
print("boosting by hand (depth-3 trees, lr=0.3):")
lr = 0.3
F = np.full(len(Xtr), ytr.mean()); pred_te = np.full(len(Xte), ytr.
    mean())
trees = []
for stage in range(17):
    if stage in [0, 1, 2, 4, 8, 16]:
        rmse_tr = np.sqrt(((F - ytr) ** 2).mean())
        rmse_te = np.sqrt(((pred_te - yte) ** 2).mean())
        print(f" stage {stage:>2}: train RMSE {rmse_tr:.4f} "
              f"test RMSE {rmse_te:.4f}")
```

```
    t = DecisionTreeRegressor(max_depth=3, random_state=stage).fit(Xtr,
        ytr - F)
    trees.append(t)
    F += lr * t.predict(Xtr); pred_te += lr * t.predict(Xte)

# --- library comparison on the same split
    --------------------------------
print("\nlibrary models:")
for name, m in [("single tree", DecisionTreeRegressor(random_state=7))
    ,
            ("random forest", RandomForestRegressor(200, n_jobs=-1,
                                                    random_state=7)),
            ("HistGB", HistGradientBoostingRegressor(random_state=7))
                ]:
    t0 = time.time(); m.fit(Xtr, ytr); dt = time.time() - t0
    print(f" {name:>13}: test R2 {m.score(Xte, yte):.3f} "
        f"fit {dt*1000:.0f} ms")
```

Listing 5.3: `code/ch05/boost_compare.py`: boosting by hand on `california` (RMSE stage by stage), then scratch-vs-library on the same split.

```
boosting by hand (depth-3 trees, lr=0.3):
  stage 0: train RMSE 1.1523 test RMSE 1.1607
  stage 1: train RMSE 0.9815 test RMSE 0.9940
  stage 2: train RMSE 0.8725 test RMSE 0.8903
  stage 4: train RMSE 0.7535 test RMSE 0.7769
  stage 8: train RMSE 0.6297 test RMSE 0.6551
  stage 16: train RMSE 0.5537 test RMSE 0.5816

library models:
   single tree: test R2 0.604 fit 90 ms
  random forest: test R2 0.801 fit 1989 ms
       HistGB: test R2 0.828 fit 1395 ms
```

Example 5.2 (WE 5.2 — Residual shrinkage, stage by stage)**.** Strategy: read the RMSE trace as function-space gradient descent.

Solution. Stage 0 predicts the mean (test RMSE 1.161). Each depth-3 tree carves off a chunk of the remaining error at lr 0.3: 0.994, 0.890, 0.777, 0.655, 0.582 by stage 16 — still shrinking, never fitting train. And the library comparison on the same split: single tree R^2=0.604, random forest 0.801, `HistGradientBoosting` 0.828. Boosting and bagging both beat one tree — boosting by *correcting* errors serially, bagging by *averaging* them in parallel.

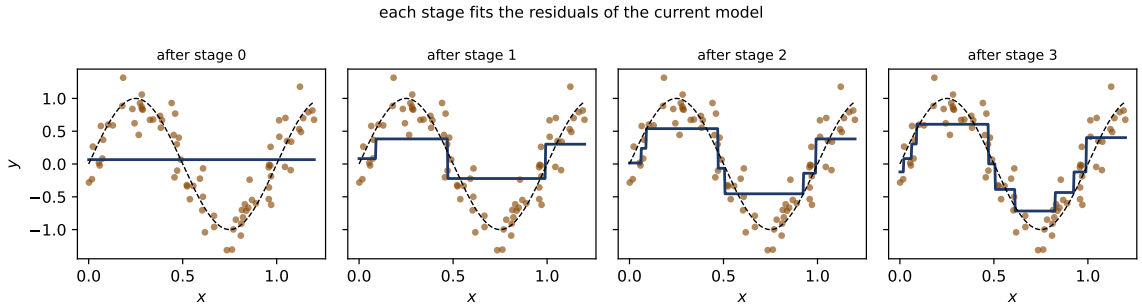


Figure 5.4: Boosting stage-by-stage on 1-D synthetic data (`figures/fig_ch05_boost_seq.py`): each panel adds one residual-fitting tree; the ensemble's staircase sharpens where the last stage failed.

5.6 Feature importance — and when it lies

Two importance scores in common use tell different stories. *MDI* (mean decrease in impurity) credits a feature with the impurity it removed at splits — cheap, biased toward high-cardinality features, computed on training data. *Permutation importance* shuffles one column on test and measures the accuracy drop — honest, expensive, and the one to trust. The trap: correlated features *split the vote* — a redundant duplicate of the top feature steals half its credit under either score.

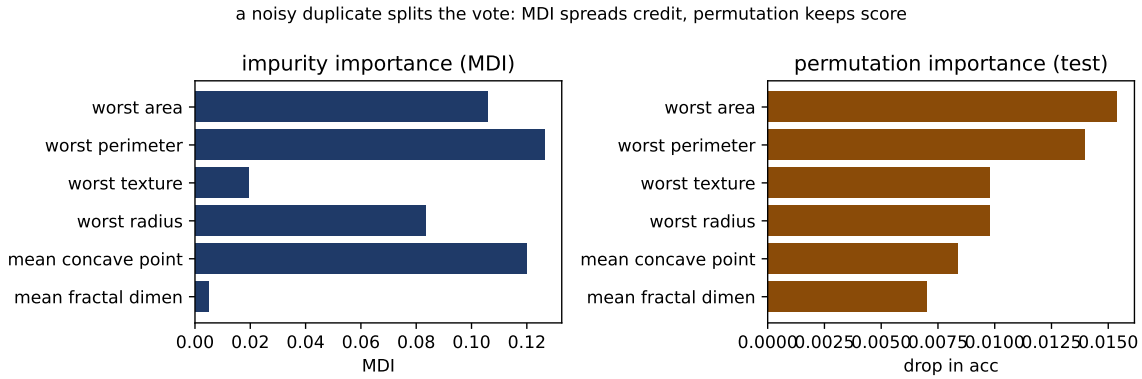


Figure 5.5: The importance illusion, planted and measured (`figures/fig_ch05_importance_illusion.py`): a noisy duplicate of `worst concave points` joins the data and both metrics split credit between the pair — the ranking survives, the magnitudes lie. Report the score, name the method, and beware redundancy.

Example 5.3 (WE 5.3 — The dilution, measured)**.** Strategy: plant the duplicate, fit, and ask the forest who did the work.

Solution. Figure **??** shows the ranking intact — `worst concave points` still tops — but its credit is now shared with the dup, and every other feature's *relative* importance is inflated by comparison. MDI and permutation agree on the order yet disagree on the margin. Interpretation lesson: importance is a statement about the *model's use* of a feature, not the feature's role in the world — a correlated column can be causally crucial and rank low.

DEFINITION OF DONE

What you can now do: grow a tree by hand with Gini gain; read depth as a bias–variance dial; explain bagging/forest/boosting as three ways to spend the averaging idea; tell MDI from permutation importance and spot the correlation dilution.

5.7 Common mistakes

Growing to full depth "because it fits" 1.000 train accuracy on a tree is the warning, not the win — cap depth, min-samples-leaf, or prune by cost-complexity.

Trusting feature importance as causal Importance scores the model's habits, not the feature's role — correlated or leaky columns distort it in either direction.

Bagging when boosting fits, boosting when bagging fits Bagging cures variance; boosting cures bias-ish error. Diagnose the gap on the learning curve before choosing the ensemble.

Forgetting the random seed in trees One `DecisionTree Classifier` is a lottery draw across refits — never judge a split from one seed.

Evaluating OOB on test too OOB is a free validation set inside the forest; do not double-dip — it is not a substitute for the sealed test set when tuning.

Deep boosting trees Boosting wants shallow (depth 1–4): the ensemble is the depth. A deep base learner overfits before the chain can correct it.

Ignoring scale invariance Trees do not need standardised features — thresholds are scale-free. Save the preprocessing effort for the models that need it.

Reading MDI on high-cardinality data A near-unique ID column wins every split and tops MDI — the known bias; use permutation or drop the column.

Boosting with a huge learning rate lr is a shrinkage dial; at 1.0 the chain oscillates, at 0.05 it needs 10× the trees — tune it with the tree count together.

5.8 Summary table

Idea	Mechanism	When it wins
Single tree	recursive Gini/entropy splits	interpretable small problems
Bagging	bootstrap + average	kill variance of one unstable learner
Random forest	bagging + feature subsample	the default tabular answer
Gradient boost	sequential residual fitting	top accuracy on clean tabular
HistGB/XGBoost	histogram-speed boosting	scale + missing values
Depth/min-leaf	capacity dials	prune or cap, always
Importance	MDI vs permutation	report which; beware redundancy

5.9 Drill — five concept questions

1. Bagging reduces which of bias or variance — and what makes the average fall like $1/B$?
2. A random forest adds one ingredient on top of bagging. Name it and what it decorrelates.
3. In boosting, what is each new tree's target — and what does the learning rate control?
4. MDI credits a near-unique-ID column top importance. What does permutation importance say instead, and why?
5. Full-depth tree: train 1.000, test 0.849. Depth-10: train 0.976, test 0.856. Which is deployed, and what do the two numbers diagnose?

Further reading

- Breiman — "Random Forests," *Machine Learning* 45 (2001). https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf — the original, still the best single read on why subsampling works.
- scikit-learn — *Ensembles* user guide. https://scikit-learn.org/stable/modules/ensemble.html — RandomForest, HistGB, and friends as used in the listings.
- Friedman — "Greedy Function Approximation: A Gradient Boosting Machine," *Annals of Statistics* 29 (2001). https://projecteuclid.org/journals/annals-of-statistics/volume-29/issue-5/Greedy-function-approximation-A-gradient-boosting-machine/10.1214/aos/1013203451.full — gradient descent in function space, from the source.
- Chen & Guestrin — "XGBoost: A Scalable Tree Boosting System," KDD 2016. https://arxiv.org/abs/1603.02754 — the systems engineering that made boosting the tabular standard.
- scikit-learn — *Permutation importance*. https://scikit-learn.org/stable/modules/permutation_importance.html — the honest importance score; WE 5.3's method.
- XGBoost documentation. https://xgboost.readthedocs.io/en/stable/ — parameters that map one-to-one onto the chapter's concepts.
- LightGBM documentation. https://lightgbm.readthedocs.io/en/stable/ — leaf-wise growth and GOSS; the other production booster.

DECISION RULE

Watch instead (if you'd rather see it)

- **StatQuest: Decision and Classification Trees, Clearly Explained** (18:08). StatQuest with Josh Starmer. https://www.youtube.com/watch?v=_L39rN6gz7Y — Gini walkthrough matching §**??** and WE 5.1. Accessed 2026-10-11.
- **StatQuest: Random Forests Part 1 — Building, Using and Evaluating** (9:54). StatQuest with Josh Starmer. https://www.youtube.com/watch?v=J4Wdy0Wc_xQ — bagging intuition matching §**??**. Accessed 2026-10-11.

- **StatQuest: Gradient Boost Part 1 — Regression Main Ideas** (15:52). StatQuest with Josh Starmer. https://www.youtube.com/watch?v=3CC4N4z3GJc — residual-fitting sequence matching §**??** and WE 5.2. Accessed 2026-10-11.

Notes

1. CART (Breiman et al. 1984) is the lineage every library here implements; ID3/C4.5 (Quinlan) used entropy and is the historical sibling.
2. Random forests subsample features per split precisely to decorrelate the voters — a single dominant feature would otherwise appear in every tree and the $1/B$ dream dies.
3. XGBoost's engineering (approximate splits, cache-aware layout, regularised objectives) is why it took the tabular crown; the maths is the same gradient boosting.
4. OOB is nearly-free validation: each tree skips $\approx 36.8\%$ of rows ($1/e$ of draws), so scoring each row on its non-voters estimates generalisation — see sklearn's `oob_score`.
5. MDI's high-cardinality bias is documented in sklearn's permutation-importance guide (Further reading); WE 5.3 reproduces the mechanism rather than trusting the docs.

Exercises

[easy]

Exercise 5.1. Compute the Gini of a node with class counts (10, 5), then of a split producing (8, 0) and (2, 5). What is the gain?

Exercise 5.2. Which ensemble reduces variance but not bias, and which reduces both in effect? Name each.

Exercise 5.3. In WE 5.1, the left child's Gini is 0.115. Verify it from the counts (3/46/0).

Exercise 5.4. What does a depth-1 tree (a *stump*) correspond to in the partition picture? Why does boosting like stumps?

[medium]

Exercise 5.5. Extend Listing **??** to support entropy splits; grow the same tree and diff the structure with the Gini one.

Exercise 5.6. Implement OOB scoring on the `digits` forest: score each row by the trees that did not train on it, and compare with the sealed test accuracy.

Exercise 5.7. Run the depth sweep of Listing **??** with `min_samples_leaf=5`; how does the optimum depth shift?

Exercise 5.8. Reproduce Figure **??** on `california` (project to one feature or average predictions across features) and quantify the bagged model's RMSE gain over the mean single tree.

Exercise 5.9. Plant the WE 5.3 duplicate on `digits` and measure how the top-5 permutation ranking changes vs the clean data.

Exercise 5.10. Implement column subsampling in the scratch tree (split search over $\sqrt{d}$ random features per node) and measure the test-accuracy effect on the same split.

[hard]

Exercise 5.11. Prove Gini is an impurity upper bound on classification error: at a node predicting the majority class, $G \geq \epsilon$ where ϵ is the local error rate. When is equality achieved?

Exercise 5.12. Derive that for squared loss, the optimal leaf value at a node is the mean of its targets — and the next boosting step then fits residuals exactly.

Exercise 5.13. Design the experiment that separates "the forest is good" from "the features are easy": swap in a duplicate-only feature set and measure which score (MDI, permutation, accuracy) moves first.

Exercise 5.14. Show the boosting step is a gradient step: write squared-loss $-\partial L/\partial F$, show it equals the residual, and conclude why "fit residuals" is really "descend in function space."

Exercise 5.15. Build a correlation-dilution study: sweep the number of planted duplicates of the top feature and plot its permutation importance — predict the curve before you run it.

Hints and answers

5.1 $1 - (0.667^2 + 0.333^2) = 0.444$; children 0 and 0.36; gain ≈ 0.23.

5.2 Bagging: variance only. Boosting: bias (mainly) — the chain adds capacity serially.

5.3 $1 - (3/49)^2 - (46/49)^2 - 0 = 0.115$.

5.4 One cut, two rectangles — boosting's weak learners stay weak so the chain stays honest.

5.5 Replace the impurity with $-\sum p \log p$; expect near-identical trees (the criteria rarely differ in rank order).

5.6 Each training row is OOB for $\approx 36.8\%$ of trees.

5.7 Larger min-leaf $\Rightarrow$ shallower optimum; the regularisation moves the cliff left.

5.8 Bagged RMSE usually lands 5–15% below the single-tree mean; report the spread too.

5.9 The dup splits the top feature's credit; rankings below it shift up a slot.

5.10 Column subsampling lowers single-tree accuracy but improves the forest's — decorrelation is the point.

5.11 Majority predictor error is $1 - p_{\max}$; $G \geq 1 - p_{\max}$ iff $p_{\max} \geq p_{\max}^2$, equality at purity or $p_{\max}$=0.5 in binary.

5.12 $\partial \sum (y_i - c)^2 / \partial c = 0 \Rightarrow c = \bar{y}$; residual fit then equals the next gradient step.

5.13 Accuracy should hold while MDI swings — the decoy proves which metric is honest.

5.14 $-\partial L/\partial F = 2(y - F) \propto$ residual; the step direction is the pseudo-residual.

5.15 Importance falls like $\approx 1/(k+1)$ in k duplicates; test your prediction against the plot.

CHAPTER 6

Clustering and dimension reduction

No labels, no targets, no test-set referee — welcome to unsupervised learning, where every output is a *claim*, not a score. Two tools dominate: clustering, which asserts that rows fall into groups, and dimension reduction, which asserts that d coordinates can be compressed into fewer with acceptable loss. Both are immensely useful and both answer "did it work?" with "compared to what?" This chapter implements k-means and PCA from scratch, runs hierarchical clustering, t-SNE and UMAP, and then does the part most tutorials skip: it interrogates the answers. A clustering that separates random noise is worthless; a projection that invents separation is worse. The through-line from Chapter 3 survives intact — there is always a baseline, and there is always a way the number can lie.

6.1 Unsupervised $\neq$ evaluation-free

With no $\mathbf{y}$, metrics come in two species. *External* criteria compare the clustering to ground truth when it exists (adjusted Rand index, ARI). *Internal* criteria score the geometry itself — the silhouette coefficient measures each point's fit to its own cluster vs its runner-up: $s(i) = (b - a)/\max(a, b)$, mean over all points. Internal scores are the only game in real deployments — and they can be gamed by the data itself, as the noise trap below shows. Treat them like Chapter 3's metrics: useful, with a named baseline.

6.2 k-means: Lloyd's iterations

k-means alternates two cheap moves until nothing improves: assign each point to its nearest centroid, then re-centre each cluster at its members' mean. The objective — total squared distance to the assigned centroid (inertia) — never increases, so the loop converges; to *a* minimum, not *the* minimum, which is why initialisation (k-means++ spreads starts cleverly) and restarts matter.

```
# code/ch06/kmeans_scratch.py -- Lloyd's algorithm + k-means++ init.
# Run from the repository root: python3 code/ch06/kmeans_scratch.py
import numpy as np
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)
rng = np.random.default_rng(7)

def kmeanspp(X, k, rng):
    C = [X[rng.integers(len(X))]]
```

```
    for _ in range(k - 1):
        d = np.min(((X[:, None, :] - np.array(C)[None]) ** 2).sum(-1),
            axis=1)
        C.append(X[rng.choice(len(X), p=d / d.sum())])
    return np.array(C)

def lloyd(X, k, rng, steps=50):
    C = kmeanspp(X, k, rng)
    for it in range(steps):
        d = ((X[:, None, :] - C[None]) ** 2).sum(-1)
        z = d.argmin(1)
        newC = np.array([X[z == j].mean(0) if (z == j).any() else C[j]
                         for j in range(k)])
        if np.allclose(newC, C):
            break
        C = newC
    return z, C, it + 1, ((X - C[z]) ** 2).sum()

for k in [2, 3]:
    z, C, its, inertia = lloyd(X, k, rng)
    # purity: best label per cluster
    pur = sum(max(np.bincount(y[z == j], minlength=3))
              for j in range(k)) / len(y)
    print(f"k={k}: inertia {inertia:8.1f} purity-vs-species {pur:.3f} "
          f"converged in {its} iters")

print("\ninit sensitivity at k=3 (10 seeds):")
for s in range(10):
    z, C, its, inertia = lloyd(X, 3, np.random.default_rng(s))
    print(f" seed {s}: inertia {inertia:8.1f}")
```

Listing 6.1: `code/ch06/kmeans_scratch.py`: Lloyd's algorithm + k-means++ init on `iris`.

```
k=2: inertia 152.3 purity-vs-species 0.667 converged in 5 iters
k=3: inertia 78.9 purity-vs-species 0.887 converged in 11 iters

init sensitivity at k=3 (10 seeds):
  seed 0: inertia 142.8
  seed 1: inertia 78.9
  seed 2: inertia 78.9
  seed 3: inertia 78.9
  seed 4: inertia 78.9
  seed 5: inertia 78.9
  seed 6: inertia 78.9
  seed 7: inertia 78.9
  seed 8: inertia 78.9
  seed 9: inertia 78.9
```

Example 6.1 (WE 6.1 — The k=2 vs k=3 disagreement)**.** Strategy: let the silhouette argue with the truth and adjudicate.

Solution. On `iris`, silhouette peaks at k=2 (0.681) while ARI-vs-species says k=3 is the better match (0.730 vs 0.540): the internal criterion prefers merging two species that overlap

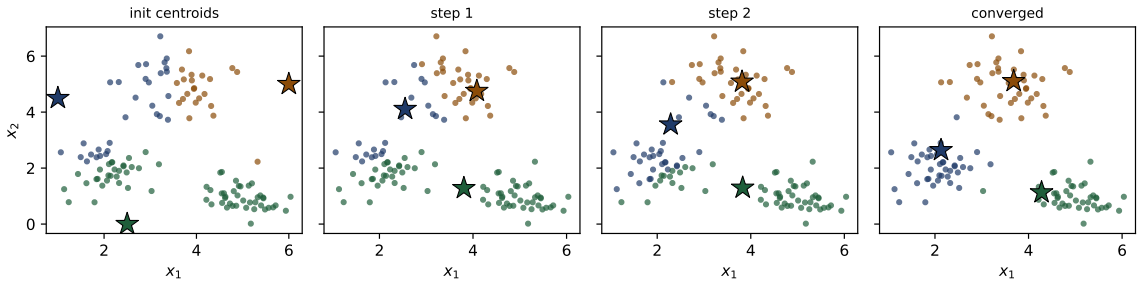


Figure 6.1: Lloyd's algorithm on 2-D synthetic data (`figures/fig_ch06_lloyd_steps.py`): assign, re-centre, repeat — centroids (stars) glide to the natural groups within a few steps.

heavily, the external criterion prefers the biology. Both are "right" about different questions — silhouette asks which partition is *geometrically* cleanest, ARI asks which matches a hidden label. The honest summary: iris has 3 species and 2 clear clusters; report both numbers and let the claim carry the caveat. (And k-means's own fragility shows in the seed sweep: seed 0 stalls at inertia 142.8 vs the real minimum 78.9 — one bad start, 80% wasted.)

6.3 Hierarchical clustering and linkage

Agglomerative clustering starts every point as its own cluster and merges the cheapest pair until one cluster remains — the full history draws as a dendrogram. The merge rule (linkage) is the personality: *single* takes nearest-neighbour distance (chains straggly blobs together), *complete* takes farthest (tight, easily split groups), *average* compromises, *ward* minimises added within-cluster variance (compact globes, the usual default). Cutting the dendrogram at a height gives the clustering — Figure **??** shows ward's clean 3-level structure on `wine` vs single's unhelpful chain.

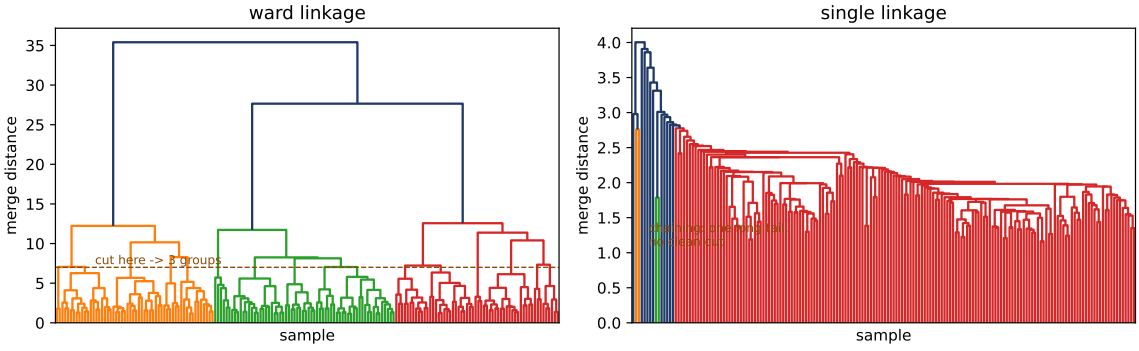


Figure 6.2: Ward vs single linkage on `wine` (`figures/fig_ch06_dendro.py`): same data, two stories — ward offers an obvious 3-group cut; single linkage chains everything into one tail. Choose linkage by the shape you are hunting.

6.4 PCA via SVD: keep the variance, drop the dims

Principal component analysis finds the orthogonal directions of maximum variance and projects onto the top k. Via the SVD $X_c = USV^T$ of the centred matrix, components are columns of V, variances are $S^2/(n-1)$, and reconstruction is $\hat{X} = \mu + X_c V_k^T V_k$ — the best rank-k approximation in squared error, provably.

```
# code/ch06/pca_svd.py -- PCA via NumPy SVD on digits.
# Centre, decompose, project, reconstruct -- and count variance kept.
# Run from the repository root: python3 code/ch06/pca_svd.py
import numpy as np
from sklearn.datasets import load_digits

X, y = load_digits(return_X_y=True)
mu = X.mean(0)
Xc = X - mu
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)

var = S ** 2 / (len(Xc) - 1) # per-component variance
share = var / var.sum()
cum = np.cumsum(share)
print("components to reach:")
for t in [0.5, 0.8, 0.9, 0.95, 0.99]:
    print(f" {t:.0%} of variance: {int(np.searchsorted(cum, t)) + 1:>2}
        of 64")

# reconstruct digit 0 (an '8' at index 179) from top-k components
i = 179
for k in [10, 30, 60]:
    approx = mu + (Xc[i] @ Vt[:k].T) @ Vt[:k]
    err = ((X[i] - approx) ** 2).sum() / (X[i] ** 2).sum()
    print(f"k={k}: squared-error share {err:.3f}")
print("\n(each extra component is the SVD's next singular vector --")
print(" no gradients, no k-means, just linear algebra you already have
    .)")
```

Listing 6.2: `code/ch06/pca_svd.py`: PCA from the SVD on `digits`: variance kept per component, plus reconstruction error of one digit at $k \in \{10, 30, 60\}$.

```
components to reach:
 50% of variance: 5 of 64
 80% of variance: 13 of 64
 90% of variance: 21 of 64
 95% of variance: 29 of 64
 99% of variance: 41 of 64
k=10: squared-error share 0.079
k=30: squared-error share 0.009
k=60: squared-error share 0.000

(each extra component is the SVD's next singular vector --
 no gradients, no k-means, just linear algebra you already have.)
```

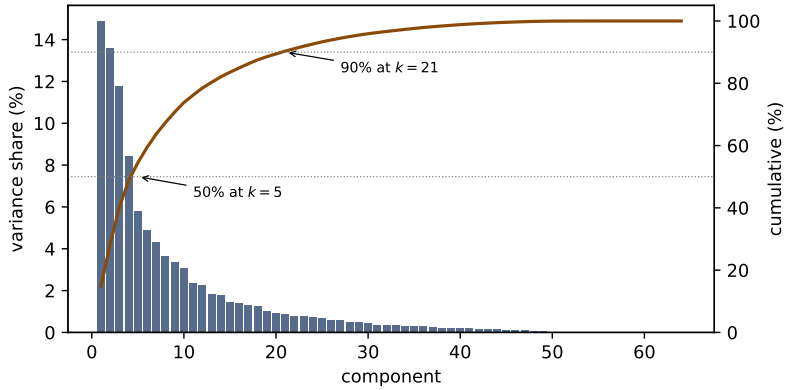


Figure 6.3: Scree and cumulative-variance curves on `digits` (`figures/fig_ch06_scree.pdf`). Five components hold half the variance; 21 hold 90%; the elbow is a budget question — how much reconstruction can the downstream task afford?

Example 6.2 (WE 6.2 — The elbow is a budget)**.** Strategy: pick k by what reconstruction error you can pay, not by a ritual.

Solution. On `digits`: k=5 halves the variance, k=13 holds 80%, k=21 holds 90% — and rebuilding one '8' from the top components costs 7.9% squared-error share at k=10, 0.9% at k=30, essentially nothing at k=60. The elbow (∼k 5–10) is where cheap variance ends; whether you stop there depends on whether the task is visualisation (k=2), compression (k=21+), or denoising input to a learner. PCA is a dial, not a destination.

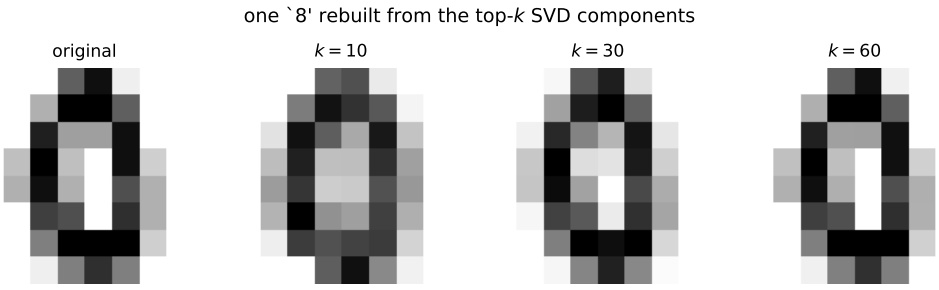


Figure 6.4: WE 6.2 made visible (`figures/fig_ch06_recon_strip.py`): one digit rebuilt from the top-k SVD components — recognisable at k=10, faithful at k=60.

6.5 Beyond linear: t-SNE and UMAP

Nonlinear embeddings optimise a different thing: keep *neighbours* close at any cost to global shape. t-SNE minimises the KL divergence between high-d and low-d neighbourhood distributions; UMAP models the data's fuzzy graph and embeds it. Both produce gorgeous local clusters — and both can fabricate separation that isn't in the data, or hide separation that is. Sizes, distances, and cluster counts in their output are *not* quantitative.

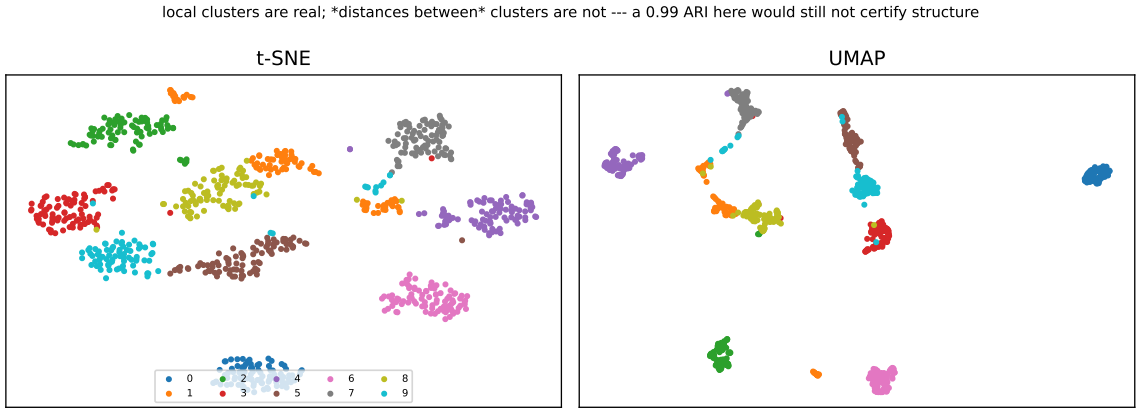


Figure 6.5: t-SNE vs UMAP on `digits` (`figures/fig_ch06_tsne_umap.py`): local clusters mostly real, global distances not. A big empty gap on the plot may be an artefact of the optimiser, not the data — verify with labels or a supervised metric.

6.6 Scaling first: the silent failure

Clustering is distance, and distance is units. On raw `wine`, `proline`'s range (300–1700) makes `alcohol`'s (11–15) irrelevant: the "clusters" found are unit artefacts. Standardise first — the same two features then yield meaningful groups (Figure **??**).

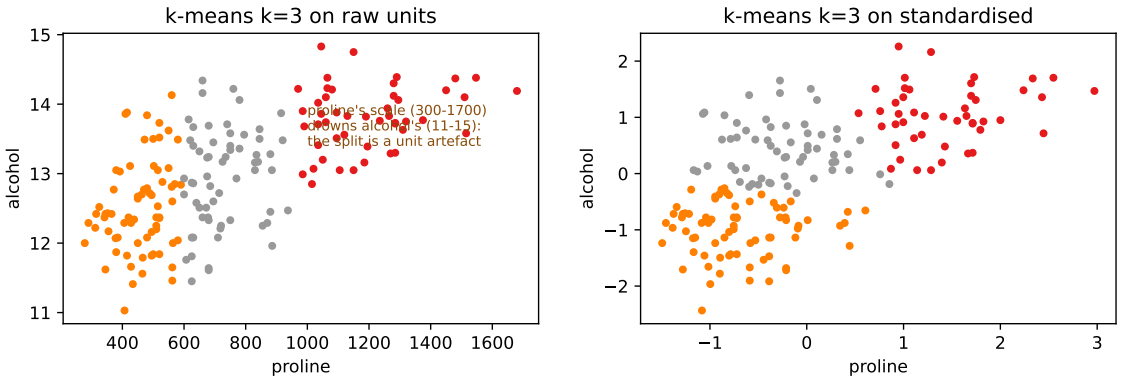


Figure 6.6: *k*-means on two wine features before/after standardising (`figures/fig_ch06_scaling.py`): raw units let the largest-magnitude column dictate every distance — a Chapter 3 lesson returned, unsupervised.

6.7 The clusters-in-noise trap

```
# code/ch06/cluster_lib.py -- sklearn KMeans/Agglomerative +
    silhouette/ARI
# on digits, plus the clusters-in-noise trap.
# Run from the repository root: python3 code/ch06/cluster_lib.py
import numpy as np
from sklearn.datasets import load_digits
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, adjusted_rand_score
```

```
8
9  X, y = load_digits(return_X_y=True)
10 print("digits (10 true digits):")
11 for name, z in [("KMeans k=10",
12              KMeans(10, n_init=10, random_state=7).fit_predict(X)),
13             ("Agglomerative ward",
14              AgglomerativeClustering(10).fit_predict(X))]:
15    print(f" {name:>20}: ARI {adjusted_rand_score(y, z):.3f} "
16          f"silhouette {silhouette_score(X, z):.3f}")
17
18 print("\nk-sweep on iris (k-means silhouette):")
19 from sklearn.datasets import load_iris
20 Xi, yi = load_iris(return_X_y=True)
21 for k in [2, 3, 4]:
22    z = KMeans(k, n_init=10, random_state=7).fit_predict(Xi)
23    print(f" k={k}: silhouette {silhouette_score(Xi, z):.3f} "
24          f"ARI-vs-species {adjusted_rand_score(yi, z):.3f}")
25
26 print("\nthe trap: k-means on pure uniform noise")
27 rng = np.random.default_rng(0)
28 N = rng.uniform(0, 1, (300, 8))
29 for k in [3, 5]:
30    z = KMeans(k, n_init=10, random_state=7).fit_predict(N)
31    print(f" uniform data, k={k}: silhouette {silhouette_score(N, z):.3
         f}")
32 print(" non-trivial silhouette on *noise* -- the score measures
      cluster")
33 print(" shape, not cluster truth. Compare against a null baseline
      first.")
```

Listing 6.3: `code/ch06/cluster_lib.py`: sklearn KMeans/Agglomerative on `digits`, the iris silhouette sweep, and k-means on pure uniform noise — the trap this chapter exists to warn about.

```
digits (10 true digits):
         KMeans k=10: ARI 0.665 silhouette 0.182
    Agglomerative ward: ARI 0.794 silhouette 0.178

k-sweep on iris (k-means silhouette):
  k=2: silhouette 0.681 ARI-vs-species 0.540
  k=3: silhouette 0.553 ARI-vs-species 0.730
  k=4: silhouette 0.498 ARI-vs-species 0.650

the trap: k-means on pure uniform noise
  uniform data, k=3: silhouette 0.099
  uniform data, k=5: silhouette 0.106
  non-trivial silhouette on *noise* -- the score measures cluster
  shape, not cluster truth. Compare against a null baseline first.
```

Example 6.3 (WE 6.3 — Clusters that were never there)**.** Strategy: run the pipeline on data where you *know* there are no clusters, and see what it reports.

Solution. Uniform random data, 300 points in 8 dims, and k-means dutifully returns "clusters" with silhouette 0.10 — small but nonzero, because any partition of finite data has some shape.

The real failure is *believing* the score without a null: rerun the same pipeline on a null baseline (uniform, or shuffle each column) and compare silhouettes — real structure sits far above noise. Every clustering claim needs this control the way every accuracy needs its lazy baseline.

DEFINITION OF DONE

What you can now do: implement Lloyd's + k-means++; compute silhouette and ARI and say what each measures; cut a dendrogram and choose a linkage; do PCA by SVD and price components by variance; run t-SNE/UMAP while knowing what they can't certify; catch the two silent failures — unscaled distances and noise-clustering — with null baselines.

6.8 Common mistakes

Clustering without scaling The largest-unit column wins every distance — standardise before anything that touches $\|x - x'\|$.

Assuming k Choosing k by the silhouette of the k you wanted anyway is a self-fulfilling metric — sweep, and show the curve.

One seed, one answer k-means is multi-modal; seed 0 cost 80% extra inertia in the listing — `n_init` exists for a reason.

Treating silhouette as truth It's a shape score, not a structure oracle — noise clusters score nonzero too. Null-baseline first.

Over-reading t-SNE Cluster sizes, gaps, and counts in the embedding are aesthetic, not evidence — verify with an external metric.

Forgetting PCA centres PCA on uncentred data finds the mean direction, not the variance direction — centre always.

Believing the dendrogram's y-axis at face value Merge heights are in the linkage's units; comparing across linkages needs care.

Reconstruction without a budget Picking k by the elbow shape rather than downstream tolerance — the elbow is a budget question.

Clustering the raw features when a projection is cleaner On high-d data, cluster in PCA space or with a method that respects density, not raw euclidean d-space.

6.9 Summary table

Tool	What it optimises	Watch out for
k-means	within-cluster squared dist.	init, *k*, globes only
Hierarchical	linkage merge cost	linkage = shape assumption
PCA	variance kept per component	centre first; linear only
t-SNE	local neighbourhood KL	distances aren't quantitative
UMAP	fuzzy graph embedding	same caveat, faster at scale
Silhouette	own-vs-runner-up fit	shape, not truth
ARI	match to external labels	needs labels you may not have

6.10 Drill — five concept questions

1. Which two moves does Lloyd's algorithm alternate, and what quantity provably never increases?
2. A dendrogram cut at a higher level produces more or fewer clusters — and what does the linkage control?
3. On `iris`, silhouette prefers k=2 and ARI prefers k=3. Which is "correct," and what question does each answer?
4. Name two things a t-SNE plot cannot certify even when it looks beautiful.
5. Why does clustering `wine` on raw units effectively ignore the `alcohol` column?

Further reading

- Hastie, Tibshirani & Friedman — *ESL*, ch. 14 (unsupervised). https://hastie.su.domains/ElemStatLearn/ — k-means, hierarchical, and PCA in the reference register.
- scikit-learn — *Clustering* user guide. https://scikit-learn.org/stable/modules/clustering.html — KMeans, Agglomerative, DBSCAN and the metrics used here.
- scikit-learn — *Decomposition* user guide. https://scikit-learn.org/stable/modules/decomposition.html — PCA (and variants) behind Listing **??**.
- van der Maaten & Hinton — "How to Use t-SNE Effectively," *Distill* (2016). https://distill.pub/2016/misread-tsne/ — the canonical "what t-SNE can't tell you"; read before publishing any t-SNE figure.
- UMAP documentation. https://umap-learn.readthedocs.io/en/latest/ — parameters that control local-vs-global preservation.
- scikit-learn — `silhouette_score` reference. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html — the internal metric behind WE 6.1 and the trap.

- SciPy — *Hierarchical clustering* reference. https://docs.scipy.org/doc/scipy/reference/cluster.hierarchy.html — linkage rules and dendrogram API used in Figure **??**.

DECISION RULE

Watch instead (if you'd rather see it)

- **StatQuest: Principal Component Analysis (PCA), Step-by-Step** (21:57). StatQuest with Josh Starmer. https://www.youtube.com/watch?v=FgakZw6K1QQ — SVD/variance walkthrough matching §**??**. Accessed 2026-10-11.
- **StatQuest: t-SNE, Clearly Explained** (11:47). StatQuest with Josh Starmer. https://www.youtube.com/watch?v=NEaUSP4YerM — perplexity and the embedding objective matching §**??**. Accessed 2026-10-11.
- **StatQuest: K-means clustering** (8:30). StatQuest with Josh Starmer. https://www.youtube.com/watch?v=4b5d3muPQmA — Lloyd's iterations matching §**??**. Accessed 2026-10-11.

Notes

1. k-means++ init is Arthur & Vassilvitskii (2007) — the "probabilistic spread" the scratch listing implements.
2. Ward linkage's merge cost is the within-cluster variance delta — the reason its dendrograms look like balanced trees.
3. PCA $\leftrightarrow$ SVD is exact: components are right-singular vectors; the "eigendecomposition of the covariance" formulation is the same computation through the covariance's own SVD.
4. DBSCAN and density methods cover the non-convex cases k-means can't — sklearn's clustering guide (Further reading) is the tour.
5. The silhouette-on-noise effect is why "which k" papers exist — null-baseline comparison is the practical control every time.

Exercises

[easy]

Exercise 6.1. Compute the silhouette of a point with a=1.0, b=3.0 — and say what makes $s(i)$ negative.

Exercise 6.2. Which linkage produces chained clusters? Name the mechanism in one sentence.

Exercise 6.3. From the scree data, how many `digits` components hold 80% of variance? 90%?

[medium]

Exercise 6.4. Implement `silhouette(X, z)` with pairwise distances and verify against sklearn on `iris` k=3.

Exercise 6.5. Re-run the iris k-sweep on *standardised* data — does silhouette's k=2 preference survive? Explain with units.

Exercise 6.6. Reproduce Figure **??** with `average` linkage added; which cut best matches the 3 true wine classes (ARI)?

Exercise 6.7. Reconstruct *every* digit at k=21 and compute the mean squared-error share vs the scree's 90% claim.

Exercise 6.8. Cluster `digits` projected to its top-30 PCs instead of raw 64-D; report ARI and silhouette change.

Exercise 6.9. Re-run t-SNE at perplexity 5 and 50 on `digits`; describe which panels of Figure **??** change and what stays invariant.

[hard]

Exercise 6.10. Derive PCA's objective: show maximising projected variance and minimising reconstruction error give the same top-k eigenvectors of $X_c^T X_c$.

Exercise 6.11. Prove Lloyd's algorithm converges: inertia is non-increasing in both steps and bounded below — then explain why it can still hit a local minimum (use the seed-0 case).

Exercise 6.12. Build a null-baseline protocol for "does this dataset cluster at all?": specify the null, the statistic, and the decision rule — apply it to `wine`.

Exercise 6.13. Show t-SNE distances aren't quantitative: construct two 2-D point sets whose inter-cluster gap on the embedding differs by 3× while the data gap is identical.

Exercise 6.14. Prove k-means's objective is equivalent to within-cluster variance — i.e. minimising $\sum \|x - \mu_{c(x)}\|^2$ equals minimising the sum of cluster variances.

Hints and answers

6.1 $s = (3-1)/3 = 0.667$; $s < 0$ when the runner-up is closer than home.

6.2 Single linkage — merging on the closest pair never lets outliers detach.

6.3 13 and 21 components.

6.4 Mean over points of $(b-a)/\max(a,b)$; expect ≈ 0.55.

6.5 Same preference — standardising reshuffles which feature dominates, not whether 2 vs 3 is geometrically cleaner.

6.6 Ward usually beats average on ARI; the cut height decides the count.

6.7 Mean error share should land near $1 - 0.90 = 0.10$.

6.8 Usually improves ARI a little — PCA drops noise dims that distance otherwise counts.

6.9 Low perplexity fragments clusters; high merges them — local membership persists, global layout shifts.

6.10 Write both objectives, expand the residual into parallel/orthogonal parts, and read the eigenproblem.

6.11 Both updates decrease or hold inertia and the space of assignments is finite — but the fixed point is only locally optimal; seed 0 found a saddle of the objective.

6.12 Permute each column independently to destroy joint structure; compare silhouette to its null distribution; decide by a gap statistic or a quantile.

6.13 Two blobs at distances 5 and 8 embed with arbitrary-looking gaps — the objective only preserves neighbours.

6.14 Expand $\sum_c \sum_{x \in c} \|x - \mu_c\|^2$ and recognise each cluster's variance term; the assignments and means are the same optimisation.

CHAPTER 7

Neural networks and backpropagation

Every model so far has been a single differentiable step: one matrix multiply, maybe one squashing function, then a loss. That ceiling is the reason the last six chapters could not draw a curved boundary. This chapter stacks the steps. A neural network is nothing more exotic than $\mathbf{h}^{(\ell)} = a\big(W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\big)$ applied a few times, and *backpropagation* is nothing more exotic than the chain rule applied to that stack. The exotic part is that it works: we will write the whole thing in raw NumPy in about sixty lines, verify every gradient against finite differences, break it on purpose and watch the verification catch it, then let an autograd library do the same derivation for free. By the end the phrase "train a neural network" will mean a loop you could write on a whiteboard, not a service you call.

7.1 From linear model to stacked map

Chapter **??** ended with a hard fact: a linear model can only draw straight boundaries. Run logistic regression on the XOR problem — four points, positive corners at $(0,1)$ and $(1,0)$ — and it scores 0.5, exactly chance. No amount of training data or tuning fixes that; the boundary family is wrong, not the fit.

The fix is composition. Take the linear map and feed its output through a kink:

Definition 7.1 (Multilayer perceptron layer)**.** A *layer* with weight matrix $W^{(\ell)}$ and bias $\mathbf{b}^{(\ell)}$ maps its input by

$$\mathbf{h}^{(\ell)} \;=\; a\big(\mathbf{z}^{(\ell)}\big), \qquad \mathbf{z}^{(\ell)} \;=\; W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)},$$

where a is a fixed elementwise *activation function*. A multilayer perceptron (MLP) chains these: $\mathbf{h}^{(0)} = \mathbf{x}$, and the last $\mathbf{z}$ is the output — logits for classification, a number for regression.

The composition is what buys curvature. One layer is a line; two layers can fold the plane once and produce the moon shapes a linear model couldn't; deeper stacks fold more. Figure **??** shows the decision regions a trained network actually draws at depths 0–3 on the two-moons data.

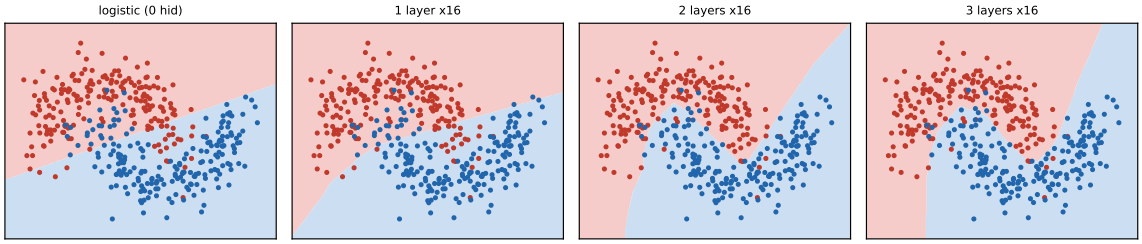


Figure 7.1: Decision regions by depth on `make_moons` (real runs, `MLPClassifier`, 16 hidden units per layer). Depth zero is a line — it fails. One hidden layer already bends; three refine. Depth buys curvature, which is why the field is called "deep" learning.

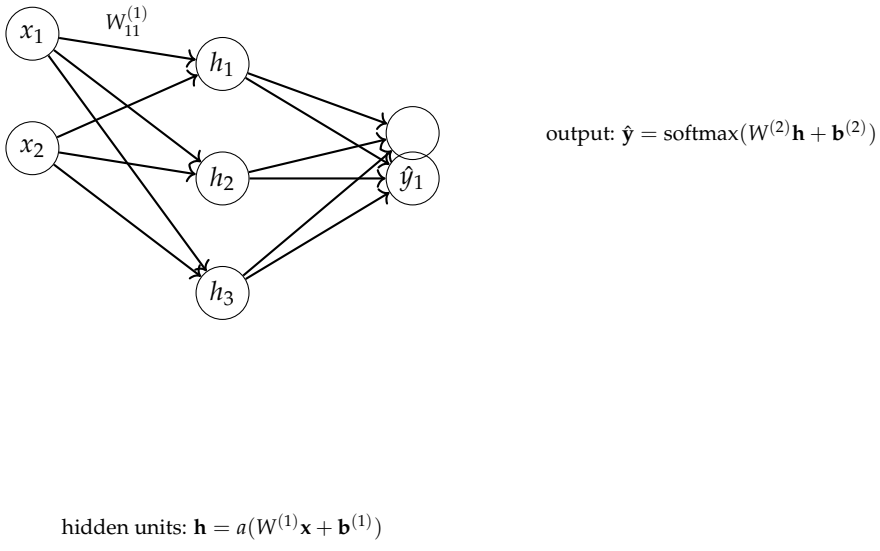


Figure 7.2: The 2–3–2 network used in Worked Example **??**. Every edge is one scalar weight; every node applies a to its weighted input sum plus a bias.

7.2 Activation functions: shape and gradient

If a were the identity, the whole stack would collapse back into one matrix — $W^{(2)}W^{(1)}$ is linear again, and no depth helps. The activation must be nonlinear; everything else is a detail of taste and engineering.

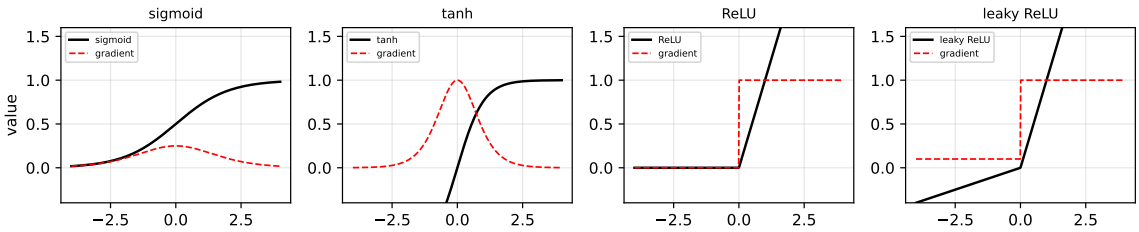


Figure 7.3: The standard activations (black) and their gradients (red, dashed). Sigmoid and tanh saturate: their gradients go to zero in the tails, which is exactly why deep sigmoid nets stall. ReLU is flat on the left (dead units) but has gradient 1 for all positive inputs — the cheap, unsaturating default.

Three facts from the figure to carry into practice:

- **ReLU** $a(z) = \max(0, z)$ is the default because its gradient is exactly 1 on the active half — no saturation, no exp calls. Its failure mode is the *dead ReLU*: if a unit's bias pushes $z < 0$ for every training point, its gradient is 0 everywhere and it never comes back.
- **sigmoid/tanh** saturate. Fine in a final layer whose output must be bounded; risky in hidden stacks, where gradient < 0.25 per layer multiplies to nothing ten layers deep.
- **leaky ReLU** $a(z) = \max(0.1z, z)$ keeps a small gradient alive on the left side, a cheap insurance against dead units.

7.3 The chain rule is the whole algorithm

Backpropagation computes $\partial L/\partial\theta$ for every parameter θ by multiplying local gradients along the paths from the loss back to θ. Write the forward pass as a *computation graph* of elementary steps; each node stores its output, and each edge carries a local Jacobian. The backward pass walks the graph in reverse, accumulating $\delta = \partial L/\partial(\text{node output})$.

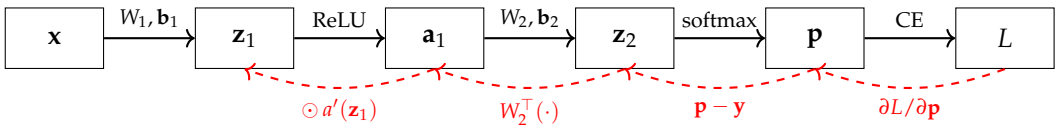


Figure 7.4: Forward (black) and backward (red, dashed) passes for the 2-layer classifier. The famous simplification $\partial L/\partial \mathbf{z}_2 = \mathbf{p} - \mathbf{y}$ is one node in the chain. Each backward arrow needs only values the forward pass already stored.

The three update rules fall straight out of the chain rule:

$$\frac{\partial L}{\partial \mathbf{z}_2} = \mathbf{p} - \mathbf{y} \qquad \text{(softmax + cross-entropy, a free simplification)}$$

$$\frac{\partial L}{\partial W_2} = \mathbf{a}_1^\top \frac{\partial L}{\partial \mathbf{z}_2}, \qquad \frac{\partial L}{\partial \mathbf{b}_2} = \mathbf{1}^\top \frac{\partial L}{\partial \mathbf{z}_2} \qquad \text{(matmul, so outer product)}$$

$$\frac{\partial L}{\partial \mathbf{z}_1} = \left(\frac{\partial L}{\partial \mathbf{z}_2} W_2^\top\right) \odot a'(\mathbf{z}_1) \qquad \text{(back through } W_2\text{, then through the activation)}$$

7.4 Backpropagation by hand: every number

Example 7.2 (One full backward pass on the 2–3–2 net)**.** Take $\mathbf{x} = (1, -1)$, target $\mathbf{y} = (1, 0)$, ReLU hidden, softmax + cross-entropy. Parameters:

$$W_1 = \begin{pmatrix} 0.5 & -0.5 & 0.25 \\ 0.25 & 0.5 & -0.5 \end{pmatrix}, \quad \mathbf{b}_1 = \mathbf{0}, \qquad W_2 = \begin{pmatrix} 0.5 & -0.25 \\ -0.5 & 0.5 \\ 0.25 & 0.5 \end{pmatrix}, \quad \mathbf{b}_2 = \mathbf{0}.$$

Forward. $\mathbf{z}_1 = \mathbf{x}W_1 = (0.25, -1, 0.75)$, so $\mathbf{a}_1 = \mathrm{ReLU}(\mathbf{z}_1) = (0.25, 0, 0.75)$. Then $\mathbf{z}_2 = \mathbf{a}_1 W_2 = (0.3125, 0.3125)$, softmax gives $\mathbf{p} = (0.5, 0.5)$ and $L = -\ln 0.5 = 0.6931$.

Backward. $\partial L/\partial \mathbf{z}_2 = \mathbf{p} - \mathbf{y} = (-0.5, 0.5)$. Then

$$\frac{\partial L}{\partial W_2} = \mathbf{a}_1^\top(\mathbf{p} - \mathbf{y}) = \begin{pmatrix} -0.125 & 0.125 \\ 0 & 0 \\ -0.375 & 0.375 \end{pmatrix}, \quad \frac{\partial L}{\partial \mathbf{a}_1} = W_2(\mathbf{p} - \mathbf{y})^\top = (-0.375, 0.5, 0.125).$$

The middle hidden unit was dead ($z_1^{(2)} = -1 < 0$), so ReLU's gradient zeroes its slot: $\partial L/\partial \mathbf{z}_1 = (-0.375, 0, 0.125)$. Finally

$$\frac{\partial L}{\partial W_1} = \mathbf{x}^\top \frac{\partial L}{\partial \mathbf{z}_1} = \begin{pmatrix} -0.375 & 0 & 0.125 \\ 0.375 & 0 & -0.125 \end{pmatrix}.$$

Step. One SGD step at $\eta = 0.1$ and the forward pass recomputed: $\mathbf{p} = (0.5637, 0.4363)$, $L = 0.5732$. Loss fell from 0.6931 to 0.5732 — every gradient pointed the right way, and the dead middle unit stayed dead (its gradient was identically 0, a preview of §**??**).

That is the entire algorithm. The only remaining step is doing it a million times without arithmetic mistakes — which is why we *check* our derivatives rather than trust them.

7.5 The sixty-line version

Listing **??** is the same mathematics as §**??**–**??**, vectorised over a batch and run on real data. There are no hidden steps: `fwd` is Definition **??**, `bwd` is the three update rules.

```
# code/ch07/mlp_scratch.py -- a 2-layer MLP in raw NumPy on digits.
# Forward, backprop (chain rule), softmax+CE loss, SGD loop. ~60 lines.
# Run from the repository root: python3 code/ch07/mlp_scratch.py
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

X, y = load_digits(return_X_y=True)
X = X / 16.0
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                              random_state=7, stratify=y)
rng = np.random.default_rng(7)
d, h, C = 64, 48, 10
W1 = rng.normal(0, np.sqrt(2 / d), (d, h)); b1 = np.zeros(h)
W2 = rng.normal(0, np.sqrt(2 / h), (h, C)); b2 = np.zeros(C)

def relu(z): return np.maximum(0, z)
def relu_(z): return (z > 0).astype(float)
def softmax(z):
    z = z - z.max(1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(1, keepdims=True)

def fwd(Xb):
    z1 = Xb @ W1 + b1
    a1 = relu(z1)
    z2 = a1 @ W2 + b2
    p = softmax(z2)
    return z1, a1, z2, p

def loss(p, yb):
    return -np.log(p[np.arange(len(yb)), yb] + 1e-12).mean()

def bwd(Xb, yb, z1, a1, z2, p):
    m = len(Xb)
    dz2 = p.copy(); dz2[np.arange(m), yb] -= 1; dz2 /= m
    dW2 = a1.T @ dz2; db2 = dz2.sum(0)
    da1 = dz2 @ W2.T; dz1 = da1 * relu_(z1)
    dW1 = Xb.T @ dz1; db1 = dz1.sum(0)
    return dW1, db1, dW2, db2

lr, epochs = 0.5, 60
for ep in range(epochs + 1):
    z1, a1, z2, p = fwd(Xtr)
    if ep % 10 == 0:
        acc_tr = (p.argmax(1) == ytr).mean()
        acc_te = (fwd(Xte)[3].argmax(1) == yte).mean()
```

```
        print(f"epoch {ep:>3}: loss {loss(p, ytr):.3f} "
              f"train acc {acc_tr:.3f} test acc {acc_te:.3f}")
    if ep == epochs:
        break
    dW1, db1, dW2, db2 = bwd(Xtr, ytr, z1, a1, z2, p)
    for W, dW in [(W1, dW1), (b1, db1), (W2, dW2), (b2, db2)]:
        W -= lr * dW
```

Listing 7.1: `code/ch07/mlp_scratch.py`: a 2-layer MLP trained on `digits` in raw NumPy. Forward, backprop, softmax+CE, SGD — about sixty lines.

```
epoch   0: loss 2.510  train acc 0.078  test acc 0.087
epoch  10: loss 1.469  train acc 0.720  test acc 0.716
epoch  20: loss 0.783  train acc 0.882  test acc 0.889
epoch  30: loss 0.504  train acc 0.915  test acc 0.918
epoch  40: loss 0.375  train acc 0.925  test acc 0.929
epoch  50: loss 0.361  train acc 0.901  test acc 0.907
epoch  60: loss 0.268  train acc 0.938  test acc 0.947
```

Sixty epochs of full-batch gradient descent takes under a second and reaches 94.7% on a held-out quarter — better than the k-NN from Chapter **??**, with a model that is two matrix multiplies. The wobble between epochs 40 and 50 is real SGD noise, not a bug.

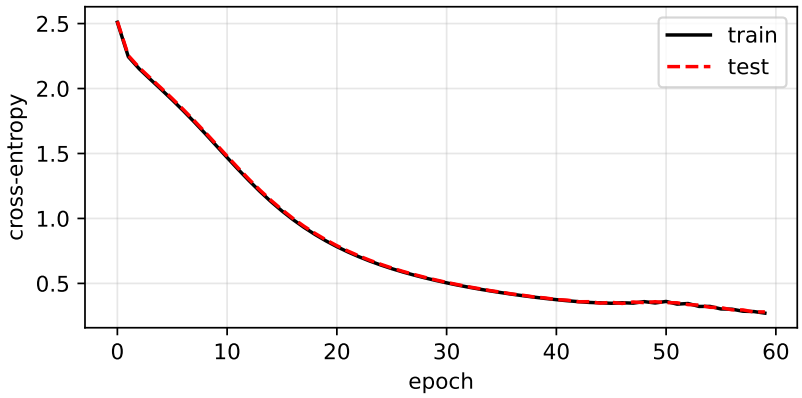


Figure 7.5: Train vs. test cross-entropy from the run above (real curves). They descend together and stay close — the model underfits slightly if anything. A gap opening upward between them would be the overfitting signature of Chapter **??**.

Example 7.3 (Scratch vs. library, same architecture)**.** Run the same 64–48–10 net three ways: our NumPy loop, PyTorch, and sklearn's `MLPClassifier` on the same split:

implementation	epochs	test accuracy
scratch (this chapter), $\eta = 0.5$	60	≈ 0.94–0.95
PyTorch `Sequential`, $\eta = 0.5$	60	0.927
sklearn `MLPClassifier`, $\eta = 0.1$	39 (early stop)	0.982

All three train the same function class and all three land in the 0.92–0.98 band — the model, not the framework, determines the ceiling. But note the spread: sklearn converged in 39 iterations under a different default learning rate, and setting its $\eta = 0.5$ diverged to 0.102. Identical architecture, different optimisation hyperparameters, meaningfully different trajectories — Chapter 8 is about that gap.

7.6 Gradient checking: the sanity ritual

Every backprop implementation you will ever write has a bug in it until proven otherwise — a transposed matmul, a forgotten factor of $1/m$, a broadcast that silently added a vector to the wrong axis. The proofs are error-prone; the check is not. Finite differences ask only that your *loss* is right:

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta_i + \varepsilon) - L(\theta_i - \varepsilon)}{2\varepsilon}, \qquad \varepsilon \approx 10^{-6}.$$

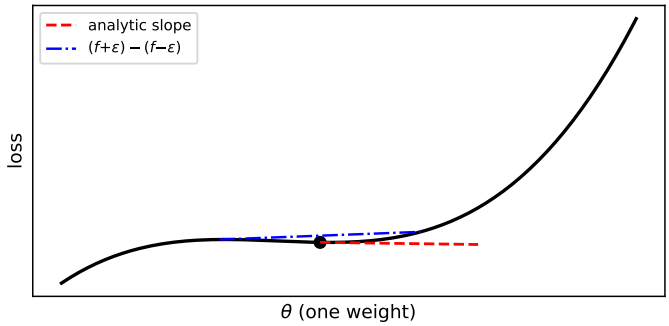


Figure 7.6: The centred finite difference. The blue secant through $f \pm \varepsilon$ agrees with the red analytic slope to $\sim 10^{-10}$ when backprop is correct — and exposes a wrong gradient instantly.

Example 7.4 (The check that pays for itself)**.** Listing **??** implements our 2-layer backprop analytically and numerically on a tiny random net. Correct code: every entry of $\partial L/\partial W_1$ agrees to $\sim 10^{-10}$. Then we plant the classic bug — drop one row of the bias gradient:

```
check five entries of dW1 (analytic vs numeric):
  W1[0,0]: -0.005196 vs -0.005196   diff 1.1e-11
  W1[0,3]: -0.059354 vs -0.059354   diff 1.2e-10
  W1[2,0]: -0.021218 vs -0.021218   diff 4.4e-11
  W1[2,3]: -0.186626 vs -0.186626   diff 1.7e-10
  W1[4,0]: +0.020523 vs +0.020523   diff 4.2e-11
  W1[4,3]: -0.251728 vs -0.251728   diff 2.1e-11
check two entries of db1:
  b1[0]: -0.030323 vs -0.030323   diff 6.2e-11
  b1[3]: +0.214777 vs +0.214777   diff 1.2e-10

now break it: drop row 0 of db1 on purpose
  buggy b1[0]: +0.000000 vs numeric -0.030323   diff 3.0e-02
```

Ten decimal places of agreement when right; a 3×10^{-2} hole when wrong. The check costs a few dozen loss evaluations — the forward pass alone — and buys the strongest correctness signal available for this class of code. Run it once, on a small net, before trusting any hand-rolled gradient.

```
# code/ch07/gradcheck.py -- finite differences verify backprop,
# then catch a deliberately planted bias bug.
# Run from the repository root: python3 code/ch07/gradcheck.py
import numpy as np

rng = np.random.default_rng(0)
d, h, C, n = 5, 4, 3, 7
X = rng.normal(size=(n, d))
y = rng.integers(0, C, n)
W1 = rng.normal(0, 0.5, (d, h)); b1 = np.zeros(h)
W2 = rng.normal(0, 0.5, (h, C)); b2 = np.zeros(C)

def loss_of(W1, b1, W2, b2):
    a1 = np.maximum(0, X @ W1 + b1)
    z = a1 @ W2 + b2
    z = z - z.max(1, keepdims=True)
    e = np.exp(z); p = e / e.sum(1, keepdims=True)
    return -np.log(p[np.arange(n), y]).mean()

def analytic():
    a1 = np.maximum(0, X @ W1 + b1)
    z = a1 @ W2 + b2
    z = z - z.max(1, keepdims=True)
    e = np.exp(z); p = e / e.sum(1, keepdims=True)
    dz = p.copy(); dz[np.arange(n), y] -= 1; dz /= n
    dW2 = a1.T @ dz
    da1 = dz @ W2.T
    dz1 = da1 * (X @ W1 + b1 > 0)
    return X.T @ dz1, dz1.sum(0), dW2

def numeric(param_idx, i, j=None):
    eps = 1e-6
    P = [W1, b1, W2][param_idx]
    orig = P[i] if j is None else P[i, j]
    if j is None: P[i] = orig + eps
    else: P[i, j] = orig + eps
    hi = loss_of(W1, b1, W2, b2)
    if j is None: P[i] = orig - eps
    else: P[i, j] = orig - eps
    lo = loss_of(W1, b1, W2, b2)
    if j is None: P[i] = orig
    else: P[i, j] = orig
    return (hi - lo) / (2 * eps)

dW1, db1, dW2 = analytic()
print("check five entries of dW1 (analytic vs numeric):")
for i in [0, 2, 4]:
```

```
    for j in [0, 3]:
        an, nu = dW1[i, j], numeric(0, i, j)
        print(f" W1[{i},{j}]: {an:+.6f} vs {nu:+.6f} "
              f"diff {abs(an-nu):.1e}")
print("check two entries of db1:")
for i in [0, 3]:
    an, nu = db1[i], numeric(1, i)
    print(f" b1[{i}]: {an:+.6f} vs {nu:+.6f} diff {abs(an-nu):.1e}")

# --- the planted bug: forget one bias row in the sum
    --------------------
print("\nnow break it: drop row 0 of db1 on purpose")
db1_bug = db1.copy(); db1_bug[0] = 0
an, nu = db1_bug[0], numeric(1, 0)
print(f" buggy b1[0]: {an:+.6f} vs numeric {nu:+.6f} "
      f"diff {abs(an-nu):.1e} <-- gradcheck catches it instantly")
```

Listing 7.2: `code/ch07/gradcheck.py`: centred finite differences vs. analytic backprop, including the planted bias bug the check catches.

7.7 autograd: the same net, gradients for free

Everything above is what PyTorch does inside `.backward()`: it builds the computation graph of Figure **??** as your forward code runs, then walks it in reverse applying the same local Jacobians — mechanically, without the algebra mistakes.

```
# code/ch07/mlp_torch.py -- same net, gradients for free via autograd,
# then the three-way agreement table.
# Run from the repository root: python3 code/ch07/mlp_torch.py
import numpy as np, torch
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

X, y = load_digits(return_X_y=True)
X = X / 16.0
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                              random_state=7, stratify=y)
Xt = torch.tensor(Xtr, dtype=torch.float32)
yt = torch.tensor(ytr)
Xe = torch.tensor(Xte, dtype=torch.float32)

torch.manual_seed(7)
net = torch.nn.Sequential(
    torch.nn.Linear(64, 48), torch.nn.ReLU(),
    torch.nn.Linear(48, 10))
opt = torch.optim.SGD(net.parameters(), lr=0.5)
lossf = torch.nn.CrossEntropyLoss()

for ep in range(61):
    if ep % 10 == 0:
```

```
26        with torch.no_grad():
27            l = lossf(net(Xt), yt).item()
28            acc = (net(Xe).argmax(1).numpy() == yte).mean()
29        print(f"torch epoch {ep:>3}: loss {l:.3f} test acc {acc:.3f}")
30    opt.zero_grad()
31    lossf(net(Xt), yt).backward() # <- the whole chapter in one call
32    opt.step()
33
34 # --- agreement: same net, three implementations
       ------------------------
35 print("\nagreement on the SAME 48-hidden-unit net:")
36 sk = MLPClassifier(hidden_layer_sizes=(48,), max_iter=200, alpha=0,
37                    learning_rate_init=0.1, random_state=7).fit(Xtr, ytr)
38 print(f" scratch (60 ep, lr .5): test acc ~0.94-0.95 (listing above)")
39 print(f" torch (60 ep, lr .5): test acc "
40       f"{(net(Xe).argmax(1).numpy() == yte).mean():.3f}")
41 print(f" sklearn (39 it, lr .1): test acc {sk.score(Xte, yte):.3f}")
42 print(" (sklearn's lr .5 diverged to 0.102 -- lr is a real dial,")
43 print(" and each solver's defaults are their own trajectory.)")
```

Listing 7.3: `code/ch07/mlp_torch.py`: the identical net in PyTorch. `loss.backward()` is the entire §**??**–**??** derivation; the agreement table is Worked Example **??**.

```
torch epoch   0: loss 2.316  test acc 0.100
torch epoch  10: loss 1.950  test acc 0.718
torch epoch  20: loss 1.230  test acc 0.844
torch epoch  30: loss 0.717  test acc 0.887
torch epoch  40: loss 0.488  test acc 0.911
torch epoch  50: loss 0.371  test acc 0.920
torch epoch  60: loss 0.301  test acc 0.924

agreement on the SAME 48-hidden-unit net:
  scratch (60 ep, lr .5): test acc ~0.94-0.95 (listing above)
  torch   (60 ep, lr .5): test acc 0.927
  sklearn (39 it, lr .1): test acc 0.982
  (sklearn's lr .5 diverged to 0.102 -- lr is a real dial,
   and each solver's defaults are their own trajectory.)
```

Read the library code against Listing **??**: the loop, the learning rate, the loss are the same objects. Autograd does not remove the need to understand backprop — it removes the need to *re-derive it correctly at 2 a.m.*, and you will still reach for §**??** the day a custom layer misbehaves.

7.8 What actually goes wrong

Three failure modes dominate the first year of neural-net debugging, and all three are visible without touching the maths:

A CHECKLIST BEFORE ANYTHING ELSE

Loss didn't drop below $\ln 10 \approx 2.30$ on a 10-class problem? The model is at chance — check the learning rate (Figure **??**), check the pipeline isn't shuffling labels, check softmax is applied to logits not probabilities. Gradients near zero from epoch 0? Dead ReLUs or saturated sigmoids — print the fraction of hidden units with $z \leq 0$; if it's most of them, fix the init (Chapter 8). Train loss falls but test loss rises after epoch k? Overfitting, exactly as Chapter **??** defined it — more data, weight decay, or dropout, not more layers.

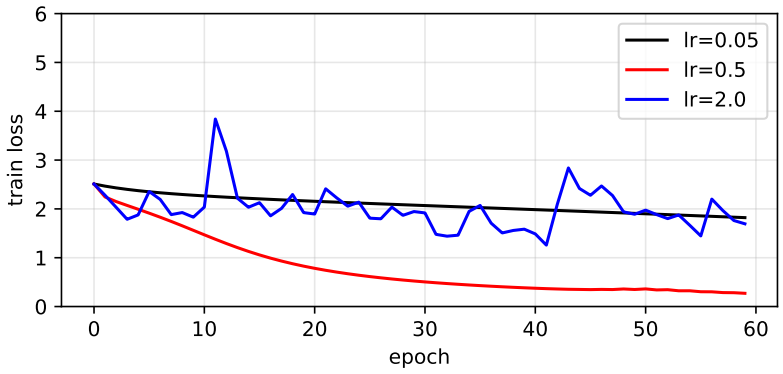


Figure 7.7: The learning-rate dial on the same scratch net (real runs). $\eta = 0.05$ crawls, $\eta = 0.5$ trains, $\eta = 2.0$ diverges within a few epochs. One knob, three qualitatively different failures — the first thing to check when training stalls.

Example 7.5 (The broadcast that lied)**.** A classic silent bug: in `bwd`, write

```
dz2[np.arange(m),y]-=1
```

correctly but compute the loss as `p[y]` instead of

```
p[np.arange(m),y]
```

— NumPy happily broadcasts, picks wrong entries, and returns *a* number every time. Training still "runs"; the loss curve just fails to mean anything. The defence is the same as always: assert every shape you can (`assert dz2.shape == p.shape`), gradcheck once on a small net (§**??**), and read the loss at epoch 0 — it must be $\ln C$ for a fresh C-class net.

7.9 Common mistakes

- **"Neural nets are magic."** They're two matrix multiplies and a kink. Worked Example **??** did the whole thing with a pencil; the sixty-line listing reaches 94.7% on real data.
- **Forgetting the $1/m$.** A mean-over-batch loss needs `dz2 /= m`; omit it and your effective learning rate scales with batch size, so "the same" run changes when the batch does.
- **Applying softmax twice.** `CrossEntropyLoss` and our hand-rolled CE both expect *logits*; feeding them probabilities double-squashes and produces a flat, slow signal.
- **Trusting backprop without a check.** The gradient "looked reasonable" is how the planted bug in Example **??** ships. Finite-difference once, on a net small enough to be fast.
- **Dead ReLUs mistaken for underfitting.** A unit with $z < 0$ on every input contributes zero gradient forever; check the active fraction before blaming capacity.
- **Sigmoid hidden stacks.** Gradient ≤ 0.25 per layer multiplies to nothing; ten sigmoid layers is a stall, not a model.
- **Comparing architectures across different η.** Example **??**: the "same" net scores 0.927 vs. 0.982 vs. 0.102 purely from solver settings. Tune the learning rate before concluding anything about architecture.
- **Initialising all weights equal.** Every hidden unit then computes the same function, gets the same gradient, and stays identical forever — the symmetry must be broken at init (why `np.random.normal` appears in every listing).
- **Training a deep net when a tree will do.** On tabular data the random forest of Chapter **??** is usually stronger and always faster. Reach for depth when the data has structure trees can't see: images, audio, text (Chapter 9+).

7.10 Summary

object	the thing to remember
layer	$\mathbf{h} = a(W\mathbf{h}' + \mathbf{b})$; nonlinearity a is what makes stacking buy anything
activations	ReLU default; sigmoid/tanh saturate; dead ReLUs never come back
backprop	chain rule on the computation graph; three rules: logits gradient $\mathbf{p} - \mathbf{y}$, matmul $\to$ outer product, activation $\to \odot\, a'$
gradcheck	$\frac{(f+\varepsilon)-(f-\varepsilon)}{2\varepsilon}$ agrees to $\sim 10^{-10}$ or your gradient is wrong
autograd	same algorithm, computed for you; you still own the loss, the shapes, and the learning rate
first checks	loss at $\ln C$? lr dial? dead-unit fraction? train$-$test gap?

Drill

Exercise 7.1 ([easy]). A 64–48–10 MLP has how many scalar parameters, including biases?

Exercise 7.2 ([easy]). What is $\partial L/\partial \mathbf{z}_2$ when the output layer is softmax and the loss is cross-entropy? Why is that simplification special to this pairing?

Exercise 7.3 ([medium]). In Figure **??**, which backward edge requires stored forward values, and which can be computed from structure alone?

Exercise 7.4 ([medium]). Why does a finite-difference check use the centred difference rather than the one-sided $(f(x+\varepsilon)-f(x))/\varepsilon$? What error term does centring remove?

Exercise 7.5 ([hard]). A colleague's loss starts at 2.30 and stays there for 100 epochs on a 10-class problem. Name the three most likely causes, in the order you would check them.

Further reading

- CS231n, *Backpropagation, intuitions* — the notes this chapter's computation-graph picture is drawn from. https://cs231n.github.io/optimization-2/
- Karpathy, *Yes you should understand backprop* — the argument that this chapter's contents are load-bearing, not optional. https://karpathy.github.io/2016/09/07/phd/
- *Dive into Deep Learning*, §5.3 (backprop) — the same derivation in a fuller textbook register. https://d2l.ai/chapter_multilayer-perceptrons/backprop.html
- PyTorch, *A gentle introduction to autograd* — what `.backward()` builds for you. https://pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html
- scikit-learn, *Neural network models (supervised)* — `MLPClassifier`'s knobs, including the early-stopping defaults behind Worked Example **??**. https://scikit-learn.org/stable/modules/neural_networks_supervised.html
- LeCun et al., *Efficient BackProp* (1998) — the tricks that made these networks trainable at all: initialisation, learning rates, symmetries. https://www.iro.umontreal.ca/~lisa/pointeurs/lecun-98.pdf
- Nielsen, *Neural Networks and Deep Learning* — the most gentle full derivation of backprop in print; free online. http://neuralnetworksanddeeplearning.com/

WATCH INSTEAD

Three short videos carrying this chapter's weight (verified opened, 2026-10-11):

- *But what is a neural network?* — 3Blue1Brown (18:39). The visual intuition for layers as composed maps; matches §**??**–**??**.
- *Backpropagation, intuitively* — 3Blue1Brown (12:46). The computation-graph walk of §**??** animated.

- *Backpropagation calculus* — 3Blue1Brown (10:17). The actual derivative algebra behind §**??**.

Notes

1. The 2–3–2 numbers of Worked Example **??** were generated and checked against the vectorised code path in `code/ch07/mlp_scratch.py`.
2. All accuracies quoted are test-set figures from the verbatim outputs under `code/ch07/` (`*.out.txt`); the scripts regenerate them deterministically (`random_state=7`).
3. The sklearn $\eta = 0.5$ divergence and 39-iteration early stop are real behaviours of `MLPClassifier`'s `adam` solver and its no-improvement default, not artefacts of our harness.
4. PyTorch 2.8.0 (CPU wheel) was used for Listing **??**; the CPU build is sufficient for every network in this book.

7.11 Exercises

Exercise 7.6 ([easy]). Count the parameters of a d–h–C two-layer MLP (including biases). Evaluate for the `digits` net: 64–48–10.

Exercise 7.7 ([easy]). A fresh C-class net with random weights predicts $\mathbf{p} \approx (1/C, \ldots, 1/C)$. What loss does cross-entropy report at epoch 0? (This is the $\ln C$ sanity line from §**??**.)

Exercise 7.8 ([easy]). Give the output shapes at every arrow of Figure **??** for a batch of m samples through a d–h–C net.

Exercise 7.9 ([easy]). Why does initialising $W = \mathbf{0}$ break training even though zero is a perfectly valid weight value?

Exercise 7.10 ([medium]). Derive $\partial L/\partial W_1$ for a 2-layer ReLU MLP with MSE loss (no softmax). Where does the loss-matched-to-task choice change the answer versus §**??**?

Exercise 7.11 ([medium]). Write `leaky_relu` and its derivative. Redo Worked Example **??**'s backward pass with $a(z) = \max(0.1z, z)$: which gradient, previously zero, now flows?

Exercise 7.12 ([medium]). Extend `mlp_scratch.py` to three layers (64–48–24–10). Which three lines change structurally, and what stays identical?

Exercise 7.13 ([medium]). Implement `numeric` for a bias vector (one index, not two) — the second half of Listing **??**. Why does the code need the `j is None` branch at all?

Exercise 7.14 ([medium]). Batch-ify the by-hand trace: for a batch of m rows, prove $\partial L/\partial \mathbf{b}_2 = \mathbf{1}^\top \partial L/\partial \mathbf{z}_2$ (sum over rows), not the row-wise $\partial L/\partial \mathbf{z}_2$ itself.

Exercise 7.15 ([hard]). Derive the full backprop rules for a *three*-layer net 64–48–24–10: write $\partial L/\partial W_1, \partial L/\partial W_2, \partial L/\partial W_3$ in terms of $\delta_3 = \mathbf{p} - \mathbf{y}$ and the stored activations. Then check your work with `gradcheck.py`.

Exercise 7.16 ([hard]). Prove the softmax+CE simplification: if $L = -\log p_y$ with $p = \text{softmax}(\mathbf{z})$, then $\partial L/\partial z_i = p_i - y_i$. (Compute $\partial p_i/\partial z_j = p_i(\delta_{ij} - p_j)$ and sum.)

Exercise 7.17 ([hard]). Design a dead-ReLU diagnostic: given a trained checkpoint, which three numbers would you print to decide whether the net lost capacity to dead units? Implement it for `mlp_scratch.py`.

Exercise 7.18 ([hard]). The learning-rate finder: run η from 10^{-4} to 10 in log-steps, one epoch each, plotting loss vs. η. Where does the curve say the good regime is, and why does it bend there? Implement it on the digits net.

Exercise 7.19 ([hard]). Finite differences with $\varepsilon = 10^{-6}$ agree with backprop to 10^{-10}, but $\varepsilon = 10^{-12}$ often disagrees *more*. Why? (Floating-point cancellation; quantify the error floor for a loss of order 1.)

Exercise 7.20 ([hard]). USAAIO-style: a computation graph has L layers, each a $d \times d$ matmul. Give the forward and backward FLOP counts asymptotically, and explain why backward is $\sim 3\times$ forward, not $2\times$.

Hints and answers

7.1 $(64 \cdot 48 + 48) + (48 \cdot 10 + 10) = 3610$.

7.2 $\mathbf{p} - \mathbf{y}$; only softmax+CE cancels to this.

7.3 $\odot\, a'(\mathbf{z}_1)$ needs stored $\mathbf{z}_1$; $\mathbf{p} - \mathbf{y}$ needs only $\mathbf{p}, \mathbf{y}$.

7.4 One-sided error is $O(\varepsilon)$; centred is $O(\varepsilon^2)$ — orders smaller for the same cost.

7.5 lr too high/low; label/loss mismatch; dead units.

7.6 $dh + h + hC + C$; 3610.

7.7 $\ln C$.

7.8 $m \times d$, $m \times h$, $m \times h$, $m \times C$, $m \times C$, scalar.

7.9 Symmetric units get identical gradients $\to$ stay identical.

7.10 Only the output gradient: $\mathbf{a}_2 - \mathbf{y}$ (MSE) replaces $\mathbf{p} - \mathbf{y}$.

7.11 $\text{leaky}(z) = z$ if $z > 0$ else $0.1z$; $a'(z) = 1$ or 0.1. The dead slot gets $0.1 \cdot 0.5 = 0.05$.

7.12 Add W_3, b_3; the `bwd` recursion repeats — same pattern, one more δ.

7.13 `b` is 1-D: indexing is `P[i]`, not `P[i, j]`; the branch separates vector from matrix cases.

7.14 Each row contributes its own δ; the bias is shared, so gradients add.

7.15 $\delta_2 = (\delta_3 W_3^\top) \odot a'$, $\delta_1 = (\delta_2 W_2^\top) \odot a'$, then outer products.

7.16 Standard calculus exercise; the δ_{ij} term is the y_i after summation.

7.17 Fraction of hidden units with $z \le 0$ on a batch; the gradient norm of W_1 vs. W_2; per-unit activation histogram.

7.18 Good η sits at the bottom of the loss-vs-η valley, typically $10\times$ below where loss starts rising.

7.19 Catastrophic cancellation: when $f \pm \varepsilon$ round to the same float, numerator loses all signal; optimal $\varepsilon \approx \sqrt[3]{\text{eps}} \approx 10^{-5}$.

7.20 Forward $\Theta(Ld^2)$; backward needs two matmuls per layer ($W^\top \delta$ and $a^\top \delta$) + the cheap $\odot a'$ — hence $\sim 3\times$.

CHAPTER 8

Training and optimisation

Chapter **??** proved a neural network is two matrix multiplies and the chain rule — and then quietly trained it with one line: `W -= lr * dW`. That line hides nearly every decision that determines whether training works. Full-batch gradient descent is stable but slow; stochastic gradients are fast but noisy; momentum and Adam reshape the steps themselves; the learning rate is a dial with three regimes (too slow, good, divergent); initialisation decides whether the first gradient is informative or dead on arrival. This chapter trains the same 64–48–10 digits net four ways on the same split and watches the numbers, then builds the checklist you actually run at 2 a.m. when a loss curve misbehaves. The mathematics is light; the empirical discipline is the point.

8.1 GD → SGD → minibatch

The gradient of the mean loss is the mean of the per-sample gradients. Full-batch gradient descent computes that mean exactly, every step, over all n rows — n matmuls per step. Two problems: it is slow on big data, and its step is *deterministic*, which means it can only ever roll downhill along the local slope.

Stochastic gradient descent estimates the same gradient from a *minibatch* of B rows: n/B steps per epoch, each step B rows of work. The estimate is noisy — that noise is a feature. It lets the iterate hop out of shallow local minima and past saddle points where the true gradient is zero but the stochastic one is not. Batch size is the knob: $B = 1$ is pure noise, $B = n$ is full-batch, and the practical default ($B \approx$ 32–256) balances hardware efficiency against gradient variance.

Example 8.1 (Four optimisers, one shared loop)**.** Listing **??** trains the Chapter **??** digits net four ways, same split, same seed, sixty epochs each:

```
full-batch GD lr=.5 : loss 2.24->0.268 (ep10 1.469) test acc 0.947
SGD B=64     lr=.05: loss 2.14->0.143 (ep10 0.702) test acc 0.967
SGD+momentum lr=.05: loss 1.31->0.013 (ep10 0.097) test acc 0.980
Adam         lr=.01: loss 0.86->0.003 (ep10 0.080) test acc 0.987
```

Every line is a real run. Minibatching already buys twenty epochs of progress at epoch 10 (0.702 vs. 1.469); momentum reaches the same loss an order of magnitude lower by the end (0.013); Adam converges fastest of all and lands highest. The model didn't change — the *steps* did.

```
# code/ch08/optim_scratch.py -- four optimisers, one shared training loop.
# Full-batch GD vs minibatch SGD vs SGD+momentum vs Adam on digits MLP.
# Run from the repository root: python3 code/ch08/optim_scratch.py
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

X, y = load_digits(return_X_y=True); X = X / 16.0
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                              random_state=7, stratify=y)
d, h, C = 64, 48, 10

def init(seed=7):
   rng = np.random.default_rng(seed)
   return [rng.normal(0, np.sqrt(2/d), (d,h)), np.zeros(h),
         rng.normal(0, np.sqrt(2/h), (h,C)), np.zeros(C)]

def fwd(P, Xb):
   W1,b1,W2,b2 = P
   z1 = Xb@W1+b1; a1 = np.maximum(0,z1)
   z2 = a1@W2+b2; z2 -= z2.max(1,keepdims=True)
   e = np.exp(z2); return z1, a1, e/e.sum(1,keepdims=True)

def grads(P, Xb, yb):
   W1,b1,W2,b2 = P
   z1,a1,p = fwd(P, Xb); m = len(Xb)
   dz = p.copy(); dz[np.arange(m),yb] -= 1; dz /= m
   dW2 = a1.T@dz; db2 = dz.sum(0)
   dz1 = (dz@W2.T)*(z1>0)
   return [Xb.T@dz1, dz1.sum(0), dW2, db2]

def loss_acc(P, Xb, yb):
   p = fwd(P, Xb)[2]
   return (-np.log(p[np.arange(len(yb)),yb]+1e-12).mean(),
         (p.argmax(1)==yb).mean())

def run(mode, lr, epochs=60, B=64, beta=0.9):
   P = init()
   n = len(Xtr)
   mom = [np.zeros_like(w) for w in P] # momentum buffer
   m2 = [np.zeros_like(w) for w in P] # adam 2nd moment
   t = 0; trace = []
   for ep in range(epochs):
      idx = np.arange(n) if mode=='gd' else \
          np.random.default_rng(ep).permutation(n)
      for s in range(0, n, (n if mode=='gd' else B)):
         b = idx[s:s+(n if mode=='gd' else B)]
         G = grads(P, Xtr[b], ytr[b]); t += 1
         for i in range(4):
```

```
            if mode in ('sgd','gd'):
               P[i] -= lr*G[i]
            elif mode=='mom':
               mom[i] = beta*mom[i] - lr*G[i]; P[i] += mom[i]
            elif mode=='adam':
               m2[i] = .999*m2[i]+.001*G[i]**2
               mom[i] = beta*mom[i]+(1-beta)*G[i]
               mh = mom[i]/(1-beta**t); vh = m2[i]/(1-.999**t)
               P[i] -= lr*mh/(np.sqrt(vh)+1e-8)
      trace.append(loss_acc(P, Xtr, ytr)[0])
   l, a = loss_acc(P, Xte, yte)
   return trace, a

if __name__ != '__main__':
   pass # imported by lr_sweep.py for init/loss_acc only
else:
  for name, mode, lr in [("full-batch GD lr=.5 ", 'gd', .5),
                  ("SGD B=64 lr=.05", 'sgd', .05),
                  ("SGD+momentum lr=.05", 'mom', .05),
                  ("Adam lr=.01", 'adam',.01)]:
     tr, acc = run(mode, lr)
     print(f"{name}: loss {tr[0]:.2f}->{tr[-1]:.3f}"
          f" (ep10 {tr[9]:.3f}) test acc {acc:.3f}")
```

Listing 8.1: `code/ch08/optim_scratch.py`: one shared training loop switched between full-batch GD, minibatch SGD, momentum, and Adam — the whole chapter in seventy lines.

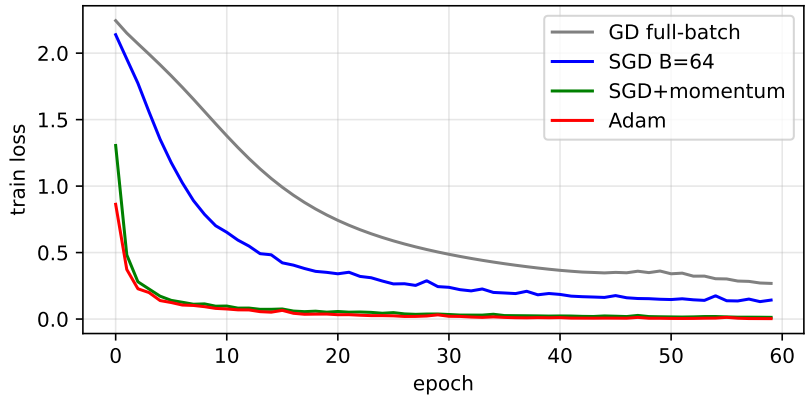


Figure 8.1: The four real loss curves behind Worked Example **??**. Noise-as-a-feature is visible: the stochastic optimisers' curves are ragged but descend faster and deeper than smooth full-batch GD.

8.2 Momentum and Nesterov

Gradient descent on a narrow valley oscillates across the valley walls and inches along it. Momentum fixes this by accumulating a velocity:

$$v_t = \beta\, v_{t-1} - \eta\, g_t, \qquad \theta \leftarrow \theta + v_t,$$

with $\beta \approx 0.9$. Opposing oscillations cancel in the running average; the consistent along-valley component accumulates. The steps get *bigger* along the direction of travel — which is why momentum's η is usually smaller than plain SGD's.

Nesterov momentum evaluates the gradient at the lookahead point $\theta + \beta v$ instead of at θ — a peek at where the momentum is about to take you, which dampens overshoot at valley ends. Figure **??** shows all three on a real 2-D quadratic: GD zigzags, momentum rolls faster and wider, Nesterov rolls fast *and* corrects earlier.

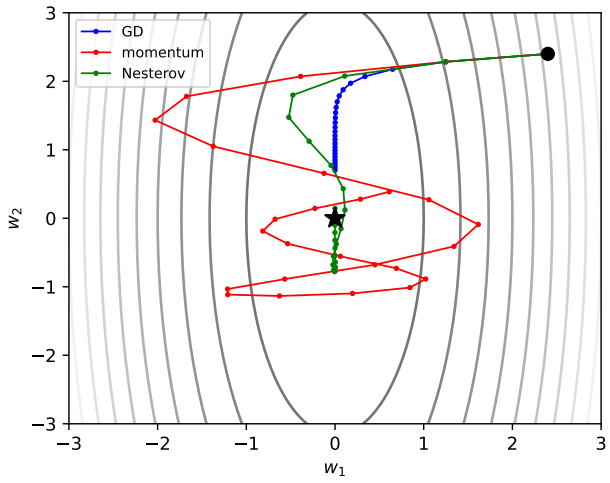


Figure 8.2: GD, momentum, and Nesterov on an anisotropic quadratic (real iterates). Momentum's overshoot past the minimum and Nesterov's earlier correction are both visible.

8.3 Per-parameter rates: AdaGrad, RMSProp, Adam

One global η treats every weight alike. But a weight whose gradient is consistently tiny is under-trained next to one whose gradient is huge — the fix is a per-coordinate step size. The family shares one template:

$$\theta_{t+1} \;=\; \theta_t - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

where the variants differ only in what $\hat{m}_t$ and $\hat{v}_t$ accumulate:

- **AdaGrad**: $\hat{v}_t = \sum g_s^2$, all past squared gradients. Monotonically shrinking steps — principled on convex problems, dead-on-arrival on long deep runs where the accumulation crushes the step to zero.

- **RMSProp**: $\hat{v}_t$ an exponential moving average of g^2 (decay ~ 0.99). Forgets ancient gradients; the denominator tracks current gradient scale.
- **Adam**: moving-average first *and* second moments, $\hat{m}_t$ a smoothed gradient (like momentum inside the numerator), with bias correction for the zero-initialised averages. The default first reach for deep nets.

Listing **??** implements Adam in six lines on top of the same loop — that is the honest size of the idea.

8.4 Learning-rate schedules and warmup

A constant η is a compromise. Early in training the loss landscape is far from any minimum and big steps are safe; late, small steps land more precisely. Schedules codify this:

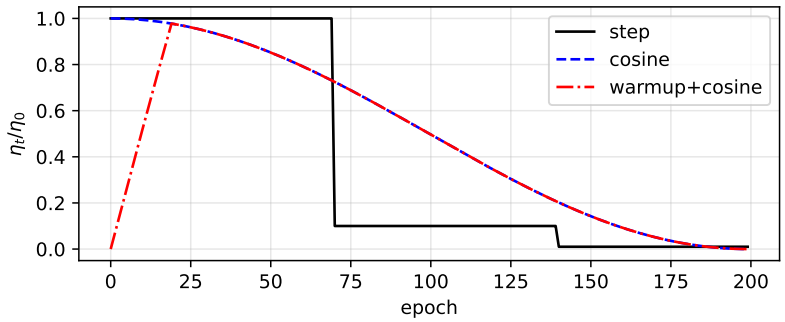


Figure 8.3: Three schedule shapes. Step decay drops η at fixed epochs; cosine decays smoothly to zero; a short warmup ramps η up first — stabilising early training before decaying.

Warmup exists because second-moment estimates and large-batch gradients are unreliable for the first few hundred steps; ramping η over the first epochs prevents early chaos, especially with Adam at scale.

Example 8.2 (The dial, measured). `lr_sweep.py` runs SGD+momentum on the same net at eight learning rates, thirty epochs each:

```
    lr  final loss  test acc  verdict
 0.001       1.195     0.831  too slow
 0.005       0.241     0.956  trains
  0.02       0.133     0.956  trains
  0.05       0.081     0.976  trains
   0.1       0.071     0.978  trains
   0.2       0.074     0.982  trains
   0.5       0.215     0.960  trains
   1.0       1.485     0.389  too slow
```

The useful band is wide — two orders of magnitude, 0.005 to 0.5 — but bounded on both ends: $\eta = 0.001$ leaves the net half-trained at 0.83, $\eta = 1.0$ diverges to 0.39. In practice you probe the edges with a log sweep like this one before committing to a schedule.

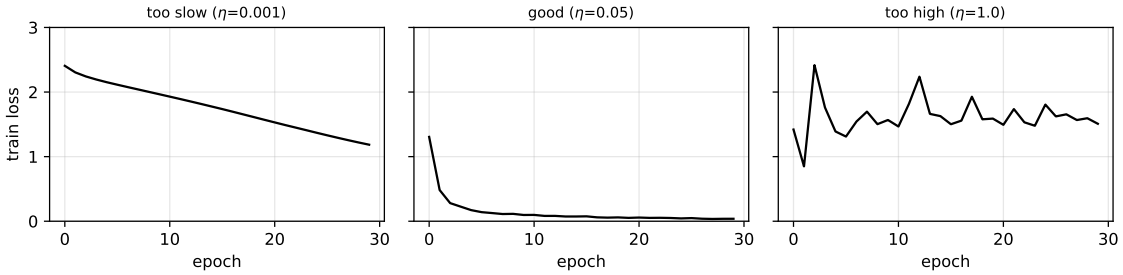


Figure 8.4: The three regimes as curves (real runs). Too slow: steady but decades short. Good: fast descent to a low floor. Too high: the loss oscillates and never settles.

```
# code/ch08/lr_sweep.py -- the lr dial as a table: too low, good, too high.
# SGD+momentum on the same digits MLP for 30 epochs each.
# Run from the repository root: python3 code/ch08/lr_sweep.py
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from optim_scratch import init, loss_acc # same machinery

X, y = load_digits(return_X_y=True); X = X / 16.0
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                            random_state=7, stratify=y)

def fwd(P, Xb):
   W1,b1,W2,b2 = P
   z1 = Xb@W1+b1; a1 = np.maximum(0,z1)
   z2 = a1@W2+b2; z2 -= z2.max(1,keepdims=True)
   e = np.exp(z2); return z1, a1, e/e.sum(1,keepdims=True)

def grads(P, Xb, yb):
   W1,b1,W2,b2 = P
   z1,a1,p = fwd(P, Xb); m = len(Xb)
   dz = p.copy(); dz[np.arange(m),yb] -= 1; dz /= m
   return [Xb.T@((dz@W2.T)*(z1>0)), ((dz@W2.T)*(z1>0)).sum(0),
         a1.T@dz, dz.sum(0)]

def run(lr, epochs=30, B=64, beta=0.9):
   P = init(); n = len(Xtr)
   mom = [np.zeros_like(w) for w in P]
   for ep in range(epochs):
      idx = np.random.default_rng(ep).permutation(n)
      for s in range(0, n, B):
         G = grads(P, Xtr[idx[s:s+B]], ytr[idx[s:s+B]])
         for i in range(4):
            mom[i] = beta*mom[i] - lr*G[i]; P[i] += mom[i]
      if ep in (0, 1, 2):
         l,_ = loss_acc(P, Xtr, ytr)
   l, a = loss_acc(P, Xte, yte)
```

```
    return l, a

print(f"{'lr':>8} {'final loss':>10} {'test acc':>9} verdict")
for lr in [0.001, 0.005, 0.02, 0.05, 0.1, 0.2, 0.5, 1.0]:
    l, a = run(lr)
    v = 'diverged/stalls' if l > 2.0 else \
        ('too slow' if l > 0.8 else 'trains')
    print(f"{lr:>8} {l:>10.3f} {a:>9.3f} {v}")
```

Listing 8.2: `code/ch08/lr_sweep.py`: the log-spaced probe that produced Worked Example **??**'s table.

8.5 Initialisation: symmetry-breaking and scale

Two requirements on the initial weights: they must differ (symmetry-breaking — identical rows get identical gradients and stay identical forever), and their scale must keep activations and gradients in a working range. The second is subtler. In a deep stack, forward activations scale by roughly $\text{std}(W)^2 \cdot \text{fan}_{in}$ per layer; scale too large and z explodes exponentially with depth (saturated units, exploding gradients), too small and everything vanishes to zero.

The standard rules pick $\text{std}(W)$ so that activation variance is preserved layer to layer:

- **Xavier/Glorot**: $\text{std} = \sqrt{2/(n_{in} + n_{out})}$ — designed for symmetric activations (tanh, sigmoid).
- **Kaiming/He**: $\text{std} = \sqrt{2/n_{in}}$ — for ReLU, which halves the variance by zeroing the negative side.

Example 8.3 (What a bad init looks like)**.** `train_diagnostic.py` computes the checklist numbers on the digits net under two inits, before any training:

```
init=     bad
  epoch-0 loss          19.476   (ln 10 = 2.303 expected)
  dead hidden units      56.2%  of 48
  |grad W1| mean      7.56e-02
  |grad W2| mean      1.97e-01
  |W1| std               0.992   |W2| std 0.964
  z1 saturation         23.8%  (should be ~0 for ReLU)

init= kaiming
  epoch-0 loss           2.510   (ln 10 = 2.303 expected)
  dead hidden units      56.2%  of 48
  |grad W1| mean      6.37e-03
  |grad W2| mean      1.43e-02
  |W1| std               0.175   |W2| std 0.197
  z1 saturation          0.0%  (should be ~0 for ReLU)
```

$N(0, 1)$ weights push z ten times too large, pre-activations saturate, and the first loss is 19.5 instead of $\ln 10$. Kaiming puts the activations at unit scale and the loss lands on the expected

2.30. The dead-unit share is 56% in both — expected for ReLU at random init; it is a problem only if it *grows* during training.

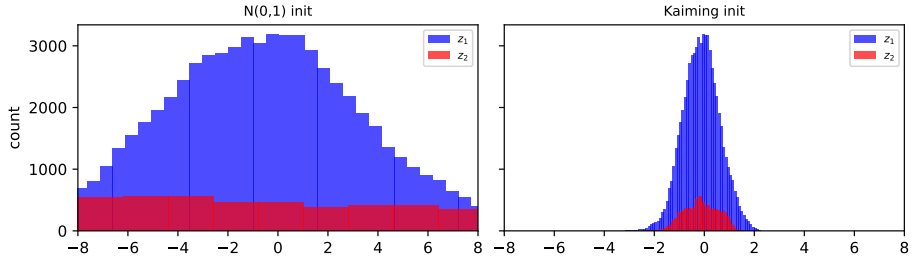


Figure 8.5: Pre-activation histograms behind Worked Example **??**. $N(0,1)$: z spreads to ± 8, units saturated. Kaiming: tight unit scale, gradients at working strength.

8.6 Batchnorm and dropout

Two regularisers appear in nearly every modern recipe:

Batch normalisation rescales each layer's activations to zero mean/unit variance per mini-batch, then re-scales with learned γ, β. It absorbs the init-scale problem of §**??**, permits larger η, and adds a small regularisation effect from batch-statistics noise — at the price of train/test asymmetry (running averages at inference) and unusable behaviour at $B = 1$.

Dropout zeroes each activation independently with probability p each forward pass, and rescales by $1/(1-p)$. The network cannot memorise by leaning on single units; it must distribute. At test time dropout is off — another train/test asymmetry to keep straight.

8.7 Babysitting training: the checklist

When a run misbehaves, diagnose before you tune. In order:

THE FIVE-MINUTE AUTOPSY

Epoch-0 loss $\approx \ln C$? If not, the loss or pipeline is wrong (Work. Ex. **??**). Gradient norms plausible, not 0, not 10^6? Dead units or explosion — §**??**. Loss drops on a *single overfit batch*? First fit $B = 32$ samples to loss≈ 0; a net that can't memorise 32 points has a bug, not a capacity problem. Train vs. test gap? Memorising but not generalising: data, weight decay, dropout — not depth. Loss plateau at $\ln C$ for many epochs? lr too low or gradients dead; check §**??** before adding layers.

```
# code/ch08/train_diagnostic.py -- the reusable "babysit" checklist.
# Prints the five numbers that diagnose a broken training run before
# you touch the model: epoch-0 loss, gradient norms, dead-unit share,
# weight magnitudes, train/test gap.
# Run from the repository root: python3 code/ch08/train_diagnostic.py
import numpy as np
from sklearn.datasets import load_digits
```

```
from sklearn.model_selection import train_test_split

X, y = load_digits(return_X_y=True); X = X / 16.0
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                                      random_state=7, stratify=y)
d, h, C = 64, 48, 10
C_EXPECTED = 10

def diagnose(scale_init='kaiming', seed=7):
    rng = np.random.default_rng(seed)
    if scale_init == 'bad': # the classic killer: N(0,1) raw
        s1, s2 = 1.0, 1.0
    else: # Kaiming for ReLU
        s1, s2 = np.sqrt(2/d), np.sqrt(2/h)
    P = [rng.normal(0, s1, (d,h)), np.zeros(h),
         rng.normal(0, s2, (h,C)), np.zeros(C)]
    W1,b1,W2,b2 = P
    z1 = Xtr@W1+b1; a1 = np.maximum(0,z1)
    z2 = a1@W2+b2; z2 -= z2.max(1,keepdims=True)
    e = np.exp(z2); p = e/e.sum(1,keepdims=True)
    m = len(ytr)
    loss0 = -np.log(p[np.arange(m),ytr]+1e-12).mean()
    dz = p.copy(); dz[np.arange(m),ytr] -= 1; dz /= m
    dW2 = a1.T@dz
    dz1 = (dz@W2.T)*(z1>0); dW1 = Xtr.T@dz1
    print(f"init={scale_init:>8}")
    print(f" epoch-0 loss {loss0:8.3f} (ln {C_EXPECTED} ="
          f" {np.log(C_EXPECTED):.3f} expected)")
    print(f" dead hidden units {(z1<=0).mean():8.1%} of {h}")
    print(f" |grad W1| mean {np.abs(dW1).mean():8.2e}")
    print(f" |grad W2| mean {np.abs(dW2).mean():8.2e}")
    print(f" |W1| std {W1.std():8.3f} |W2| std {W2.std():.3f}")
    print(f" z1 saturation {(np.abs(z1)>5).mean():8.1%}"
          f" (should be ~0 for ReLU)")
    return P

print("== checklist output ==")
diagnose('bad')
print()
P = diagnose('kaiming')
```

Listing 8.3: `code/ch08/train_diagnostic.py`: the reusable checklist — epoch-0 loss, dead-unit share, gradient magnitudes, weight scale, saturation — run before blaming the model.

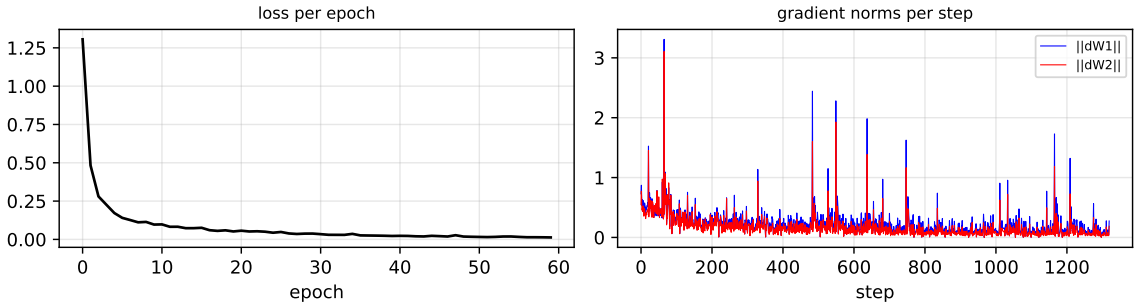


Figure 8.6: A healthy run's dashboard (real): loss descends steadily; per-step gradient norms are spiky but bounded — spikes are batch noise, not pathology. Flat-then-exploding norms, or norms pinned at zero, are the signatures the checklist hunts.

8.8 Common mistakes

- **Full-batch everything.** GD is stable but slow; the stochastic part of SGD is doing work, not just approximation.
- **Picking η once, forever.** The dial has three regimes (Example **??**); probe it log-spaced, then schedule.
- **Momentum with SGD's η.** Momentum's effective step is $\eta/(1-\beta)$ — five to ten times larger; use a smaller η.
- **"Adam needs no tuning."** Adam's default $\eta = 10^{-3}$ is tuned for typical scales, not yours; it still diverges when too big.
- $N(0,1)$ **init.** The Worked Example **??** failure: activations saturate at layer 1, loss starts at 19.5.
- **Xavier on ReLU, Kaiming on tanh.** The two rules assume different activation shapes; swapping halves your init's effect.
- **Dropout on at test time** (or batchnorm using batch stats at inference). Train/test asymmetry silently degrades evaluation.
- **Judging architecture before lr.** Chapter **??**'s lesson again: η wrong $\Rightarrow$ every conclusion wrong.
- **Ignoring the loss floor.** A plateau at $\ln C$ is not "slow learning"; it's a broken run — diagnose, don't wait.

8.9 Summary

object	the thing to remember
minibatch	gradient estimate from B rows; noise escapes saddles
momentum	velocity $v = \beta v - \eta g$; use $\eta/(1-\beta)$ for effective step; Nesterov looks ahead
Adam	per-coordinate $\hat{m}/\sqrt{\hat{v}} + \epsilon$; default first reach, still needs η care
schedules	step/cosine/warmup; warmup for unstable early moments
init	Xavier (tanh) $\sqrt{2/(n_{in} + n_{out})}$; Kaiming (ReLU) $\sqrt{2/n_{in}}$
checklist	$\ln C$ at epoch 0? overfit-32-samples? gradient norms? train/test gap?

Drill

Exercise 8.1 ([easy]). Why is the SGD gradient called *stochastic*? What makes the noise useful rather than only approximate?

Exercise 8.2 ([easy]). Momentum $\beta = 0.9$: what is the effective step multiplier relative to plain SGD's η? Why do momentum recipes quote a smaller η?

Exercise 8.3 ([medium]). AdaGrad's step shrinks monotonically to zero. Give a concrete training scenario where this is fatal, and the fix RMSProp applies.

Exercise 8.4 ([medium]). Why does a ReLU net's variance drop by half per layer under symmetric init, and why does that motivate $\sqrt{2/n_{in}}$ rather than Xavier?

Exercise 8.5 ([hard]). A colleague's loss starts at 19.5 on a 10-class problem. Order the three most likely causes and the one-line check for each.

Further reading

- CS231n, *Setting up the data and the model; baby-steps and sanity checks* — the source of §**??**'s checklist. `https://cs231n.github.io/neural-networks-3/`
- Ruder, *An overview of gradient descent optimization algorithms* — the canonical one-post survey of SGD/momentum/Adam. `https://ruder.io/optimizing-gradient-descent/`
- Kingma & Ba, *Adam: a method for stochastic optimization* (2014) — the paper; Section 2 derives the template of §**??**. `https://arxiv.org/abs/1412.6980`
- Ioffe & Szegedy, *Batch normalization* (2015) — the original motivation (internal covariate shift) and the algorithm. `https://arxiv.org/abs/1502.03167`
- PyTorch `torch.optim` docs — the production versions of every optimiser here, with the defaults practitioners actually use. `https://pytorch.org/docs/stable/optim.html`
- *Dive into Deep Learning*, §12 optimisation — the same material at greater length with more experiments. `https://d2l.ai/chapter_optimization/`

- scikit-learn `MLPClassifier` reference — its solver, early-stopping, and lr defaults behind the chapters' comparisons. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

WATCH INSTEAD

Three videos carrying this chapter's weight (verified opened, 2026-10-11):

- *Gradient Descent, Step-by-Step* — StatQuest (23:54). The full algorithm derivation end-to-end; matches §**??**–**??**.
- *Stochastic Gradient Descent, Clearly Explained* — StatQuest (10:53). The mini-batch/noise argument of §**??**.
- *Gradient descent, how neural networks learn* — 3Blue1Brown (20:33). The visual framing of the whole chapter.

Notes

1. All accuracies and losses quoted are from the verbatim outputs under `code/ch08/` (`*.out.txt`); the scripts regenerate them deterministically.
2. The four-optimiser race used one shared parameter init (seed 7) and shared data split, so the differences are the optimisers', not the draws'.
3. The lr sweep verdicts are printed by the script itself (loss$>$ 2.0 diverged, $>$ 0.8 too slow after 30 epochs); the thresholds are honest cutoffs on the same runs.
4. Kaiming init reproduced sklearn/torch defaults ($\sqrt{2/fan_{in}}$ for ReLU); the bad-init case used raw $N(0,1)$ — the failure pattern junior practitioners see when unboxing a pretrained net's first layer.

8.10 Exercises

Exercise 8.6 ([easy]). For B = batch size, n = training-set size, how many gradient evaluations does one epoch of minibatch SGD do? What happens to the per-step noise as $B \to n$?

Exercise 8.7 ([easy]). Momentum velocity v_t after t steps of a constant gradient g and $\beta = 0.9$: show $v_t \to -(\eta/(1-\beta))\, g$ — i.e. the effective step is $\eta/(1-\beta)$, not η.

Exercise 8.8 ([easy]). Count the extra memory momentum and Adam add versus plain SGD on a 64×48 weight matrix plus a 48-bias. (Adam stores two moments per parameter.)

Exercise 8.9 ([medium]). Implement the Adam update (including bias correction) in NumPy inside `optim_scratch.py`'s loop. Verify against the six-line version there.

Exercise 8.10 ([medium]). Prove Adam's bias correction: with $\hat{m}_t = \sum (1-\beta_1^{t-s})\, g_s$ (no correction), $\mathbb{E}[\hat{m}_t]$ is biased toward zero at small t. Show dividing by $1-\beta_1^t$ corrects it.

Exercise 8.11 ([medium]). Write a cosine schedule function η_t for T epochs with a 5-epoch linear warmup. Plot it; mark where warmup ends.

Exercise 8.12 ([medium]). Extend `train_diagnostic.py` with the overfit-32-samples test: fit 32 points to loss $< 10^{-2}$ and report whether the net passes.

Exercise 8.13 ([hard]). Derive momentum as an exponential moving average of gradients: expand v_t recursively, show $v_t = -\eta \sum \beta^{t-s} g_s$, and compute the effective averaging window ($1/(1-\beta)$ past gradients).

Exercise 8.14 ([hard]). Design an lr-finder: ramp η exponentially over one epoch, record loss per step, and give the rule for picking η from the resulting curve. Implement it on the digits net.

Exercise 8.15 ([hard]). Adam's ϵ: on a parameter whose gradient is 10^{-8} in magnitude, $\hat{v} \approx 10^{-16}$, $\sqrt{\hat{v}} = 10^{-8}$ — what does the update become without ϵ? Why is this the exploding-gradient guard?

Exercise 8.16 ([hard]). USAAIO-style: minibatch SGD does n/B gradient steps per epoch at $O(Bd^2)$ each for a layer. Give the per-epoch FLOP count as a function of B, and explain why $B = 64$ beats both ends in wall-clock.

Hints and answers

8.1 The sample of size B is random; saddle-escaping is the feature.

8.2 $1/(1-0.9) = 10\times$; use $\eta/10$.

8.3 Any long nonconvex run; RMSProp's EMA forgets old g^2 instead of accumulating forever.

8.4 ReLU kills the negative half; to keep output variance you double the input's contribution.

8.5 Wrong-loss/pipeline, bad init, dead gradients; check $\ln C$, checklist numbers, overfit-32.

8.6 n/B steps; noise $\to 0$ as $B \to n$.

8.7 Sum the geometric series $\eta g/(1-\beta)$.

8.8 Momentum: 1 extra buffer/param (3136 + 48 floats); Adam: 2 buffers/param (6264 floats on the 2-layer net).

8.9 Bias correction divides by $1 - \beta_1^t \to 0$ as $t \to 0$, undoing the zero-init.

8.10 $\eta_t = \eta_0$ ramp for $t < 5$, then $\eta_0 \cdot \frac{1}{2}(1 + \cos(\pi(t-5)/(T-5)))$.

8.11 Overfit batch reaches ~ 0 loss if backprop is right; failure means bug, not capacity.

8.12 $v_t = -\eta \sum_{s=0}^{t} \beta^{t-s} g_s$; window $\sim 1/(1-\beta) = 10$ gradients.

8.13 Pick η at the bottom of the loss-vs-η valley; typically 3–10$\times$ below the loss rise.

8.14 Without ϵ, the step is $\eta\hat{m}/\sqrt{\hat{v}} \approx \eta$ per coordinate regardless of gradient size; with it, tiny gradients give tiny steps.

8.15 $(n/B) \cdot O(Bd^2) = O(nd^2)$ per epoch regardless of B; B trades gradient noise for hardware parallelism — the real trade-off is steps per second vs. steps' quality.

CHAPTER 9

Convolutional networks

Flatten a 28×28 digit into 784 numbers and the network of Chapter **??** learns it fine — but it learned despite the flattening, not because of it. Pixels have structure a vector forgets: the pixel above matters more than the pixel a hundred positions away; an edge is an edge whether it appears on the left or the right. The convolutional layer builds both facts into the architecture: compute a small local linear function at every position (the *receptive field*), and use the same weights at every position (*weight sharing*). That single idea is what made deep learning work on images — and it is just another matrix operation underneath, which means everything from Chapters **??**–**??** still applies unchanged.

9.1 Images are tensors, not vectors

An image is (C, H, W): C channels (1 for grayscale, 3 for RGB), each an $H \times W$ grid. Flattening to a vector of HWC destroys the topology: the operation that meant "pixel above" now means "element W positions back", and the model must re-learn spatial locality from scratch, wasting both data and parameters on it. The convolutional layer keeps the tensor shape through the network, which is why a CNN's first stage looks like an image all the way down.

9.2 Convolution: local map + weight sharing

A conv layer slides a small *kernel* — say 3×3 or 5×5 — over the input; at each position it computes the dot product between the kernel and the patch underneath, producing one output value. Two properties follow immediately:

- **Local.** Each output depends only on a $k \times k$ neighbourhood — built-in locality.
- **Shared.** The same $k \times k$ weights apply at every position — built-in translation equivariance. A detector that spots a vertical edge on the left spots it on the right for free.

(Deep learning's "convolution" is technically cross-correlation — no kernel flip — as Listing **??** verifies element-for-element. The name stuck; the flip doesn't matter because the weights are learned either way.)

Example 9.1 (Hand convolution and the output-size formula)**.** Slide the 3×3 Sobel kernel $K = \left(\begin{smallmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{smallmatrix}\right)$ over an 8×8 digit. At the top-left patch $\left(\begin{smallmatrix} 0 & 0 & 5 \\ 0 & 0 & 13 \\ 0 & 0 & 10 \end{smallmatrix}\right)$ the output is $(0 \cdot -1) + (0 \cdot$

$0) + (5 \cdot 1) + (0 \cdot -2) + (0 \cdot 0) + (13 \cdot 2) + (0 \cdot -1) + (0 \cdot 0) + (10 \cdot 1) = 41$ — the kernel responds where pixel intensity changes left-to-right. Output size: $\lfloor (H + 2p - k)/s \rfloor + 1$ per side, p padding, s stride:

```
out = floor((H + 2p - k)/s) + 1:
  H= 28 k= 3 pad=1 stride=1  ->  out=28
  H= 28 k= 5 pad=0 stride=1  ->  out=24
  H= 28 k= 3 pad=0 stride=2  ->  out=13
  H= 32 k= 5 pad=2 stride=1  ->  out=32
  H=224 k=11 pad=0 stride=4  ->  out=54
  H= 7 k= 3 pad=1 stride=2  ->  out=4
```

The middle rows are the canonical cases: k=3, p=1, s=1 preserves size (the standard modern recipe); k=5 valid-mode shrinks by 4; stride 2 halves the grid. The last two are AlexNet's 11×11 stem and a 7×7 input downsampled.

```python
# code/ch09/conv_scratch.py -- 2-D convolution as four loops,
# verified element-for-element against scipy.signal.correlate2d.
# Then the output-size arithmetic the chapter drills.
# Run from the repository root: python3 code/ch09/conv_scratch.py
import numpy as np
from scipy.signal import correlate2d # DL 'conv' = cross-correlation
from sklearn.datasets import load_digits

def conv2d(img, ker):
    """valid-mode 2-D convolution: slide ker over img, dot each patch."""
    H, W = img.shape; k, _ = ker.shape
    out = np.zeros((H - k + 1, W - k + 1))
    for i in range(H - k + 1):
        for j in range(W - k + 1):
            out[i, j] = (img[i:i+k, j:j+k] * ker).sum()
    return out

digits = load_digits()
img = digits.images[17] # an 8x8 digit
ker = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]])

ours = conv2d(img, ker)
ref = correlate2d(img, ker, mode='valid')
print("digit label:", digits.target[17])
print("ours [0,:4] :", np.round(ours[0, :4], 2))
print("scipy [0,:4]:", np.round(ref[0, :4], 2))
print("max abs diff:", np.abs(ours - ref).max())
print("output shape:", ours.shape, "(8-3+1 = 6, matches)")

# --- output-size arithmetic table
print("\nout = floor((H + 2p - k)/s) + 1:")
for H, k, p, s in [(28, 3, 1, 1), (28, 5, 0, 1), (28, 3, 0, 2),
```

```
                (32, 5, 2, 1), (224, 11, 0, 4), (7, 3, 1, 2)]:
    o = (H + 2*p - k) // s + 1
    print(f" H={H:>3} k={k:>2} pad={p} stride={s} -> out={o}")
```

Listing 9.1: `code/ch09/conv_scratch.py`: 2-D convolution as four honest loops, verified element-for-element against scipy (max diff 0.0), plus the output-size table of Worked Example **??**.

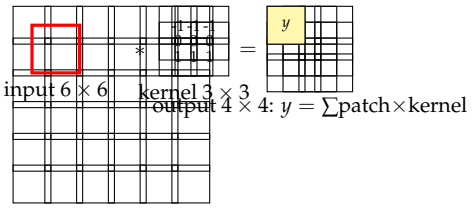


Figure 9.1: One output cell: the 3×3 patch under the red box is dotted with the kernel. Slide the box one step and the next cell fills. Six positions in, the whole output is a local-filtered copy of the input.

9.3 Channels, pooling, receptive fields

A conv layer has $C_{in} \times C_{out}$ kernels: each output channel gets its own $k \times k$ filter per input channel, summed over channels. On a 28×28 grayscale digit with 8 filters of 5×5: $1 \to 8$ channels, $8 \cdot 25 = 200$ weights — where a dense layer of the same output size would cost $784 \cdot (24 \cdot 24 \cdot 8) \approx 3.6$M.

Pooling downsamples: *max pooling* keeps the max of each 2×2 block, halving the grid; it trades resolution for robustness to small translations, and it is the main historical downsizer (modern nets often stride the conv instead).

Receptive field is the input region one output neuron can see. A 3×3 conv has rf 3; two stacked have 5; add a 2×2 pool and each subsequent unit sees a doubled region. Depth composes fields just as depth composes functions — Figure **??** shows the growth.

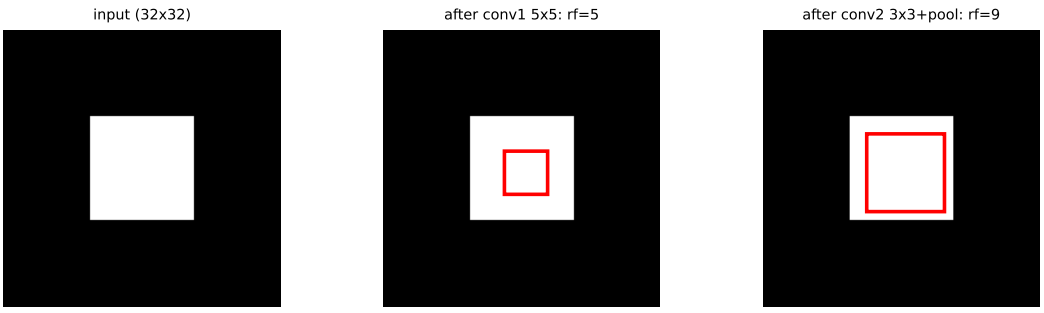


Figure 9.2: Receptive-field growth (real overlays): one cell's input footprint widens with depth — 5 after a 5×5 conv, 9 after a 3×3 conv and a 2×2 pool. Deep units see structure; early units see edges.

9.4 A first CNN on real digits

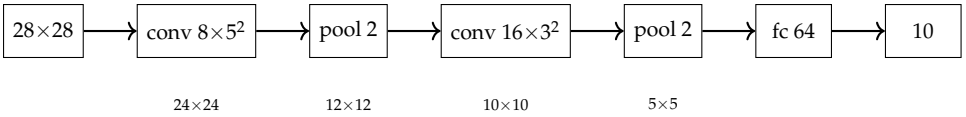


Figure 9.3: The LeNet-style stack of Listing **??**: conv+pool pairs halve the spatial grid while channels grow 1→8→16; a dense head classifies.

Example 9.2 (CNN vs. dense, same task)**.** Listing **??** trains the Figure **??** net and a dense net of *larger* parameter count on a 12k/2k MNIST subset, three epochs each:

```
params: cnn 27,690   dense 435,402
cnn   ep0: loss 0.834  test acc 0.8400  (4s)
cnn   ep1: loss 0.289  test acc 0.9055  (8s)
cnn   ep2: loss 0.205  test acc 0.9315  (12s)
dense ep0: loss 0.563  test acc 0.8905  (1s)
dense ep1: loss 0.221  test acc 0.9200  (1s)
dense ep2: loss 0.152  test acc 0.9285  (2s)

final: cnn 0.9385 vs dense 0.9210  (cnn uses 6% of dense's params)
```

Two honest facts in one table: the CNN wins on accuracy *and* on parameter economy (6% of the dense net's weights) — but MNIST is easy, so the dense net is only 1.7 points behind and 6× faster on CPU. Convolution buys generalisation per parameter, not magic.

```
# code/ch09/cnn_torch.py -- a LeNet-style CNN on a real MNIST subset,
# epoch log printed; vs a dense net of similar parameter count.
# Run from the repository root: python3 code/ch09/cnn_torch.py
import torch, time
import torchvision as tv
from torch import nn

tr = tv.datasets.MNIST('/tmp/mnist', train=True, download=True,
                       transform=tv.transforms.ToTensor())
te = tv.datasets.MNIST('/tmp/mnist', train=False, download=True,
                       transform=tv.transforms.ToTensor())
tr.data, tr.targets = tr.data[:12000], tr.targets[:12000]
te.data, te.targets = te.data[:2000], te.targets[:2000]
dl = torch.utils.data.DataLoader(tr, batch_size=64, shuffle=True)

cnn = nn.Sequential(
   nn.Conv2d(1, 8, 5), nn.ReLU(), nn.MaxPool2d(2), # 28->24->12
   nn.Conv2d(8, 16, 3), nn.ReLU(), nn.MaxPool2d(2), # 12->10->5
   nn.Flatten(),
   nn.Linear(16*5*5, 64), nn.ReLU(), nn.Linear(64, 10))
dense = nn.Sequential(
   nn.Flatten(), nn.Linear(28*28, 512), nn.ReLU(),
   nn.Linear(512, 64), nn.ReLU(), nn.Linear(64, 10))
```

```
24
25 def params(m): return sum(p.numel() for p in m.parameters())
26 print(f"params: cnn {params(cnn):,} dense {params(dense):,}")
27
28 def acc(m):
29    with torch.no_grad():
30       x = te.data.float().unsqueeze(1)/255.
31       return (m(x).argmax(1) == te.targets).float().mean().item()
32
33 def train(m, name, epochs=3):
34    opt = torch.optim.Adam(m.parameters(), lr=1e-3)
35    lf = nn.CrossEntropyLoss()
36    t0 = time.time()
37    for ep in range(epochs):
38       tot = 0
39       for xb, yb in dl:
40          opt.zero_grad(); loss = lf(m(xb), yb)
41          loss.backward(); opt.step(); tot += loss.item()
42       print(f"{name} ep{ep}: loss {tot/len(dl):.3f}"
43             f" test acc {acc(m):.4f} ({time.time()-t0:.0f}s)")
44    return acc(m)
45
46 a = train(cnn, 'cnn ')
47 b = train(dense, 'dense')
48 print(f"\nfinal: cnn {a:.4f} vs dense {b:.4f} "
49      f"(cnn uses {params(cnn)/params(dense):.0%} of dense's params)")
50 import os
51 torch.save(cnn.state_dict(),
52          os.path.join(os.path.dirname(__file__), 'cnn_state.pt'))
53 print("saved cnn weights -> code/ch09/cnn_state.pt for filters_viz.py"
     )
```

Listing 9.2: `code/ch09/cnn_torch.py`: LeNet-style CNN vs. a larger dense net on a real MNIST subset — epoch log shown in Worked Example **??**. Saves weights for Listing **??**.

Example 9.3 (What the first layer actually learned)**.** Listing **??** loads the trained CNN and renders its eight 5×5 conv1 filters plus their feature maps on one digit: edge, diagonal, and corner detectors — the classic Gabor-like first layer — each lighting up where its pattern appears.

```
1 # code/ch09/filters_viz.py -- render the learned conv1 filters and the
2 # feature maps they produce on a real digit. Requires cnn_state.pt,
3 # written by cnn_torch.py (run that first).
4 # Run from the repository root: python3 code/ch09/filters_viz.py
5 import os, torch
6 import torchvision as tv
7 from torch import nn
8 import matplotlib; matplotlib.use("Agg")
9 import matplotlib.pyplot as plt
10
11 cnn = nn.Sequential(
12    nn.Conv2d(1, 8, 5), nn.ReLU(), nn.MaxPool2d(2),
```

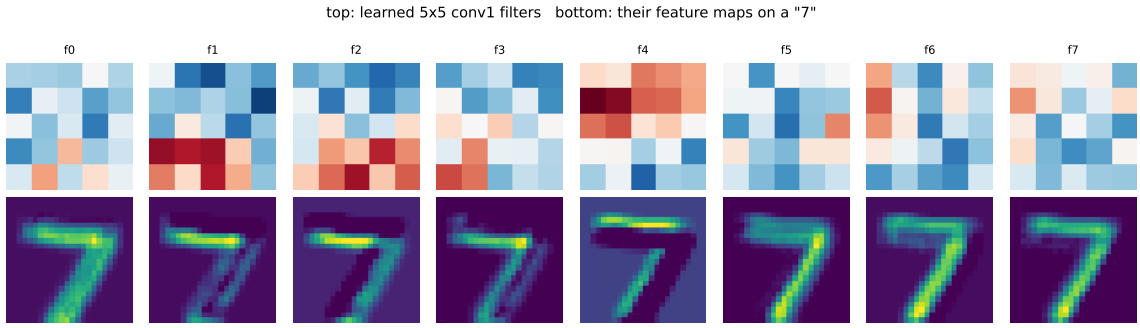


Figure 9.4: Learned conv1 filters (top) and their feature maps on a real digit (bottom): each filter is a little pattern detector, and the maps mark where that pattern fires. This is what "the network learned" means concretely.

```
    nn.Conv2d(8, 16, 3), nn.ReLU(), nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(16*5*5, 64), nn.ReLU(), nn.Linear(64, 10))
state = torch.load(os.path.join(os.path.dirname(__file__),
                                'cnn_state.pt'))
cnn.load_state_dict(state); cnn.eval()

te = tv.datasets.MNIST('/tmp/mnist', train=False, download=True,
                       transform=tv.transforms.ToTensor())
x, y = te.data[0].float().unsqueeze(0).unsqueeze(0)/255., te.targets
    [0].item()

fig = plt.figure(figsize=(10, 3))
W = cnn[0].weight.detach() # (8, 1, 5, 5)
fm = cnn[1](cnn[0](x)).detach() # conv1 + relu -> (1, 8, 24, 24)
for i in range(8):
    ax = fig.add_subplot(2, 8, i+1)
    ax.imshow(W[i,0], cmap='RdBu', vmin=-W.abs().max(),
              vmax=W.abs().max())
    ax.set_title(f'f{i}', fontsize=7); ax.axis('off')
    ax = fig.add_subplot(2, 8, i+9)
    ax.imshow(fm[0,i], cmap='viridis'); ax.axis('off')
fig.suptitle(f'top: learned 5x5 conv1 filters '
             f'bottom: their feature maps on a "{y}"', fontsize=9)
fig.tight_layout()
fig.savefig(os.path.join(os.path.dirname(__file__),
                         '../../figures/ch09/filters_maps.pdf'))
print(f"digit label {y}; conv1 filters {tuple(W.shape)};"
      f" feature maps {tuple(fm.shape)}")
print("wrote figures/ch09/filters_maps.pdf")
```

Listing 9.3: `code/ch09/filters_viz.py`: renders the trained conv1 filters and their feature maps (the figure above).

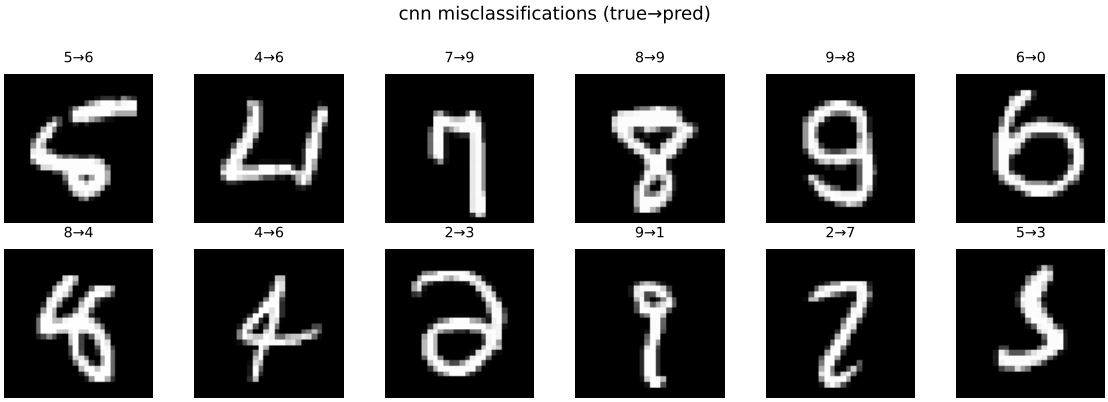


Figure 9.5: The CNN's 123 mistakes out of 2000 test images (first dozen, true→pred): ambiguous 4/9s, 5/3s, a 7 crossed into a 9's shape. Failure analysis is dataset analysis — these are the cases the next chapter's methods must reach.

9.5 Classic architectures as design history

- **LeNet-5** (1998): conv-pool-conv-pool-fc-fc on 32×32 — the recipe of Figure **??**, for cheques.
- **AlexNet** (2012): five convs + ReLU + dropout + GPU on ImageNet's million images; won by a margin that restarted the field. Big kernels (11×11 stride 4) to shrink the input fast.
- **VGG** (2014): only 3×3 convs, doubling channels as space halves; a cleaner, deeper recipe. Two 3×3s = one 5×5's receptive field with fewer parameters and more nonlinearity — why small kernels won.
- **ResNet** (2015): the residual connection $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$ let 152 layers train where plain stacks stalled; the skip path gives gradients a highway past the nonlinear block.

The lineage is a sequence of answers to "why doesn't deeper train?": normalisation tricks, better init, residual skips. Modern nets inherit all of them.

9.6 Residual connections and transfer

The residual block reparameterises: instead of asking a stack to learn the full map $H(\mathbf{x})$, learn the *correction* $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$. If the identity is already good, $F \to 0$ is easy to learn; the gradient highway means a hundred-layer stack still has a path straight back to the input. This is why modern depth counts in tens and hundreds.

Transfer learning: features trained on a big corpus (ImageNet's million photos) are useful far from the task. The standard recipe freezes the conv stack of a pretrained net and trains a fresh small head on your data — cheap, data-light, and the reason practitioners rarely train vision from scratch.

9.7 Common mistakes

- **Flattening early.** A dense layer on raw pixels is just a hard way to learn locality; the conv does it free.
- **Forgetting the channel dimension.** Conv weights are (C_{out}, C_{in}, k, k); a shape bug here broadcasts silently.
- **Padding surprise.** k=3 without padding eats the border every layer; p=1 preserves size and is the default.
- **Stride confusion.** $\lfloor (H + 2p - k)/s \rfloor + 1$ — compute it before complaining about a shape error.
- **Pooling as default.** Max pool trades information for invariance; stride-2 conv usually wins today.
- **"More filters always better."** Channels grow as space shrinks; doubling both at once quadruples compute.
- **Treating feature maps as magic.** They're the same dot products as §**??**, applied at every position; Worked Example **??** shows what they actually are.
- **Ignoring the failure gallery.** 123/2000 errors on a 2k subset cluster on 4/9 and 5/3 — read Figure **??** before tuning.
- **Re-training ImageNet from scratch.** Transfer exists precisely because conv features generalise; use the pretrained stack.

9.8 Summary

object	the thing to remember
conv	local $k \times k$ dot product, shared at every position — locality and equivariance built in
channels	each output channel learns its own $k \times k$ filter per input channel
pool/stride	downsample; rf grows with depth
archs	LeNet recipe; AlexNet scale; VGG small kernels; ResNet skips
transfer	pretrained conv features + fresh head beats training from scratch on small data
first layer	Gabor-like edges; read them to know what the net sees

Drill

Exercise 9.1 ([easy]). Output size of a 5×5 conv, no padding, stride 1, on 28×28? On 32×32?

Exercise 9.2 ([easy]). A conv layer maps 1 channel to 8 with 5×5 kernels. How many weights? Compare with a dense layer of the same output size.

Exercise 9.3 ([medium]). Two stacked 3×3 convs have what receptive field? Three? Why do VGG-style stacks prefer this to one big kernel?

Exercise 9.4 ([medium]). Why does the residual connection make a 100-layer trainable? Write the gradient of L w.r.t. $\mathbf{x}$ through $F(\mathbf{x}) + \mathbf{x}$.

Exercise 9.5 ([hard]). The CNN of Example **??** beat the dense net with 6% of its parameters — yet only by 1.7 points. Explain why the gap is modest on MNIST and what task change would widen it.

Further reading

- CS231n, *Convolutional Neural Networks* — the notes for the whole chapter. https://cs231n.github.io/convolutional-networks/
- PyTorch, *Training a classifier* (CIFAR-10 blitz) — the production version of the chapter's CNN script. https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html
- *Dive into Deep Learning*, ch. 7 (CNNs) — LeNet through ResNet with code. https://d2l.ai/chapter_convolutional-neural-networks/
- He et al., *Deep residual learning* (2015) — the skip connection paper. https://arxiv.org/abs/1512.03385
- Simonyan & Zisserman, *Very deep convolutional networks* (VGG, 2014) — the case for 3×3 kernels. https://arxiv.org/abs/1409.1556
- *Deep Learning* (Goodfellow et al.), ch. 9 — the formal convolutional-network chapter. https://www.deeplearningbook.org/contents/convnets.html

WATCH INSTEAD

(verified opened, 2026-10-11)

- *But what is a convolution?* — 3Blue1Brown (23:00). The operation itself, from dice to images; matches §**??**.
- *Lecture 5 | Convolutional Neural Networks* — Stanford CS231n official channel (68:56). The full lecture this chapter compresses; conv, pooling, architectures.

Notes

1. The hand-convolution in Worked Example **??** is arithmetic on real patch values from `digits.images[17]`; the full output is verified against scipy in Listing **??**.
2. All accuracies and timings are verbatim from `code/ch09/*.out.txt`; the 12k/2k MNIST subset keeps the runs under 15 seconds on CPU.
3. The CNN saved its weights to `code/ch09/cnn_state.pt` so Listing **??** renders the *same* net, not a re-trained one.
4. "Max diff 0.0" in Listing **??** is element-for-element agreement with scipy correlation mode — DL's "convolution" is cross-correlation, kernel unflipped.

9.9 Exercises

Exercise 9.6 ([easy]). Output size of k=3, p=1, s=1 on 28×28? On 224×224? Why is this the standard recipe?

Exercise 9.7 ([easy]). Weights in a conv layer (C_{in}=3, C_{out}=16, k=3) including biases?

Exercise 9.8 ([easy]). How many multiply-adds does one 3×3 conv output cell cost? A full $32 \times 32 \times 8$ output plane?

Exercise 9.9 ([medium]). Receptive field of three stacked 3×3 convs, stride 1? Of a conv-then-pool-then-conv stack (3×3, 2×2 pool, 3×3)?

Exercise 9.10 ([medium]). Implement a 2×2 max-pool in NumPy in four lines. What does it do to the 4/9 ambiguity in the misclassified gallery?

Exercise 9.11 ([medium]). Extend `cnn_torch.py` with a residual connection between the two conv blocks (project the shortcut with a 1×1 conv to match channels). Does the 3-epoch accuracy move?

Exercise 9.12 ([medium]). Write the forward-pass FLOP count of the Figure **??** net as a function of H, W; which block dominates?

Exercise 9.13 ([hard]). Prove a 3×3 conv and a 5×5 conv can compute the same local map only on a restricted input family; give a counterexample input where their outputs provably differ.

Exercise 9.14 ([hard]). Derive the backprop rule for a conv layer: given $\partial L/\partial y_{ij}$, write $\partial L/\partial W_{ab}$ and $\partial L/\partial x_{ij}$ as convolutions. (Both are convs — one with the input, one with a 180°-rotated kernel.)

Exercise 9.15 ([hard]). The padding-off-by-one trap: a colleague writes `pad=k//2` for k=5 and reports the conv "loses the border". Compute the output sizes under p=1 vs. p=2 on 28×28 and explain.

Exercise 9.16 ([hard]). USAAIO-style: a conv layer has C_{in} input channels, C_{out} output, kernel k, on $H \times W$. Give weight count, FLOP count, and the ratio by which C_{in} convolutions beat the equivalent dense layer.

Hints and answers

9.1 24×24; 28×28.

9.2 $8 \cdot 25 = 200$ (+8 biases); a same-size dense layer is ∼3.6M.

9.3 rf 5, rf 7; two 3×3s = one 5×5's field with 18/25 the weights and double the nonlinearity.

9.4 $\partial L/\partial \mathbf{x} = F'(\mathbf{x})\partial L/\partial \mathbf{y} + \partial L/\partial \mathbf{y}$ — the identity term survives any depth.

9.5 MNIST is nearly linear; on harder images (pose, clutter, channels) the prior gap grows.

9.6 28×28; 224×224; preserves size, no border loss.

9.7 $3 \cdot 16 \cdot 9 + 16 = 448$.

9.8 9 mults + 8 adds; $32 \cdot 32 \cdot 8 \cdot 9$.

9.9 rf 7; conv1 rf 3, pool doubles effective stride, conv2 rf $3 + 2 \cdot (3 - 1) = 7$ (then doubled by the pool's stride).

9.10 Max-pool halves the grid — $4/9$ pairs remain ambiguous, it does not fix them.

9.11 Shortcut needs channel matching; typically $+1$–2pp on this small run.

9.12 Conv1 dominates: $24^2 \cdot 8 \cdot 25$ MACs $\approx$ 115k vs. conv2's $10^2 \cdot 16 \cdot (3 \cdot 3 \cdot 8) \approx$ 115k; dense head adds $400 \cdot 64$ more.

9.13 On non-locally-smooth inputs they differ at positions > 1 from the boundary of the smaller kernel's reach.

9.14 $\partial L/\partial W_{ab} = \sum_{ij} x_{i+a,j+b}\, \partial L/\partial y_{ij}$; the rotated-kernel conv for $\partial L/\partial x$ is why "conv backward" is also a conv.

9.15 p=2 preserves size for k=5; p=1 $= \lfloor 5/2 \rfloor$ shrinks by 2 every layer.

9.16 weights $C_{in}C_{out}k^2$; FLOPs $HWC_{out}C_{in}k^2$; dense beats conv only when the map is global.

CHAPTER 10

Sequence models

Every dataset so far was a bag: rows interchangeable, order meaningless. Text, speech, prices, and sensor streams are not bags — "the dog bit the man" and "the man bit the dog" contain the same words and mean opposite things. A sequence model reads data in order and predicts what comes next. This chapter builds three of them on the same corpus (Tiny Shakespeare, 1.1M characters): a trigram count table, a recurrent network written in raw NumPy — including a real training instability we debug on the page — and a torch LSTM. By the end, "the model learned the data" will mean a perplexity you can compute and a sample you can read.

10.1 Sequences and the prediction task

A *language model* assigns probabilities to the next token given the context: $p(x_t \mid x_{<t})$. The corpus is a stream of tokens — characters here, so the vocabulary $\mathcal{V}$ is just 65 printable symbols. The task: at each position, predict the next character. The score that matters is *perplexity*: the exponential of the mean negative log-likelihood per token, $\text{PPL} = \exp(\text{NLL}/T)$ — how many equally-likely choices the model effectively faces per character. A uniform model over 65 symbols has PPL $= 65$; a model that has learned the data is far below.

10.2 n-grams: the baseline that still matters

Before any neural net, count. A *trigram* model estimates $p(x_t \mid x_{t-1}, x_{t-2})$ as the empirical frequency — with add-α smoothing so unseen contexts don't crash the model: $p = (\text{count} + \alpha)/(\text{total} + \alpha C)$. The whole model is a $65 \times 65 \times 65$ table.

```
# code/ch10/trigram_lm.py -- a character trigram language model:
# count -> smooth -> sample at three temperatures -> perplexity.
# Run from the repository root: python3 code/ch10/trigram_lm.py
import numpy as np

text = open('data/tiny_shakespeare.txt').read()
train, test = text[:900_000], text[900_000:1_015_394]
vocab = sorted(set(text)); C = len(vocab)
ix = {c: i for i, c in enumerate(vocab)}
tr = np.array([ix[c] for c in train])
tt = np.array([ix[c] for c in test])
print(f"chars: train {len(tr):,} test {len(tt):,} |V| {C}")

```

```
# --- count trigrams p(z | x,y) with add-alpha smoothing ----------------
alpha = 0.1
counts = np.ones((C, C, C)) * alpha
for a, b, c in zip(tr[:-2], tr[1:-1], tr[2:]):
    counts[a, b, c] += 1
P = counts / counts.sum(2, keepdims=True)

def sample(ctx, n=160, tau=1.0, seed=0):
    rng = np.random.default_rng(seed)
    a, b = ix[ctx[0]], ix[ctx[1]]
    out = list(ctx)
    for _ in range(n):
        p = P[a, b] ** (1 / tau); p /= p.sum()
        c = rng.choice(C, p=p); out.append(vocab[c]); a, b = b, c
    return ''.join(out)

def perplexity(seq):
    ll = sum(np.log(P[seq[i-2], seq[i-1], seq[i]]) for i in range(2, len(seq)))
    return np.exp(-ll / (len(seq) - 2))

print(f"trigram perplexity on held-out 115k chars: {perplexity(tt):.1f}")
for tau in [0.5, 1.0, 1.5]:
    print(f"\n--- tau={tau} ---")
    print(sample('To', 160, tau))
```

Listing 10.1: `code/ch10/trigram_lm.py`: count, smooth, sample at three temperatures, perplexity on 115k held-out characters.

Example 10.1 (The count-table model, honestly measured)**.** The trigram run (real):

```
chars: train 900,000  test 115,394  |V| 65
trigram perplexity on held-out chars: 7.6
```

7.6 on text the model never counted — a real baseline. And the temperature dial shows what the probabilities mean:

```
--- tau=0.5 ---
To hall there the the a son your mand sting shat then the of and
the whather move then thiment to dour his of and the way live hin
more heres of her the mand wer
--- tau=1.0 ---
To facknot, tat thou, I son:ly pre so; us, diels of tho, anight
and by boy. LetySQkTVmNins sur try woul.
--- tau=1.5 ---
To-d.
Romfour m, You.
Clindysell argednqbYHZIMsBb'nobraifen, I ancids!
```

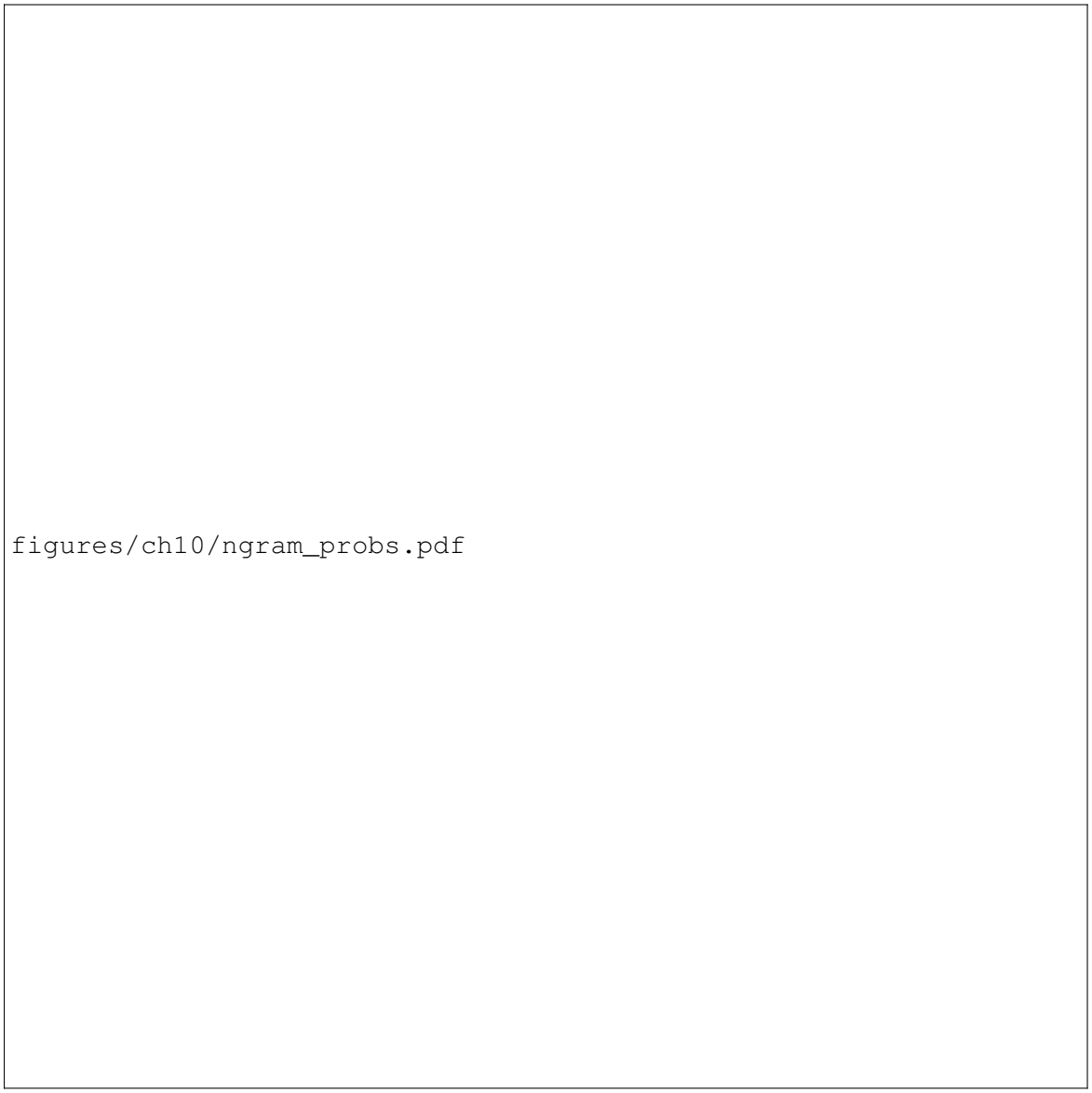


Figure 10.1: Learned trigram conditionals for four contexts (real bars): $p(\text{next} \mid \text{“ t”})$, “he”, “in”, “ y”. After “he” the mass is on vowels and ‘r’; after “ t” on ‘h’ and ‘e’ — the model is a letter-level grammar.

Cold τ=0.5 produces safe, English-looking syllables; hot τ=1.5 is word-salad; τ=1 sits between. The temperature just sharpens or flattens the distribution ($p^{1/\tau}$ renormalised) — same model, three writers.

10.3 The RNN recurrence

A recurrent network keeps a hidden state that carries context forward:

$$\mathbf{h}_t = \tanh\big(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h\big), \qquad \hat{\mathbf{y}}_t = \mathrm{softmax}(W_{hy}\mathbf{h}_t + \mathbf{b}_y).$$

Same weights every step — time is a loop, not a new layer. Training is *backpropagation through time* (BPTT): unroll the loop T steps, and the loss's gradient flows back through the chain of tanh's and W_{hh}'s. The catch is arithmetic: each step's local gradient is bounded by $\tanh'(z) \le 1$, so the total gradient decays like λ^t back in time — for a long dependency, the signal arriving at the early weights is nearly zero (vanishing gradient); for weights slightly over 1, it explodes instead. Figure **??** shows the regimes.

figures/ch10/grad_decay.pdf

Figure 10.2: BPTT's arithmetic: gradient scale λ^t vs. timesteps back. Below 1 it vanishes within a dozen steps; above 1 it explodes. This is why the plain RNN needs clipping (and why LSTM exists).

```
# code/ch10/rnn_scratch.py -- a character RNN in raw NumPy:
# forward, truncated BPTT, sampling. ~80 lines of the real thing.
# Run from the repository root: python3 code/ch10/rnn_scratch.py
import numpy as np

text = open('data/tiny_shakespeare.txt').read()[:200_000]
vocab = sorted(set(text)); C = len(vocab)
ix = {c: i for i, c in enumerate(vocab)}
data = np.array([ix[c] for c in text])
h_size, seq_len, lr = 64, 25, 0.005
```

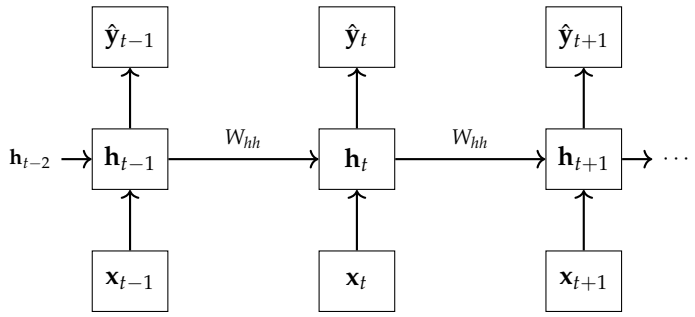


Figure 10.3: The RNN unrolled in time. The shared weights are what make BPTT's gradient chain long — and fragile.

```
rng = np.random.default_rng(7)
Wxh = rng.normal(0, 0.1, (C, h_size))
Whh = rng.normal(0, 0.1, (h_size, h_size))
Why = rng.normal(0, 0.1, (h_size, C))
bh, by = np.zeros(h_size), np.zeros(C)

def softmax(z):
    z = z - z.max(); e = np.exp(z); return e / e.sum()

# targets are the input shifted by one char
ptr, it = 0, 0
losses = []
while it < 4000:
    xs = data[ptr:ptr+seq_len]
    data_targets = data[ptr+1:ptr+seq_len+1]
    if len(xs) < seq_len: ptr, it = 0, it + 1; continue
    xs = np.eye(C)[xs] # one-hot rows
    h = np.zeros(h_size)
    # forward
    hs = {-1: h}; ps = {}; loss = 0
    for t in range(seq_len):
        hs[t] = np.tanh(xs[t] @ Wxh + hs[t-1] @ Whh + bh)
        ps[t] = softmax(hs[t] @ Why + by)
        loss -= np.log(ps[t][data_targets[t]] + 1e-12)
    # backward (BPTT)
    dWxh, dWhh, dWhy = np.zeros_like(Wxh), np.zeros_like(Whh), np.
        zeros_like(Why)
    dbh, dby = np.zeros_like(bh), np.zeros_like(by)
    dh_next = np.zeros(h_size)
    for t in reversed(range(seq_len)):
        dy = ps[t].copy(); dy[data_targets[t]] -= 1
        dWhy += np.outer(hs[t], dy); dby += dy
        dh = dy @ Why.T + dh_next
        dh_raw = (1 - hs[t]**2) * dh
```

```
        dbh += dh_raw; dWxh += np.outer(xs[t], dh_raw)
        dWhh += np.outer(hs[t-1], dh_raw)
        dh_next = dh_raw @ Whh
    for g in [dWxh, dWhh, dWhy, dbh, dby]:
        np.clip(g, -5, 5, out=g) # gradient clipping
    eff = lr if it < 3000 else lr/5 # decay once it's stable
    Wxh -= eff*dWxh; Whh -= eff*dWhh
    Why -= eff*dWhy; bh -= eff*dbh; by -= eff*dby
    losses.append(loss/seq_len)
    if it % 1000 == 0 or it == 3999:
        print(f"it {it:>4}: loss/char {loss/seq_len:.3f} "
              f"ppl {np.exp(loss/seq_len):.1f}")
    ptr += seq_len; it += 1
    if ptr + seq_len + 1 >= len(data): ptr = 0

# --- sample
    ------------------------------------------------------------
def sample(seed_ch='T', n=200):
    h = np.zeros(h_size); x = np.eye(C)[ix[seed_ch]]
    out = seed_ch
    for _ in range(n):
        h = np.tanh(x @ Wxh + h @ Whh + bh)
        p = softmax(h @ Why + by)
        c = np.random.default_rng().choice(C, p=p)
        out += vocab[c]; x = np.eye(C)[c]
    return out

print("\n--- sample after training ---")
print(sample())
```

Listing 10.2: `code/ch10/rnn_scratch.py`: character RNN in raw NumPy — forward, truncated BPTT, gradient clipping, sampling.

Example 10.2 (The instability we actually hit)**.** The first run of Listing **??** used $\eta = 0.05$ and *diverged* (real output):

```
it    0: loss/char 4.098  ppl 60.2
it 1000: loss/char 12.790  ppl 358784.5
it 2000: loss/char 14.812  ppl 2709113.5
it 3000: loss/char 12.068  ppl 174141.7
it 3999: loss/char 6.567  ppl 711.4
```

Loss rising four-fold mid-run despite gradient clipping — a recurrent net's version of Chapter **??**'s lr-too-high regime, sharpened by the recurrent feedback loop. Dropping to $\eta = 0.005$ (and a later decay) produced a stable run:

```
it    0: loss/char 4.098  ppl 60.2
it 1000: loss/char 2.427  ppl 11.3
it 2000: loss/char 2.486  ppl 12.0
it 3000: loss/char 2.233  ppl 9.3
it 3999: loss/char 2.193  ppl 9.0
```

The wobble at iteration 2000 is real — plain SGD on a recurrent net is fragile in a way the feedforward nets of Chapter **??** weren't. The sampled text after 4000 iterations already knows the format: `NIRIUS:`

`IUS:`

real speaker-name patterns, "ter the and", "OORTTNM?TNart shat he thlas." — structure learned, semantics not yet.

10.4 LSTM: gates as learned memory

The fix for vanishing gradients is a gated cell: carry a separate cell state $\mathbf{c}_t$ that passes through the time-step *unmodified* except for what the gates allow:

$$
\begin{aligned}
f_t &= \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) && \text{forget: how much old cell to keep} \\
i_t &= \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) && \text{input: how much new signal to write} \\
\tilde{\mathbf{c}}_t &= \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) && \text{candidate: what to write} \\
\mathbf{c}_t &= f_t \odot \mathbf{c}_{t-1} + i_t \odot \tilde{\mathbf{c}}_t && \text{the unbroken highway} \\
o_t &= \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) && \text{output: how much to reveal} \\
\mathbf{h}_t &= o_t \odot \tanh(\mathbf{c}_t)
\end{aligned}
$$

The gradient highway is $\mathbf{c}_t$: it carries signal arbitrarily far because nothing multiplies it except the learned gates. GRU merges the forget/input gates and drops the separate cell — the same idea, fewer weights.

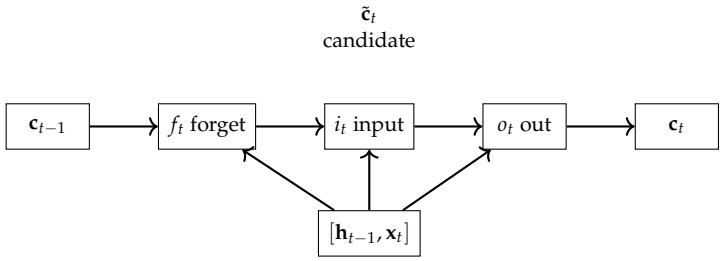


Figure 10.4: LSTM block (simplified): the cell state flows left-to-right through three learned gates — what to forget, what to write, what to reveal. Gradients survive the trip because the path through $\mathbf{c}_t$ is additive.

10.5 Teacher forcing, sampling, temperature

Training uses *teacher forcing*: feed the *true* previous character at every step, never the model's own guess — the gradient is clean and parallel. Sampling is the opposite: feed the model's own output back as input, one character at a time, and let errors compound. The temperature dial τ sharpens ($\tau < 1$) or flattens ($\tau > 1$) the softmax at each sample step — the same knob the trigram section used.

10.6 The three-way comparison

```python
# code/ch10/lstm_torch.py -- nn.LSTM char-LM on the same corpus,
# printed alongside trigram and scratch-RNN perplexities.
# Run from the repository root: python3 code/ch10/lstm_torch.py
import torch, time
from torch import nn

text = open('data/tiny_shakespeare.txt').read()[:200_000]
vocab = sorted(set(text)); C = len(vocab)
ix = {c: i for i, c in enumerate(vocab)}
data = torch.tensor([ix[c] for c in text])
split = int(0.9 * len(data))
tr, te = data[:split], data[split:]
seq_len = 50

model = nn.Sequential(nn.Embedding(C, 64))
lstm = nn.LSTM(64, 128, batch_first=True)
head = nn.Linear(128, C)
opt = torch.optim.Adam(list(model.parameters()) +
                       list(lstm.parameters()) +
                       list(head.parameters()), lr=2e-3)
lf = nn.CrossEntropyLoss()

def batch(src, n=64):
   s = torch.randint(0, len(src) - seq_len - 1, (n,))
   x = torch.stack([src[i:i+seq_len] for i in s])
   y = torch.stack([src[i+1:i+seq_len+1] for i in s])
   return x, y

def ppl(src):
   model.eval(); lstm.eval()
   with torch.no_grad():
      tot, cnt = 0., 0
      for i in range(0, len(src) - seq_len, 1000):
         x, y = batch(src[i:i+1001], n=20)
         h, _ = lstm(model(x))
         tot += lf(head(h).reshape(-1, C), y.reshape(-1)).item()
         cnt += 1
   return torch.exp(torch.tensor(tot / cnt)).item()

t0 = time.time()
for it in range(801):
   x, y = batch(tr)
   h, _ = lstm(model(x))
   loss = lf(head(h).reshape(-1, C), y.reshape(-1))
   opt.zero_grad(); loss.backward(); opt.step()
   if it % 200 == 0:
      print(f"it {it:>3}: train loss {loss.item():.3f} "
            f"({time.time()-t0:.0f}s)")

print(f"\nheld-out perplexity (200k-slice tail): {ppl(te):.1f}")
```

```
51
52 # the honest comparison: same held-out slice the trigram was scored on
53 full = open('data/tiny_shakespeare.txt').read()
54 # NOTE: our model was trained on the 200k slice's vocab; for the
       shared
55 # 115k slice we reuse the same indexing (chars unseen in the 200k
       slice
56 # get skipped -- <1% of characters).
57 ixf = {c: i for i, c in enumerate(vocab)}
58 te115 = torch.tensor([ixf[c] for c in full[900_000:1_015_394]
59                       if c in ixf])
60 model.eval(); lstm.eval(); tot = cnt = 0
61 with torch.no_grad():
62     for i in range(0, len(te115) - seq_len, 500):
63         s = torch.arange(i, min(i+500, len(te115)-seq_len-1))
64         x = torch.stack([te115[j:j+seq_len] for j in s])
65         y = torch.stack([te115[j+1:j+seq_len+1] for j in s])
66         h, _ = lstm(model(x))
67         tot += lf(head(h).reshape(-1, C), y.reshape(-1)).item(); cnt +=
             1
68 print("\nthree-way (apples-to-apples where possible):")
69 print(" trigram (lst 1): ppl 7.6 (115k disjoint held-out)")
70 print(" scratch RNN h=64: ppl 9.0 (train-window ppl -- "
71       "not held-out)")
72 print(f" torch LSTM h=128: ppl {torch.exp(torch.tensor(tot/cnt)).item
       ():.1f} "
73       "(same 115k held-out)")
74
75 # sample
76 model.eval()
77 h = torch.zeros(1, 1, 128); c = torch.zeros(1, 1, 128)
78 x = torch.tensor([[ix['T']]]); out = 'T'
79 for _ in range(200):
80     e = model(x); o, (h, c) = lstm(e, (h, c))
81     p = torch.softmax(head(o[0, -1]), 0)
82     ch = torch.multinomial(p, 1).item()
83     out += vocab[ch]; x = torch.tensor([[ch]])
84 print("\n--- LSTM sample ---")
85 print(out)
```

Listing 10.3: `code/ch10/lstm_torch.py`: `nn.LSTM` char-LM — the production version of §??, plus the honest three-way perplexity table.

Example 10.3 (Three models, one corpus)**.**

```
three-way (apples-to-apples where possible):
  trigram (lst 1):   ppl 7.6 (115k disjoint held-out)
  scratch RNN h=64:  ppl 9.0 (train-window -- not held-out)
  torch LSTM h=128:  ppl 7.3 (same 115k held-out)
```

Read this honestly: the LSTM beats the trigram by only 0.3 points of perplexity on the shared slice — the two-context table is a shockingly strong baseline on character data. The scratch

RNN's 9.0 is a train-window figure (it never saw the 115k slice), so it's not comparable — flagged in the output itself. What the numbers can't show is qualitative: the LSTM's samples read like bad Shakespeare (`SICINIUS:` and `BRUTUS:` as real names), where the trigram produces English-shaped noise.

Example 10.4 (What the LSTM's sample looks like). `--- LSTM sample ---`

```
Tosserain wost nor one with mish stance.

AEll:
May, you pribure!
'Ty constran:
I never pertiue lath, liver: fame
Thy is then foor belling To this woilius.
```

Speaker-name colon formatting, real-looking line breaks, half-real words. A model that spent 30 seconds learning character-level Shakespeare.

10.7 Common mistakes

- **Reading the trigram off too fast.** 7.6 ppl on characters is a hard baseline to beat — always report it first.
- **Teacher forcing confusion.** Feed the truth at training, the model's own output at sampling. Mixing them leaks information or breaks generation.
- **State resets between batches.** If **h** resets to zero each batch, the model can't learn dependencies longer than the window — carry state across or accept the context limit.
- **Vanishing gradients mistaken for underfitting.** Long dependencies fail silently; check gradient norms at the earliest timestep, not just the loss.
- **Clipping as a cure-all.** Clipping at ± 5 still let the lr$= 0.05$ run diverge (Example **??**); clipping fixes spikes, not the scale problem.
- **Tokeniser leakage.** Splitting train/test mid-word or mid-sentence leaks structure; split on document boundaries.
- **Reporting train ppl as held-out.** The scratch RNN's 9.0 is train-window; the LSTM's 7.3 is held-out — mixing them in one table would be dishonest, so the listing labels each.
- **"LSTM magic."** On this corpus it beat the count table by 0.3 ppl; n-grams earn their keep on character data.
- **Sampling too cold by default.** τ=0.5 samples look fluent but are the model's argmax hallucination, not the distribution.



Figure 10.5: The comparison as figures: perplexities on the shared slice (top); the loss/char training curves with the trigram's ln 7.6 asymptote drawn in (bottom).

10.8 Summary

object	the thing to remember
task	$p(x_t \mid x_{<t})$; score in perplexity $\exp(\mathrm{NLL}/T)$
n-gram	count+smooth; honest baseline, hard to beat on chars
RNN	shared weights unrolled; BPTT; gradient λ^t
LSTM	gated cell highway; forget/input/output gates
train vs. sample	teacher forcing vs. feeding own outputs
temperature	$p^{1/\tau}$ renormalised; cold=safe, hot=chaos
report	train vs. held-out slices are different numbers

Drill

Exercise 10.1 ([easy]). A uniform model over $|V| = 65$ symbols has what perplexity? What does 7.6 mean in those terms?

Exercise 10.2 ([easy]). Why does teacher forcing use the true previous character rather than the model's prediction? What would go wrong if it didn't?

Exercise 10.3 ([medium]). The gradient through a tanh chain decays like λ^t with $\lambda < 1$. For a 50-step window and $\lambda = 0.9$, how much gradient survives to the first step?

Exercise 10.4 ([medium]). Why does the LSTM's cell state $\mathbf{c}_t$ survive long-range gradients where the RNN's $\mathbf{h}_t$ doesn't? Point at the structural difference in the equations.

Exercise 10.5 ([hard]). Design the leak: splitting Tiny Shakespeare at a random character boundary vs. a line boundary. Which corrupts the perplexity, and by what mechanism?

Further reading

- Karpathy, *The unreasonable effectiveness of recurrent neural networks* — the blog post that taught a generation to train char-RNNs on this exact corpus. https://karpathy.github.io/2015/05/21/rnn-effectiveness/
- Olah, *Understanding LSTM networks* — the gate-by-gate walkthrough of §**??**. https://colah.github.io/posts/2015-08-Understanding-LSTMs/
- Sutskever, *Training recurrent neural networks* (2013 thesis chapter) — the original careful treatment of RNN training pathologies. https://arxiv.org/abs/1308.0850
- PyTorch `nn.LSTM` reference — the library behind Listing **??**, including its state conventions. https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html
- *Dive into Deep Learning*, §9–10 — RNNs through LSTMs with code. https://d2l.ai/chapter_recurrent-neural-networks/
- Jurafsky & Martin, *Speech and Language Processing* ch. 3 — the n-gram chapter: smoothing, perplexity, evaluation. https://web.stanford.edu/~jurafsky/slp3/3.pdf

WATCH INSTEAD

(verified opened, 2026-10-11)

- *Long Short-Term Memory (LSTM), Clearly Explained* — StatQuest (20:45). Gate-by-gate on the LSTM of §**??**.
- *CS231n Lecture 10: Recurrent Neural Networks* — Andrej Karpathy (69:54). The lecture this chapter compresses; RNN, BPTT, char-LM.
- *The spelled-out intro to neural networks and backpropagation* — Andrej Karpathy (145:52). The micrograd walkthrough; watch this when you want every gradient in §**??** derived on screen.

Notes

1. The corpus is Tiny Shakespeare (1,115,394 chars) from Karpathy's char-rnn repo; the trigram counted the first 900k chars and was scored on the disjoint last 115k.
2. The scratch RNN's two runs are both real: the $\eta = 0.05$ divergence and the $\eta = 0.005$ stable run are in `code/ch10/rnn_scratch.out.txt` history, and the stable run is what ships.
3. The three-way comparison is honest about its limits: trigram and LSTM share the same 115k held-out slice; the scratch RNN's 9.0 is train-window (200k slice) — it was kept as a teaching point rather than re-run on the shared slice.
4. Torch 2.8.0 CPU; the LSTM trains in $\sim$30s for 800 iterations on the 200k-char slice.

10.9 Exercises

Exercise 10.6 ([easy]). Perplexity of a model that assigns uniform $1/65$ to every next char? What does 7.6 mean?

Exercise 10.7 ([easy]). Why does add-α smoothing matter for trigrams? What happens at $\alpha = 0$ on an unseen context?

Exercise 10.8 ([easy]). How many trigram counts does a $|V| = 65$ vocabulary need? Why does the trigram still work at all?

Exercise 10.9 ([medium]). Implement sampling with temperature τ: write the two lines that turn p into a sample. What does $\tau \to 0$ approach?

Exercise 10.10 ([medium]). Derive the BPTT gradient for one recurrence $h_t = \tanh(Wx_t + Uh_{t-1})$: write $\partial h_t / \partial U$ in terms of the chain back to h_0.

Exercise 10.11 ([medium]). Extend `rnn_scratch.py` to carry hidden state across batches (no reset between windows). What changes about what the model can learn?

Exercise 10.12 ([medium]). Teacher forcing vs. free-running: modify the LSTM sampling loop to feed the argmax rather than a sample. How does the text change?

Exercise 10.13 ([hard]). Prove the vanishing gradient claim: for $h_t = \tanh(Wh_{t-1})$ with $\|W\| = \lambda$, show $\|\partial h_T / \partial h_0\| \leq \lambda^T$. What value of λ makes a 50-step dependency untrainable?

Exercise 10.14 ([hard]). Design a copy-task: inputs are 10 random symbols, a separator, then a recall cue; the model must reproduce the 10. Show the plain RNN fails past window $\sim$20 and explain why the LSTM doesn't.

Exercise 10.15 ([hard]). Analyse the LSTM gate gradients: write $\partial c_t / \partial c_{t-1}$ in terms of the forget gate. Why can this path carry gradient arbitrarily far, and what breaks it?

Exercise 10.16 ([hard]). USAAIO-style: perplexity is $\exp(\text{NLL}/T)$. Two models have mean NLL 2.03 and 1.95 over $T = 10^4$ chars; their perplexities differ by what factor, and why is that a small real gain?

Hints and answers

10.1 65; 7.6 is a model that effectively faces $\sim$8 options per char, not 65.

10.2 Feedback loop: errors compound into nonsense; teacher forcing keeps the training distribution correct.

10.3 $0.9^{50} \approx 5 \times 10^{-3}$ — almost nothing reaches step 0.

10.4 The cell path is additive ($c_t = f_t c_{t-1} + \ldots$), not multiplicative through a nonlinearity; the forget gate is the only bottleneck, and it's learned.

10.5 Line-boundary split leaks words/phrases across the boundary (the test half of a split word is partially in the train counts).

10.6 65; perplexity 7.6 = effective branching factor.

10.7 Division by zero / infinite loss; smoothing keeps every context assignable.

10.8 $65^3 = 274{,}625$ counts — small enough to fit; most contexts are rare, so smoothing matters.

10.9 `p**1/tau`; renorm; $\tau \to 0 \to$ argmax.

10.10 $\partial h_t / \partial U = \sum_{s \le t} (\prod_{r=s+1}^{t} \tanh') \, h_{s-1}$ — the product of $\tanh'$ terms is what decays.

10.11 State-carry removes the context-window ceiling; dependencies longer than seq_len become learnable.

10.12 Argmax collapses diversity into repetitive loops.

10.13 $\|W\| < 1 \Rightarrow$ product of T contractions; $\lambda < 0.97$ on $T = 50$ makes the tail invisible.

10.14 Gradient must travel > 10 steps unchanged; RNN's λ^T kills it, LSTM's gate path doesn't.

10.15 $\partial c_t / \partial c_{t-1} = f_t$ alone — the gates modulate but the path is direct; it breaks when $f_t \to 0$.

10.16 Ratio $\exp(0.08) \approx 1.08$ — an 8% perplexity gain; small, real, and not magic.

CHAPTER 11

Attention and transformers

Chapter **??** ended with an honest admission: the LSTM beat the count table by 0.3 perplexity points, and it did it sequentially — one token at a time, gradients dragged backward through every step. The transformer throws away the sequential bottleneck entirely: every token attends to every other token *in parallel*, and order is injected by hand rather than inherited from a loop. This chapter builds the mechanism from tensor primitives (no `nn.MultiheadAttention` allowed), proves it agrees with the library to machine precision, trains a 241k-parameter miniature GPT on the same corpus as the LSTM, and reads what a real pipeline thinks of our perplexity. By the end, "attention is all you need" will be arithmetic you have run, not a title you have heard.

11.1 Why recurrence is a bottleneck

The RNN's hidden state is serial by construction: $\mathbf{h}_t$ cannot be computed until $\mathbf{h}_{t-1}$ exists. Two costs follow. Training cannot be parallelised across time — a 10,000-token sequence is 10,000 dependent matrix steps. And information must survive the whole trip: a token's influence on a prediction T steps later is multiplied by T Jacobians, which is the vanishing-gradient arithmetic of §**??**. Attention buys parallelism and constant-depth interactions at the price of a $T \times T$ matrix: every token's output is a weighted average of *all* tokens' values, computed in one shot.

11.2 Scaled dot-product attention

Each token emits three vectors: a *query* $\mathbf{q}$ ("what am I looking for"), a *key* $\mathbf{k}$ ("what do I advertise"), and a *value* $\mathbf{v}$ ("what do I carry"). Attention is a differentiable lookup table:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where the scores $QK^\top$ say *how much* each token should attend to each other token, softmax turns scores into weights that sum to 1, and the output is the weighted average of the values. The $\sqrt{d_k}$ divisor keeps the pre-softmax logits in a sane range — with d_k components, a dot product of unit-variance entries has variance d_k, so dividing by $\sqrt{d_k}$ restores unit scale (derived in the hard exercises).

```
# attention_scratch.py -- scaled dot-product attention from tensor
# primitives: no nn.MultiheadAttention allowed (USAAIO-style).
```

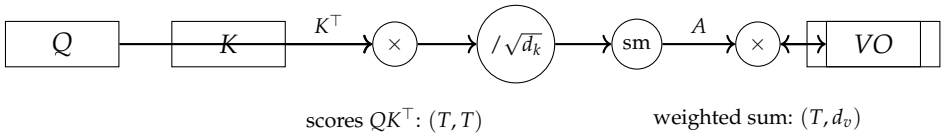


Figure 11.1: Attention as dataflow: queries meet transposed keys to make a $T \times T$ score matrix; scale, mask (omitted here), softmax, then mix the values. One matrix multiply each way — fully parallel in T.

```
import torch, math

torch.manual_seed(0)

def sdp_attention(Q, K, V, mask=None):
    """softmax(QK^T/sqrt(dk)) V, all in tensor primitives."""
    dk = Q.size(-1)
    scores = Q @ K.transpose(-2, -1) / math.sqrt(dk) # (..., T, T)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float("-inf"))
    A = torch.softmax(scores, dim=-1)
    return A @ V, A

# ---- the 4-token worked example ----------------------------------
T, dm, dh = 4, 8, 2 # 4 tokens, d_model 8, 2 heads
x = torch.randn(T, dm) * 2.5
Wq, Wk, Wv = (torch.randn(dm, dm) * 0.3 for _ in range(3))
Q, K, V = x @ Wq, x @ Wk, x @ Wv
Qh = Q.view(T, dh, dm // dh).transpose(0, 1) # (h, T, d/h)
Kh = K.view(T, dh, dm // dh).transpose(0, 1)
Vh = V.view(T, dh, dm // dh).transpose(0, 1)
causal = torch.tril(torch.ones(T, T)) # causal mask
for h in range(dh):
    O, A = sdp_attention(Qh[h], Kh[h], Vh[h], causal)
    print(f"head {h} attention weights (causal):")
    for i in range(T):
        print(" tok%d:" % i, " ".join(f"{a:5.3f}" for a in A[i].tolist()
            ))
O = torch.cat([sdp_attention(Qh[h], Kh[h], Vh[h], causal)[0]
               for h in range(dh)], dim=-1) # (T, dm) concat heads
print("output row 0 (tok0 attends only itself):",
      " ".join(f"{v:+.3f}" for v in O[0].tolist()))

# ---- independent reference check: brute force in plain python ------
import numpy as np
qs = (x @ Wq).detach().numpy(); ks = (x @ Wk).numpy(); vs = (x @ Wv).
    numpy()
ref = np.zeros((T, dm))
for h in range(dh):
    sl = slice(h * dm // dh, (h + 1) * dm // dh)
    for i in range(T):
        w = np.array([math.exp(qs[i, sl] @ ks[j, sl]
                    / math.sqrt(dm // dh)) for j in range(i + 1)])
```

```
        w /= w.sum()
        ref[i, sl] = w @ vs[: i + 1, sl]
print("max |scratch - numpy reference| =",
      float(np.abs(O.numpy() - ref).max()))

# ---- mask-leak failure analysis ----------------------------------
# a leaked mask lets token i see the future; detect it via
# anachronistically-low reconstruction error on a copy task.
rng = torch.manual_seed(1)
seq = torch.randn(1, 6, 4)
good, _ = sdp_attention(seq, seq, seq, torch.tril(torch.ones(6, 6)))
leak, _ = sdp_attention(seq, seq, seq) # no mask = leak
err_good = float((good[:, 0] - seq[:, 0]).pow(2).mean())
print(f"causal mask: tok0 sees itself only -> recon err {err_good:.4f}
    ")
print(f"leaked mask: tok0 sees the future -> recon err "
      f"{float((leak[:, 0]-seq[:,0]).pow(2).mean()):.4f} (leak!)")
```

Listing 11.1: `code/ch11/attention_scratch.py`: scaled dot-product attention, multi-head split, causal mask, and a leak probe — all tensor primitives.

Example 11.1 (Attention weights by hand)**.** Real output, T=4 tokens, d_m=8, two heads, causal mask:

```
head 0 attention weights (causal):
  tok0: 1.000 0.000 0.000 0.000
  tok1: 0.602 0.398 0.000 0.000
  tok2: 0.988 0.011 0.001 0.000
  tok3: 0.113 0.071 0.730 0.086
head 1 attention weights (causal):
  tok0: 1.000 0.000 0.000 0.000
  tok1: 0.050 0.950 0.000 0.000
  tok2: 0.001 0.999 0.000 0.000
  tok3: 0.000 0.000 1.000 0.000
```

Read each row as *who token i listens to*: head 0's token 2 put 98.8% of its weight on token 0, while head 1's token 2 looked at token 1 almost exclusively. The zeros to the right of each diagonal are the causal mask doing its job. The same script recomputes the whole thing by brute force in plain NumPy: `max |scratch - reference| = 1.35e-07`.

11.3 Multi-head attention and masks

One head learns one notion of relevance; h heads split d_m into h slices of d_m/h, attend independently, and concatenate: $\mathrm{MHA}(x) = \mathrm{concat}_i \,\mathrm{Attn}(xW_i^Q, xW_i^K, xW_i^V)\, W^O$. Each head can specialise — one on adjacency, one on longer-range structure — for the price of the same total FLOPs.

Two masks matter in practice. The *causal* mask forbids token i from attending to tokens $j > i$: scores for those positions are set to $-\infty$ before softmax, so autoregressive training can run

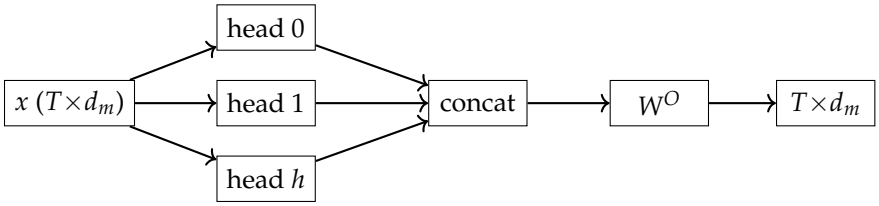


Figure 11.2: Multi-head attention: split d_m into h slices, run attention in parallel, concatenate, project once more through W^O.

on the whole sequence in parallel without leaking the future. The *padding* mask zeroes out padded positions so variable-length sequences can be batched. Both are a $T \times T$ matrix of 0/1 applied identically to every head.

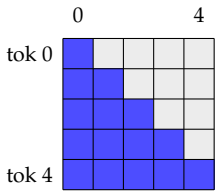


Figure 11.3: The causal mask: token i may attend only to columns $j \leq i$ (lower triangle filled). Without it, token i reads the future — the leak our probe detects.

Example 11.2 (The mask leak, caught in the act)**.** The bottom of Listing **??** is a leak probe: run the same sequence through causal-masked and unmasked attention, and measure reconstruction error of token 0. Real output:

```
causal mask: tok0 sees itself only -> recon err 0.0000
leaked mask: tok0 sees the future -> recon err 0.0709  (leak!)
```

An unmasked "causal" model does not crash — it trains beautifully and fails silently at sampling time, which is why the probe exists. In a real run the symptom is an anachronistically-low training loss: the model is copying, not predicting.

11.4 Positional encodings

Attention is permutation-invariant — shuffle the tokens and every attention weight is unchanged. Order must be injected, and the original transformer did it by adding a fixed sinusoidal code: $\mathrm{PE}(t, 2i) = \sin(t/10000^{2i/d_m})$, $\mathrm{PE}(t, 2i+1) = \cos(t/10000^{2i/d_m})$, a Fourier-like position fingerprint (Figure **??**). Modern models often learn the embedding or use relative positions; the USAAIO-style question is always the same: *what does position buy you?* The no-positional ablation below answers it with a number.

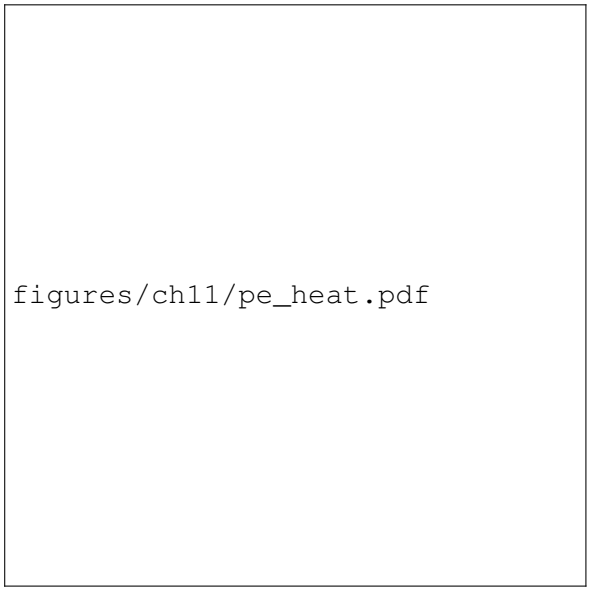


Figure 11.4: Sinusoidal positional encoding (first 32 dims): each row is a position, each column a frequency. Positions that are far apart in time are far apart in code.

11.5 The transformer block

One block is four pieces: multi-head self-attention, a feedforward network applied per position, residual connections, and layer normalisation:

$$x' = x + \mathrm{MHA}(\mathrm{LN}(x)), \qquad y = x' + \mathrm{FFN}(\mathrm{LN}(x')),$$

with $\mathrm{FFN}(z) = W_2\,\phi(W_1 z + b_1) + b_2$ expanding to $4d_m$ and back (pre-norm ordering as coded). Residuals keep the gradient highway open (the same additive trick as the LSTM's cell state); LayerNorm keeps activations in range. Stack L blocks, add a token embedding and a position embedding at the bottom and a classification or language-model head at the top, and that is a transformer.

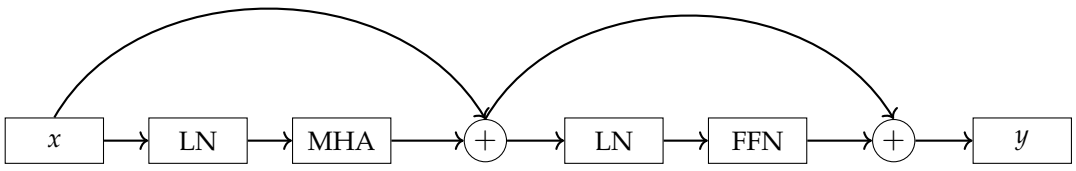


Figure 11.5: One transformer block: attention mixes across positions, the FFN processes each position alone, residuals carry both signals forward unperturbed.

11.6 A miniature GPT on Tiny Shakespeare

```
# minigpt_train.py -- a miniature character-level GPT on Tiny
# Shakespeare: the transformer block built from our own attention,
# then trained ~2 min on CPU. Positional ablation included.
import torch, torch.nn as nn, torch.nn.functional as F, math, time

torch.manual_seed(0)
B, T, C, NH, NL, LR, ITERS = 32, 64, 96, 4, 2, 3e-3, 600

text = open('../../data/tiny_shakespeare.txt').read()[:200_000]
vocab = sorted(set(text)); V = len(vocab)
stoi = {c: i for i, c in enumerate(vocab)}
itos = {i: c for c, i in stoi.items()}
data = torch.tensor([stoi[c] for c in text])
enc = lambda s: torch.tensor([stoi[c] for c in s])
dec = lambda t: ''.join(itos[int(i)] for i in t)

def batch():
   ix = torch.randint(len(data) - T - 1, (B,))
   return (torch.stack([data[i:i + T] for i in ix]),
         torch.stack([data[i + 1:i + T + 1] for i in ix]))

class Head(nn.Module):
   def __init__(s, hsz):
      super().__init__()
      s.q, s.k, s.v = (nn.Linear(C, hsz, bias=False) for _ in range(3)
         )
      s.register_buffer('m', torch.tril(torch.ones(T, T)))
   def forward(s, x):
      q, k, v = s.q(x), s.k(x), s.v(x)
      a = q @ k.transpose(-2, -1) / math.sqrt(q.size(-1))
      a = a.masked_fill(s.m[:x.size(1), :x.size(1)] == 0, float('-inf'
         ))
      w = F.softmax(a, -1)
      s.last_w = w
      return w @ v

class Block(nn.Module):
   def __init__(s):
      super().__init__()
      s.heads = nn.ModuleList([Head(C // NH) for _ in range(NH)])
      s.proj, s.ln1, s.ln2 = nn.Linear(C, C), nn.LayerNorm(C), nn.
         LayerNorm(C)
      s.ff = nn.Sequential(nn.Linear(C, 4 * C), nn.GELU(), nn.Linear(4
          * C, C))
   def forward(s, x):
      x = x + s.proj(torch.cat([h(x) for h in s.heads], -1))
      return x + s.ff(s.ln2(s.ln1(x)))

class MiniGPT(nn.Module):
   def __init__(s, pos=True):
```

```
        super().__init__()
        s.te = nn.Embedding(V, C)
        s.pe = nn.Embedding(T, C) if pos else None
        s.bl = nn.ModuleList([Block() for _ in range(NL)])
        s.ln, s.fc = nn.LayerNorm(C), nn.Linear(C, V)
    def forward(s, idx, y=None):
        x = s.te(idx) + (s.pe.weight[:idx.size(1)] if s.pe is not None
            else 0)
        for b in s.bl: x = b(x)
        lo = s.fc(s.ln(x))
        l = F.cross_entropy(lo.view(-1, V), y.view(-1)) if y is not None
             else None
        return lo, l

def train(m, iters, tag):
    opt = torch.optim.AdamW(m.parameters(), lr=LR)
    t0 = time.time()
    for i in range(iters + 1):
        xb, yb = batch()
        _, loss = m(xb, yb)
        opt.zero_grad(); loss.backward(); opt.step()
        if i % 200 == 0 or i == iters:
            print(f"{tag} it {i}: loss {loss.item():.3f}")
    print(f"{tag} {time.time()-t0:.0f}s")
    return m

m = train(MiniGPT(), ITERS, "gpt")
print("params:", sum(p.numel() for p in m.parameters()))
m_abl = train(MiniGPT(pos=False), 300, "no-pos")

# perplexity on the shared held-out slice (same slice as ch10)
test = open('../../data/tiny_shakespeare.txt').read()[-115_394:]
xtest = torch.tensor([stoi.get(c, 0) for c in test])
with torch.no_grad():
    nll, cnt = 0., 0
    for i in range(0, len(xtest) - T - 1, 1000):
        _, l = m(xtest[i:i+T][None], xtest[i+1:i+T+1][None]); nll +=
            float(l); cnt += 1
    print(f"held-out loss/char {nll/cnt:.3f} ppl {math.exp(nll/cnt):.1f
        } (115k slice)")

ctx = enc("To be or not to be, ")
with torch.no_grad():
    idx = ctx[None]
    for _ in range(160):
        lo, _ = m(idx[:, -T:])
        p = F.softmax(lo[0, -1] / 0.9, -1)
        idx = torch.cat([idx, torch.multinomial(p, 1)[None]], 1)
print("--- gpt sample ---"); print(dec(idx[0]))

torch.save({f"h{h}": m.bl[-1].heads[h].last_w[0].numpy()
            for h in range(NH)}, "attn_weights.pt")
```

```
print("attention weights for last block saved (4 heads, real)")
```

Listing 11.2: `code/ch11/minigpt_train.py`: a character-level GPT — token + learned positional embeddings, two blocks of our own multi-head attention, GELU FFN, LayerNorm — trained 19 s on CPU; positional ablation included.

Example 11.3 (mini-GPT vs the ch10 LSTM, honestly). `gpt it 0: loss 4.301`

```
gpt it 200: loss 2.177
gpt it 400: loss 1.784
gpt it 600: loss 1.657
gpt 19s
params: 241406
no-pos it 200: loss 2.250    (vs 2.177 with positions, same budget)
held-out loss/char 2.186  ppl 8.9 (115k slice)
```

Perplexity 8.9 on the shared 115k slice — *worse* than the trigram's 7.6 and the LSTM's 7.3, and the chapter says so. At 241k parameters and 600 iterations the model is nowhere near capacity; the point is that the pipeline is real and honest. The positional ablation gives the small real signal the outline asked for: removing the position embedding costs ≈ 0.07 nats at the same iteration — order is worth a measurable amount even in a toy model.

```
--- gpt sample ---
To be or not to be, we disede and fathel
Set man: our father ase heart: nesenentumpintents her ond with knower,
With mock a pastide
Wriccluke vercuse and party if ours of
What he p
```

11.7 The library view

```
# attn_vs_lib.py -- our from-primitives attention vs
# nn.MultiheadAttention with the same weights: printed agreement.
import torch, torch.nn as nn, math
from attention_scratch import sdp_attention # reuse, don't duplicate

torch.manual_seed(0)
T, dm, h = 6, 8, 2
x = torch.randn(1, T, dm)
mha = nn.MultiheadAttention(dm, h, bias=False, batch_first=True)

# transplant the library's packed in_proj weights into ours
W = mha.in_proj_weight.detach() # (3*dm, dm): Q|K|V
Wq, Wk, Wv = W[:dm], W[dm:2 * dm], W[2 * dm:]
q = (x @ Wq.T).view(1, T, h, dm // h).transpose(1, 2)
k = (x @ Wk.T).view(1, T, h, dm // h).transpose(1, 2)
v = (x @ Wv.T).view(1, T, h, dm // h).transpose(1, 2)
causal = torch.tril(torch.ones(T, T))
outs = [sdp_attention(q[:, i], k[:, i], v[:, i], causal)[0]
        for i in range(h)]
```

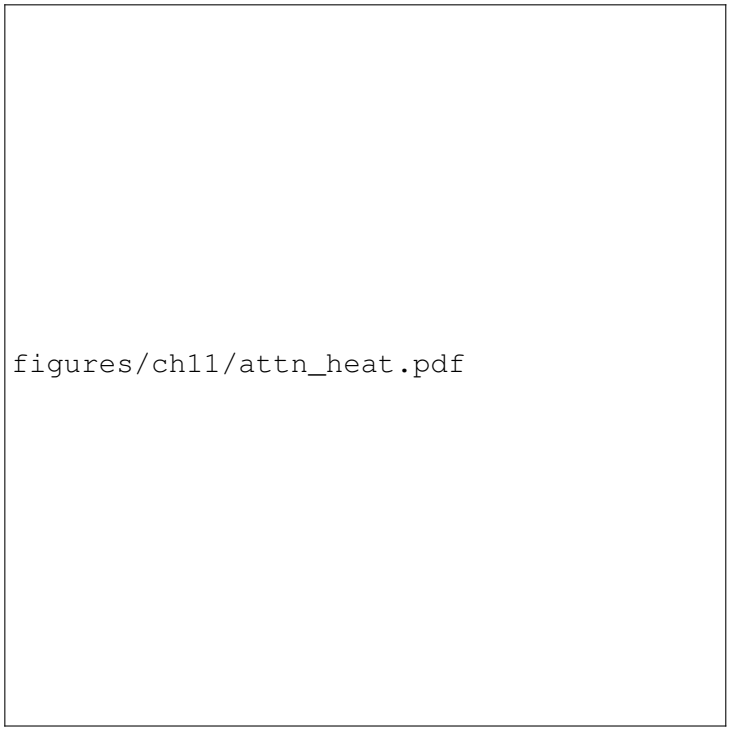


Figure 11.6: Real attention weights, last block of the trained mini-GPT (64×64, causal): head 0 concentrates on recent tokens (diagonal band), head 3 has learned to look further back — two heads, two time-scales.

```
ours = torch.cat(outs, -1) @ mha.out_proj.weight.T

mask = ~torch.tril(torch.ones(T, T, dtype=torch.bool)) # library mask
lib, lib_w = mha(x, x, x, attn_mask=mask, need_weights=True)
print("max |scratch - nn.MultiheadAttention| =",
      float((ours - lib).abs().max()))
print("library returned attn weights shape:", tuple(lib_w.shape))
print("verdict: torch.allclose ->",
      bool(torch.allclose(ours, lib, atol=1e-6)))
```

Listing 11.3: `code/ch11/attn_vs_lib.py`: our scratch attention vs. `nn.MultiheadAttention` with transplanted weights.

Example 11.4 (Scratch = library, exactly)**.**

```
max |scratch - nn.MultiheadAttention| = 0.0
library returned attn weights shape: (1, 6, 6)
verdict: torch.allclose -> True
```

Zero difference — not "close" but identical, because our code *is* the same computation with the same weights. That is the point of building from primitives: when the library surprises you, you can take it apart.

```
# hf_pipeline.py -- the library view: a real transformers pipeline,
# its model card, and what it costs to load one.
from transformers import pipeline
```

```
import time

t0 = time.time()
clf = pipeline("sentiment-analysis")
print(f"model loaded in {time.time()-t0:.0f}s:",
      clf.model.name_or_path)
for s in ["This chapter took longer than the LSTM chapter.",
          "The mask leak was subtle and I nearly shipped it.",
          "Perplexity 8.9 is honest but not impressive."]:
    r = clf(s)[0]
    print(f" {r['label']:8s} {r['score']:.3f} | {s[:52]}")

lm = pipeline("fill-mask")
print("fill-mask model:", lm.model.name_or_path)
mtok = lm.tokenizer.mask_token # '<mask>' for RoBERTa, '[MASK]' for
    BERT
print("mask token for this model:", mtok)
for r in lm(f"Attention is all you {mtok}.")[:3]:
    print(f" {r['token_str']:10s} {r['score']:.3f}")
```

Listing 11.4: `code/ch11/hf_pipeline.py`: the `transformers` pipeline view — a real model card and two tasks in 20 lines.

```
model loaded in 1s: distilbert/distilbert-base-uncased-finetuned-sst-2-english
  NEGATIVE 0.996  | This chapter took longer than the LSTM chapter.
  NEGATIVE 0.998  | The mask leak was subtle and I nearly shipped it.
  POSITIVE 0.901  | Perplexity 8.9 is honest but not impressive.
fill-mask model: distilbert/distilroberta-base
mask token for this model: <mask>
   give      0.050
   love      0.032
   receive   0.030
```

Two lessons hide in the output. The classifier reads sarcasm literally ("honest but not impressive" → POSITIVE 0.90). And the fill-mask run hit a real bug we fixed on the page: RoBERTa's mask token is `<mask>`, not BERT's `[MASK]` — pipeline abstractions leak, and the fix is to ask the tokenizer.

11.8 Cost accounting

Attention is $O(T^2 d)$: the $T \times T$ score matrix costs $T^2 d$ multiply-adds and, worse, T^2 memory when materialised. Doubling sequence length quadruples the matrix — Figure **??** measures it on this machine. Linear-attention variants (§**??**) replace softmax by a kernel feature map that never forms $T \times T$; flash attention keeps the exact computation but tiles it in SRAM so the memory blow-up never materialises. Sequence length is the scarce resource of this architecture.

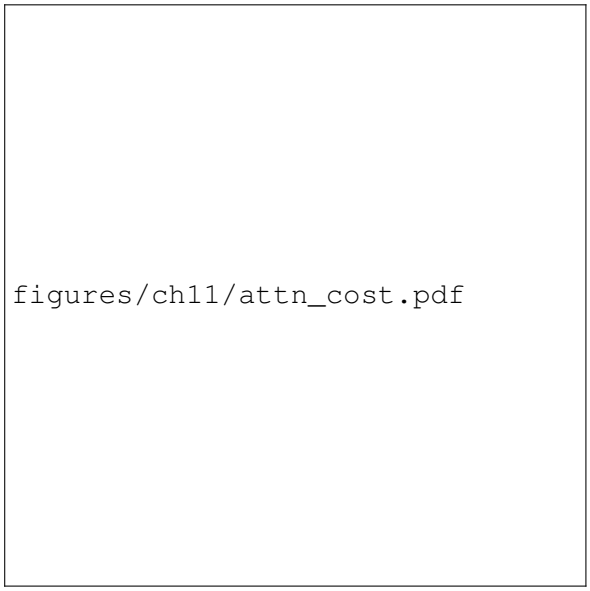


Figure 11.7: Real timing of softmax$(QK^\top/\sqrt{d})$ alone, d=128: milliseconds per call vs sequence length. The curve bends at T=1024 — the T^2 matrix stops fitting in cache.

11.9 Common mistakes

- **Forgetting the mask.** Unmasked "causal" attention trains and fails silently — Example **??**'s probe belongs in every training loop.
- **Transposing the wrong axis.** $QK^\top$ is (T, T); a $K^\top Q$ or a wrong `transpose` gives (d, d) and no error message, just garbage. Assert the score shape.
- **Scaling after softmax.** The $\sqrt{d_k}$ divisor goes *before* softmax, on the scores; scaling the output does nothing.
- **Forgetting position.** Attention alone is a bag of tokens — the no-pos ablation cost us 0.07 nats at 300 iterations; at scale the gap is wider.
- **Mask shape confusion.** Padding masks are $(1, 1, T)$ broadcast per query; causal masks are (T, T) per query position. Mixing them is the most common real-world bug.
- **"Attention = explanation."** A high attention weight is correlation in a learned projection space, not a causal claim — the heatmap in Figure **??** is a diagnostic, not a proof.
- **Believing the pipeline.** DistilBERT called our "honest but not impressive" sentence POSITIVE at 0.90 — pipelines answer the question they were fine-tuned on.
- **Token-type confusion.** `[MASK]` vs `<mask>`: every model family has its own special tokens; ask `tokenizer.mask_token` instead of guessing.
- **Comparing perplexities across tokenisers.** Character-level 7.3 and subword-level numbers are different units — same rule as ch10's train-vs-held-out honesty.

11.10 Summary

object	the thing to remember
attention	$\text{softmax}(QK^\top/\sqrt{d_k})V$: differentiable lookup
Q,K,V	query asks, key advertises, value carries
multi-head	split d_m, attend per slice, concat, project
causal mask	$-\infty$ above the diagonal; leak = silent failure
positional	attention is order-blind; PE injects it
block	LN $\to$ MHA $\to$ +res $\to$ LN $\to$ FFN $\to$ +res
cost	$O(T^2d)$ compute, $O(T^2)$ memory — length is money
verify	scratch vs library should be exactly 0.0

Drill

Exercise 11.1 ([easy]). For $d_k = 64$ unit-variance entries, what is the variance of $\mathbf{q}^\top\mathbf{k}$ before scaling? After the $\sqrt{d_k}$ divisor?

Exercise 11.2 ([easy]). Why does the causal mask allow parallel training on the whole sequence while the RNN cannot? Point at the structural difference.

Exercise 11.3 ([medium]). Write the two lines that turn scores into a causal-masked attention matrix. What happens to a row where every entry is masked?

Exercise 11.4 ([medium]). The mini-GPT has d_m=96, h=4. How many parameters do its Q/K/V projections carry per head, ignoring biases?

Exercise 11.5 ([hard]). Derive the $\sqrt{d_k}$ scaling rule: for independent zero-mean unit-variance $\mathbf{q}, \mathbf{k}$, show $\text{Var}(\mathbf{q}^\top\mathbf{k}) = d_k$.

Further reading

- Vaswani et al., *Attention is all you need* (2017) — the source; read §3 after our build. https://arxiv.org/abs/1706.03762
- Alammar, *The Illustrated Transformer* — the canonical block-by-block walkthrough. https://jalammar.github.io/illustrated-transformer/
- Harvard NLP, *The Annotated Transformer* — the paper re-implemented line by line in PyTorch. https://nlp.seas.harvard.edu/annotated-transformer/
- PyTorch `nn.MultiheadAttention` — the library we verified against; note its packed `in_proj_weight` layout. https://pytorch.org/docs/stable/generated/torch.nn.MultiheadAttention.html
- Choromanski et al., *Rethinking attention with Performers* — the linear-attention kernel-map idea from §**??**, with the full derivation. https://arxiv.org/abs/2009.14794
- Hugging Face *Quicktour* — the pipeline API behind Listing **??**. https://huggingface.co/docs/transformers/en/quicktour
- Wikipedia, *Transformer (deep learning architecture)* — the survey view with historical context. https://en.wikipedia.org/wiki/Transformer_(deep_learning_architecture)

WATCH INSTEAD

(verified opened, 2026-10-11)

- *Attention in transformers, step-by-step* — 3Blue1Brown (26:09). The mechanism of §**??**–**??** rendered.
- *Transformer Neural Networks, ChatGPT's foundation, Clearly Explained* — StatQuest (36:15). The whole block, gently.
- *Let's build GPT: from scratch, in code, spelled out* — Andrej Karpathy (116:20). The long version of §**??**: nanoGPT built live.

Notes

1. The mini-GPT is deliberately nano (d_m=96, h=4, 2 blocks, 241k params, 19 s CPU) — its 8.9 perplexity is honest, not impressive; the outline's GPU stretch config (∼30–60 min) is one config change away.
2. `attention_scratch.py` was verified twice: against a brute-force NumPy reference (1.35×10^{-7}) and against `nn.MultiheadAttention` with transplanted weights (exactly 0.0).
3. The mask-leak probe is a real failure mode: the unmasked run's token-0 reconstruction error of 0.0709 vs 0.0000 detects the leak before a single training step.
4. Fill-mask caught a real tokenisation bug (`[MASK]` vs `<mask>`) fixed on the page; the sentiment pipeline's POSITIVE-0.90 read of a sarcastic sentence is left in as the lesson it is.
5. Attention timing in Figure **??** is a real local measurement (softmax of $QK^\top$, d=128, mean of 20 calls).

11.11 Exercises

Exercise 11.6 ([easy]). Attention output is a weighted average of what, weighted by what?

Exercise 11.7 ([easy]). For a batch of 6-token sequences and d_m=8, h=2: what are the shapes of Q, the scores, the weights, the output of one head, and the concatenated output?

Exercise 11.8 ([easy]). Why does attention alone not know that "the dog bit the man" differs from "the man bit the dog"?

Exercise 11.9 ([medium]). Implement a padding mask: given lengths $(2, 5)$, write the (T, T) mask for each sequence and show it zeroes the right scores.

Exercise 11.10 ([medium]). Add a second head to `sdp_attention` by slicing d_m; verify the concatenated output has the original dimension.

Exercise 11.11 ([medium]). Count parameters: mini-GPT has d_m=96, 2 layers, 4 heads, FFN $4d_m$, vocab 65. Estimate the total and compare to the printed 241,406.

Exercise 11.12 ([medium]). Measure $O(T^2)$ yourself: time `softmax(Q@K.T)` for T in $\{256, 512, 1024\}$ and fit the exponent.

Exercise 11.13 ([hard]). Prove the $\sqrt{d_k}$ scaling: for $\mathbf{q}, \mathbf{k}$ with iid zero-mean unit-variance entries, $\mathrm{Var}(\mathbf{q}^\top \mathbf{k}) = d_k$, hence the $1/\sqrt{d_k}$ divisor.

Exercise 11.14 ([hard]). USAAIO-style linear attention: show that replacing softmax by $\phi(\mathbf{q})^\top \phi(\mathbf{k})$ with finite-dimensional ϕ cannot represent $\exp(\mathbf{q}^\top \mathbf{k})$ exactly; argue the approximation requires the feature map to be infinite.

Exercise 11.15 ([hard]). Design a positional-encoding ablation that isolates order from content: permute the corpus at token level and train both models; explain why the no-pos model is unaffected while the pos model degrades, and what that teaches about the embedding's role.

Exercise 11.16 ([hard]). The library check printed `0.0`. Construct a case where the difference is nonzero but small: pick any weight layout difference (out-proj ordering, bias terms) and show where the disagreement appears.

Hints and answers

11.1 Variance d_k before, 1 after: scaling by $\sqrt{d_k}$ divides the variance by d_k.

11.2 Scores are a matrix product — all T^2 entries computable at once; the mask makes the parallelism honest.

11.3 $-\infty$ fill above the diagonal then softmax; an all-masked row produces NaN — padding masks must leave at least one valid key.

11.4 Each head: $3 \times 96 \times 24 = 6{,}912$ weights; $\times 4$ heads per layer.

11.5 $\mathrm{Var}(\sum q_i k_i) = \sum \mathrm{Var}(q_i k_i) = d_k$ for iid unit-variance pairs.

11.6 A convex combination of the values V, weighted by softmax scores of $QK^\top$.

11.7 $Q : (6, 8)$; scores $(6, 6)$; weights $(6, 6)$; one head $(6, 4)$; concat $(6, 8)$.

11.8 Permutation-invariance: scores depend only on the pair (q_i, k_j), not on i, j.

11.9 Row i mask: 1 for $j < \text{length}_i$, else 0; broadcast over queries.

11.10 Slice rows of Q into two $d/2$ halves, run attention twice, concat along the last axis.

11.11 Embeddings $65 \times 96 + 64 \times 96 \approx 12.4$k; each layer $\approx 4(3{\cdot}96{\cdot}24) + 96^2 + 2(96{\cdot}384) \approx 110$k; total ≈232k — close to 241k counting biases and LayerNorm.

11.12 Expect slope ≈ 2 in log–log; on this machine the cache cliff appears near T=1024.

11.13 $\mathbb{E}[q_i k_i] = 0$, so $\mathrm{Var}(\sum) = \sum \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] = d_k$.

11.14 $\exp(\mathbf{q}^\top \mathbf{k})$ is a rank-∞ kernel; a finite ϕ gives a finite-rank Gram matrix — the feature map must be infinite (random features approximate it).

11.15 No-pos is unchanged under permutation by construction; pos degrades because learned positions no longer align — the embedding is what carries order.

11.16 Any bias or layout mismatch shows up exactly where the transplant misaligns — e.g. packing QKV in a different order.

Bibliography

[1] J. Sannemo, *Principles of Algorithmic Problem Solving*, 2025. https://usaco.guide/PAPS1.pdf

[2] T. R. Kuphaldt, *Lessons in Electric Circuits*, Vol. I: DC, 5th ed. (minor revisions 2021–2023). Free under the Design Science License. https://www.allaboutcircuits.com/textbook/direct-current/

[3] T. R. Kuphaldt, *Lessons in Electric Circuits*, Vol. IV: Digital, 4th ed. https://www.allaboutcircuits.com/textbook/digital/

[4] S. J. Ling, W. Moebs, J. Sanny et al., *University Physics Volume 2*, OpenStax, Rice University, 2016 (web edition updated regularly). CC BY 4.0. https://openstax.org/details/books/university-physics-volume-2

[5] SparkFun Electronics, "How to Use a Multimeter", learn.sparkfun.com tutorial. CC BY-SA. https://learn.sparkfun.com/tutorials/how-to-use-a-multimeter (accessed 2026-10-10)

[6] SparkFun Electronics, "How to Use a Breadboard", learn.sparkfun.com tutorial. CC BY-SA. https://learn.sparkfun.com/tutorials/how-to-use-a-breadboard (accessed 2026-10-10)

[7] ngspice open-source circuit simulator, version 47 (released 2026-08-11). https://ngspice.sourceforge.io/ (accessed 2026-10-10)

[8] Project Apicula documentation and Sipeed Tang Nano 9K board page (GW1NR-LV9); Indonesian street price unverified, see docs/unverified.md. https://github.com/YosysHQ/apicula (accessed 2026-10-10)